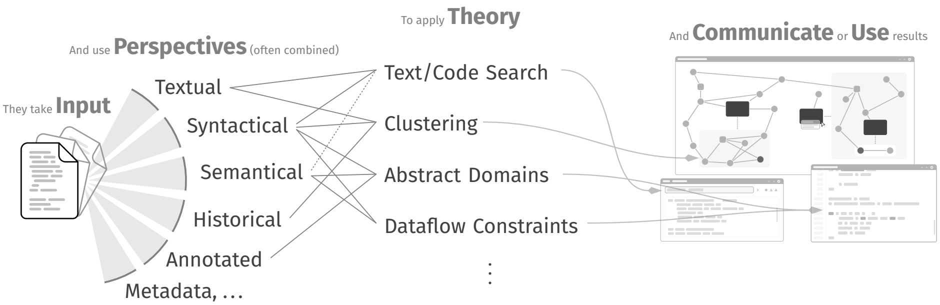


Static Analysis in the Real World

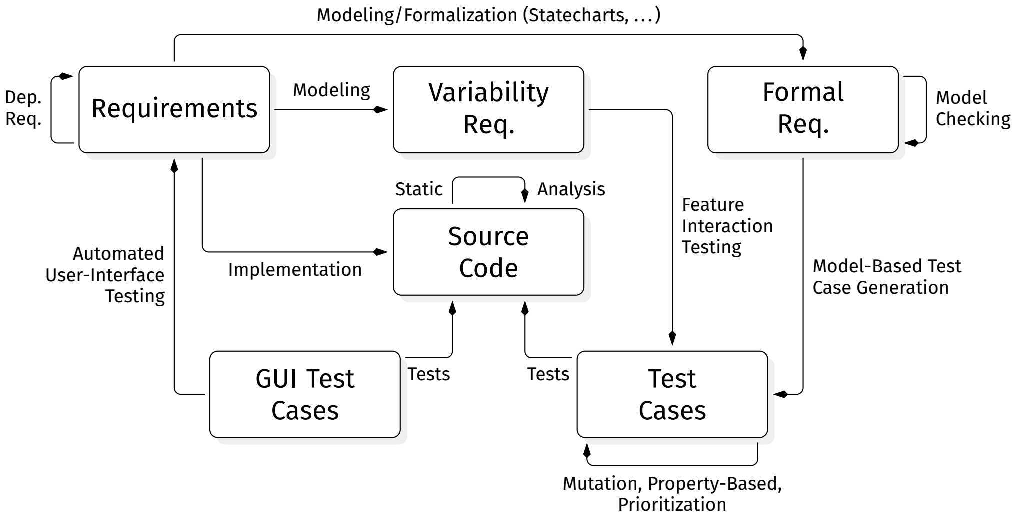
Software Quality Assurance — Static Code Analysis, III | Florian Sihler | December 17, 2025

Software Engineering
Programming Languages

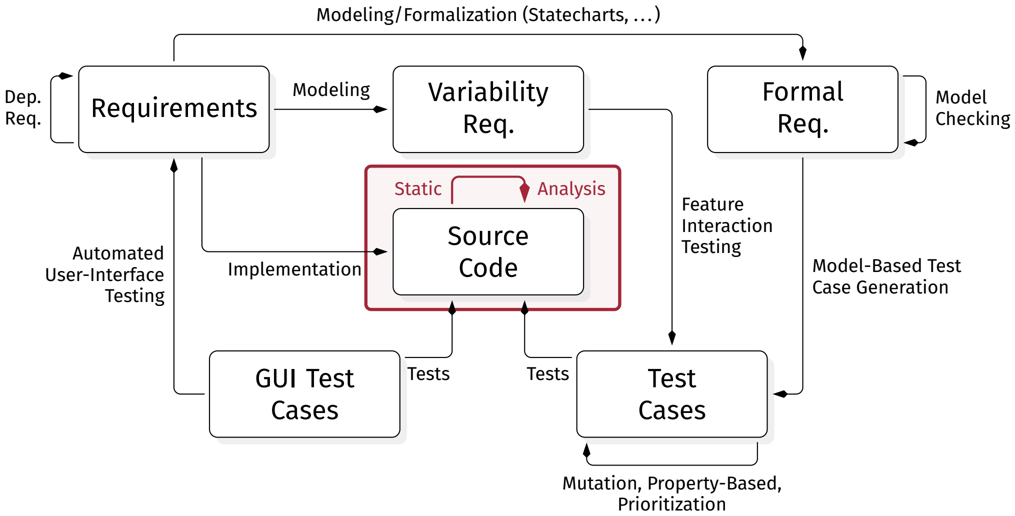
universität ulm

1. A Small Recap

Embedding a Landscape



Embedding a Landscape



What is static analysis?

What is static analysis?

Discover *syntactic/semantic properties* of programs **without** running them.[RY20]

What is static analysis?

Discover *syntactic/semantic properties* of programs **without** running them.[RY20]

What is abstract interpretation (in this context)?

What is static analysis?

Discover *syntactic/semantic properties* of programs **without** running them.[RY20]

What is abstract interpretation (in this context)?

A theory and framework to *systematically* derive sound static analyses by **abstracting** program semantics.[Cou21]

What is static analysis?

Discover *syntactic/semantic properties* of programs **without** running them.[RY20]

What is abstract interpretation (in this context)?

A theory and framework to *systematically* derive sound static analyses by **abstracting** program semantics.[Cou21]

What is static analysis?

Discover *syntactic/semantic properties* of programs **without** running them.[RY20]

What is abstract interpretation (in this context)?

A theory and framework to *systematically* derive sound static analyses by **abstracting** program semantics.[Cou21]

A way to soundly over-approximate all possible program behaviors.

On the Bread and Butter of Static Analysis

On the Bread and Butter of Static Analysis

1. What are examples for semantic properties?

On the Bread and Butter of Static Analysis

1. What are examples for semantic properties?

2. How do we describe semantic properties of programs?

On the Bread and Butter of Static Analysis

1. What are examples for semantic properties?

2. How do we describe semantic properties of programs?

3. What is a (complete) lattice, why do we need it?

On the Bread and Butter of Static Analysis

1. What are examples for semantic properties?

2. How do we describe semantic properties of programs?

3. What is a (complete) lattice, why do we need it?

4. What is described by a Galois connection?

On the Bread and Butter of Static Analysis

1. What are examples for semantic properties?

2. How do we describe semantic properties of programs?

3. What is a (complete) lattice, why do we need it?

4. What is described by a Galois connection?

5. Given a closed interval domain $[l \mathrel{..} u]$, what are the abstract semantics of addition?

On the Bread and Butter of Static Analysis

1. What are examples for semantic properties?
   "We never dereference null pointers", "variable x is always positive", "array a is sorted at a given point", "we never divide by zero", …

2. How do we describe semantic properties of programs?

3. What is a (complete) lattice, why do we need it?

4. What is described by a Galois connection?

5. Given a closed interval domain $[l \mathinner{..} u]$, what are the abstract semantics of addition?

On the Bread and Butter of Static Analysis

1. What are examples for semantic properties?
   "We never dereference null pointers", "variable x is always positive", "array a is sorted at a given point", "we never divide by zero", …

2. How do we describe semantic properties of programs?
   We use (po-)sets containing all program states that satisfy the property.

3. What is a (complete) lattice, why do we need it?

4. What is described by a Galois connection?

5. Given a closed interval domain $[l\,..\,u]$, what are the abstract semantics of addition?

On the Bread and Butter of Static Analysis

1. What are examples for semantic properties?
   “We never dereference null pointers”, “variable x is always positive”, “array a is sorted at a given point”, “we never divide by zero”, …

2. How do we describe semantic properties of programs?
   We use (po-)sets containing all program states that satisfy the property.

3. What is a (complete) lattice, why do we need it?
   A poset, in which all subsets have a least upper bound (join), a greatest lower bound (meet), as well as a top and bottom element.

4. What is described by a Galois connection?

5. Given a closed interval domain $[l \mathrel{..} u]$, what are the abstract semantics of addition?

On the Bread and Butter of Static Analysis

1. What are examples for semantic properties?
   “We never dereference null pointers”, “variable x is always positive”, “array a is sorted at a given point”, “we never divide by zero”, ...

2. How do we describe semantic properties of programs?
   We use (po-)sets containing all program states that satisfy the property.

3. What is a (complete) lattice, why do we need it?
   A poset, in which all subsets have a least upper bound (join), a greatest lower bound (meet), as well as a top and bottom element. We need lattices to describe how to combine properties (e.g., at control-flow joins) and to ensure that our analysis terminates.

4. What is described by a Galois connection?

5. Given a closed interval domain $[l \mathinner{..} u]$, what are the abstract semantics of addition?

On the Bread and Butter of Static Analysis

1. What are examples for semantic properties?
   "We never dereference null pointers", "variable x is always positive", "array a is sorted at a given point", "we never divide by zero", …

2. How do we describe semantic properties of programs?
   We use (po-)sets containing all program states that satisfy the property.

3. What is a (complete) lattice, why do we need it?
   A poset, in which all subsets have a least upper bound (join), a greatest lower bound (meet), as well as a top and bottom element. We need lattices to describe how to combine properties (e.g., at control-flow joins) and to ensure that our analysis terminates.

4. What is described by a Galois connection?
   Mapping concrete program states to abstract properties (abstraction, α) and back (concretization, γ).

5. Given a closed interval domain $[l \mathinner{..} u]$, what are the abstract semantics of addition?

On the Bread and Butter of Static Analysis

1. What are examples for semantic properties?
   "We never dereference null pointers", "variable x is always positive", "array a is sorted at a given point", "we never divide by zero", ...

2. How do we describe semantic properties of programs?
   We use (po-)sets containing all program states that satisfy the property.

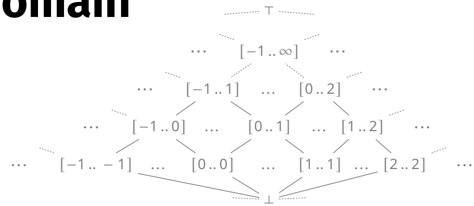
3. What is a (complete) lattice, why do we need it?
   A poset, in which all subsets have a least upper bound (join), a greatest lower bound (meet), as well as a top and bottom element. We need lattices to describe how to combine properties (e.g., at control-flow joins) and to ensure that our analysis terminates.

4. What is described by a Galois connection?
   Mapping concrete program states to abstract properties (abstraction, α) and back (concretization, γ).

5. Given a closed interval domain $[l \mathinner{..} u]$, what are the abstract semantics of addition?
   $\mathbb{E}[\![\ a\ +\ b\ ]\!]\rho \overset{\text{def}}{=} [l_a + l_b \mathinner{..} u_a + u_b] \qquad (a \in [l_a \mathinner{..} u_a], b \in [l_b \mathinner{..} u_b])$
   "We add the lower bounds and the upper bounds, assuming the environment ρ."
   (assuming a language like Java and ignoring overflows)

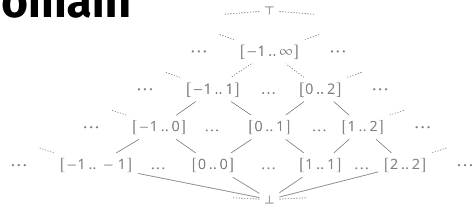
Recap: Applying the Interval Domain

Recap: Applying the Interval Domain



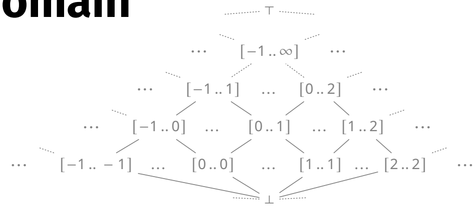
Recap: Applying the Interval Domain

6. Do we need widening (∇) for this domain?

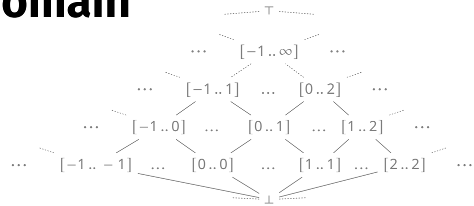


Recap: Applying the Interval Domain

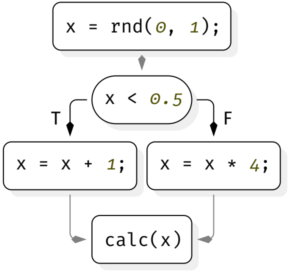
6. Do we need widening (∇) for this domain?
   Yes, because the lattice contains infinite ascending chains,
   e.g., $[0\,..\,0] \sqsubseteq [0\,..\,1] \sqsubseteq [0\,..\,2] \sqsubseteq \dots$



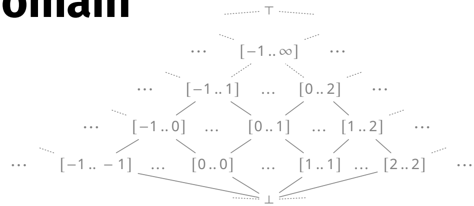
Recap: Applying the Interval Domain



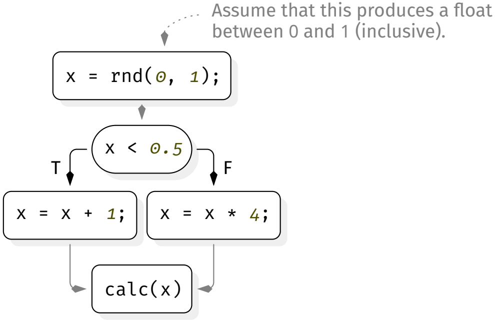
6. Do we need widening (∇) for this domain?
   Yes, because the lattice contains infinite ascending chains,
   e.g., $[0 \mathinner{..} 0] \sqsubseteq [0 \mathinner{..} 1] \sqsubseteq [0 \mathinner{..} 2] \sqsubseteq \ldots$



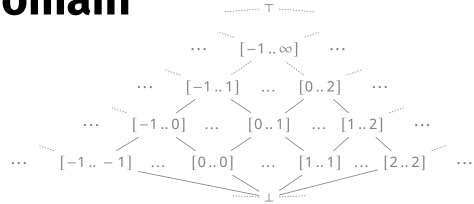
Recap: Applying the Interval Domain



6. Do we need widening (∇) for this domain?
   Yes, because the lattice contains infinite ascending chains,
   e.g., $[0\,..\,0] \sqsubseteq [0\,..\,1] \sqsubseteq [0\,..\,2] \sqsubseteq \ldots$

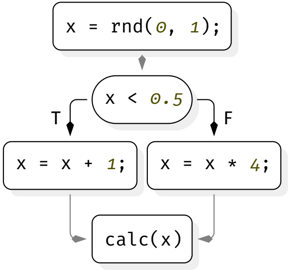


Recap: Applying the Interval Domain



6. Do we need widening (∇) for this domain?
   Yes, because the lattice contains infinite ascending chains, e.g., $[0\,..\,0] \sqsubseteq [0\,..\,1] \sqsubseteq [0\,..\,2] \sqsubseteq \ldots$

Assume that this produces a float between 0 and 1 (inclusive).

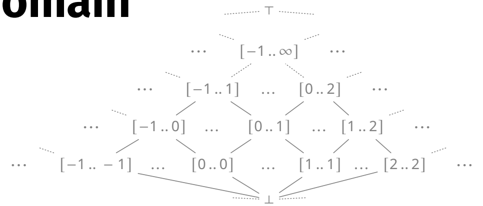


We use these semantics (use your intuition for the rest):

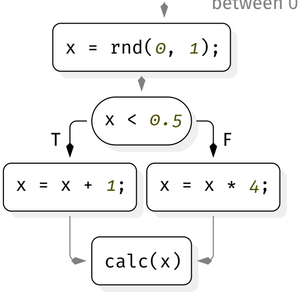
$$\mathbb{E}[\![\, \texttt{if(c) a else b} \,]\!]\rho \overset{\text{def}}{=} \mathbb{E}[\![\, \texttt{a} \,]\!](\mathbb{C}[\![\, \texttt{c} \,]\!]\rho) \sqcup \mathbb{E}[\![\, \texttt{b} \,]\!](\mathbb{C}[\![\, \neg\texttt{c} \,]\!]\rho)$$

Recap: Applying the Interval Domain

6. Do we need widening (∇) for this domain?
   Yes, because the lattice contains infinite ascending chains,
   e.g., $[0\,..\,0] \sqsubseteq [0\,..\,1] \sqsubseteq [0\,..\,2] \sqsubseteq \ldots$



Assume that this produces a float between 0 and 1 (inclusive).

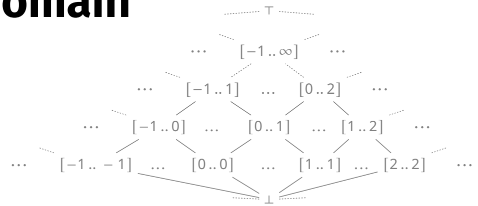


We use these semantics (use your intuition for the rest):

$$\mathbb{E}[\![\, \texttt{if(c) a else b} \,]\!]\rho \overset{\text{def}}{=} \mathbb{E}[\![\, \texttt{a} \,]\!](\mathbb{C}[\![\, \texttt{c} \,]\!]\rho) \sqcup \mathbb{E}[\![\, \texttt{b} \,]\!](\mathbb{C}[\![\, \neg\texttt{c} \,]\!]\rho)$$

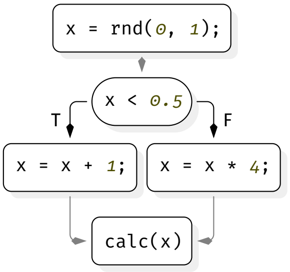
"Evaluate a assuming that c is true, evaluate b assuming that c is false, and join the results."

Recap: Applying the Interval Domain



6. Do we need widening (∇) for this domain?
   Yes, because the lattice contains infinite ascending chains,
   e.g., $[0\,..\,0] \sqsubseteq [0\,..\,1] \sqsubseteq [0\,..\,2] \sqsubseteq \dots$

Assume that this produces a float between 0 and 1 (inclusive).



We use these semantics (use your intuition for the rest):

$\mathbb{E}[\![\, \texttt{if(c) a else b} \,]\!]\rho \overset{\text{def}}{=} \mathbb{E}[\![\, \texttt{a} \,]\!](\mathbb{C}[\![\, \texttt{c} \,]\!]\rho) \sqcup \mathbb{E}[\![\, \texttt{b} \,]\!](\mathbb{C}[\![\, \neg\texttt{c} \,]\!]\rho)$

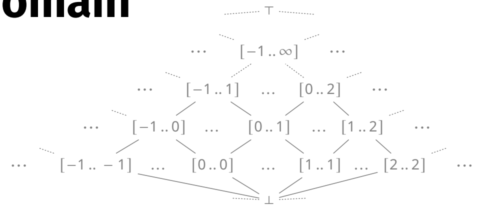
"Evaluate a assuming that c is true, evaluate b assuming that c is false, and join the results. "

$\mathbb{E}[\![\, \texttt{a + b} \,]\!]\rho \overset{\text{def}}{=} [l_a + l_b \,..\, u_a + u_b]$

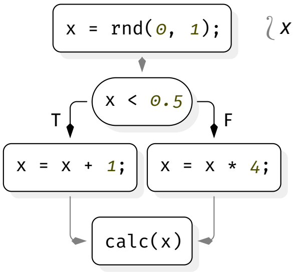
$\mathbb{E}[\![\, \texttt{a * b} \,]\!]\rho \overset{\text{def}}{=} [\min S \,..\, \max S]$ with $S = \{\, l_a l_b, l_a u_b, u_a l_b, u_a u_b \,\}$
$(\texttt{a} \in [l_a \,..\, u_a], \texttt{b} \in [l_b \,..\, u_b])$

Recap: Applying the Interval Domain

6. Do we need widening (∇) for this domain?
   Yes, because the lattice contains infinite ascending chains, e.g., $[0\,..\,0] \sqsubseteq [0\,..\,1] \sqsubseteq [0\,..\,2] \sqsubseteq \dots$



Assume that this produces a float between 0 and 1 (inclusive).



$\wr\, x \in [0\,..\,1] \,\wr$

We use these semantics (use your intuition for the rest):

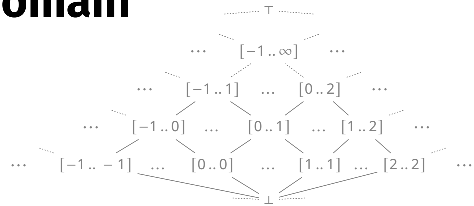
$\mathbb{E}[\![\, \texttt{if(c) a else b} \,]\!]\rho \overset{\text{def}}{=} \mathbb{E}[\![\, \texttt{a} \,]\!](\mathbb{C}[\![\, \texttt{c} \,]\!]\rho) \sqcup \mathbb{E}[\![\, \texttt{b} \,]\!](\mathbb{C}[\![\, \neg\texttt{c} \,]\!]\rho)$

"Evaluate a assuming that c is true, evaluate b assuming that c is false, and join the results. "

$\mathbb{E}[\![\, \texttt{a + b} \,]\!]\rho \overset{\text{def}}{=} [l_a + l_b\,..\,u_a + u_b]$

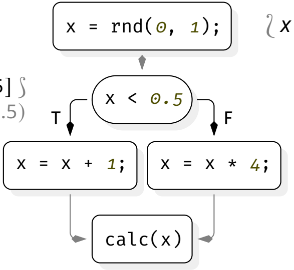
$\mathbb{E}[\![\, \texttt{a * b} \,]\!]\rho \overset{\text{def}}{=} [\min S\,..\,\max S]$ with $S = \{\, l_a l_b, l_a u_b, u_a l_b, u_a u_b \,\}$
$(\texttt{a} \in [l_a\,..\,u_a], \texttt{b} \in [l_b\,..\,u_b])$

Recap: Applying the Interval Domain



6. Do we need widening (∇) for this domain?
   Yes, because the lattice contains infinite ascending chains,
   e.g., $[0\,..\,0] \sqsubseteq [0\,..\,1] \sqsubseteq [0\,..\,2] \sqsubseteq \dots$

Assume that this produces a float between 0 and 1 (inclusive).



$\wr\, x \in [0\,..\,1]\, \wr$

$\wr$ [in T] $x \in [0\,..\,0.5]$ $\wr$
$[0\,..\,1] \cap [0\,..\,0.5)$

We use these semantics (use your intuition for the rest):

$\mathbb{E}[\![\ \mathtt{if(c)\ a\ else\ b}\ ]\!]\rho \overset{\text{def}}{=} \mathbb{E}[\![\ \mathtt{a}\ ]\!](\mathbb{C}[\![\ \mathtt{c}\ ]\!]\rho) \sqcup \mathbb{E}[\![\ \mathtt{b}\ ]\!](\mathbb{C}[\![\ \neg\mathtt{c}\ ]\!]\rho)$

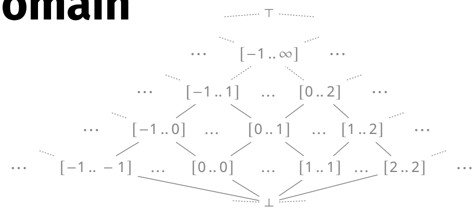
"Evaluate a assuming that c is true, evaluate b assuming that c is false, and join the results. "

$\mathbb{E}[\![\ \mathtt{a\ +\ b}\ ]\!]\rho \overset{\text{def}}{=} [l_a + l_b\,..\,u_a + u_b]$

$\mathbb{E}[\![\ \mathtt{a\ *\ b}\ ]\!]\rho \overset{\text{def}}{=} [\min S\,..\,\max S]$ with $S = \{\, l_a l_b, l_a u_b, u_a l_b, u_a u_b \,\}$
$(\mathtt{a} \in [l_a\,..\,u_a], \mathtt{b} \in [l_b\,..\,u_b])$

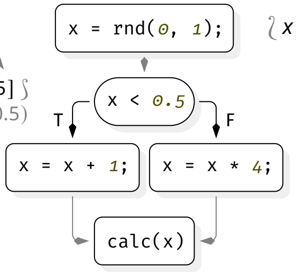
Recap: Applying the Interval Domain

6. Do we need widening (∇) for this domain?
   Yes, because the lattice contains infinite ascending chains,
   e.g., $[0\mathinner{..}0] \sqsubseteq [0\mathinner{..}1] \sqsubseteq [0\mathinner{..}2] \sqsubseteq \dots$



Assume that this produces a float between 0 and 1 (inclusive).

Over-Approx. to ignore the 0.499… problem



$\wr\, x \in [0\mathinner{..}1]\, \wr$

$\wr$ [in T] $x \in [0\mathinner{..}0.5]$ $\wr$
$[0\mathinner{..}1] \cap [0\mathinner{..}0.5)$

We use these semantics (use your intuition for the rest):

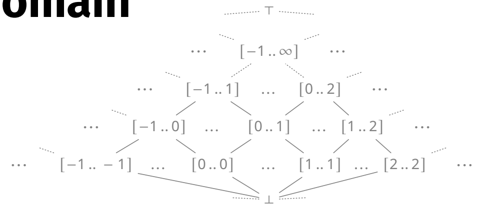
$\mathbb{E}[\![\, \texttt{if(c) a else b} \,]\!]\rho \overset{\text{def}}{=} \mathbb{E}[\![\, \texttt{a} \,]\!](\mathbb{C}[\![\, \texttt{c} \,]\!]\rho) \sqcup \mathbb{E}[\![\, \texttt{b} \,]\!](\mathbb{C}[\![\, \neg\texttt{c} \,]\!]\rho)$

"Evaluate a assuming that c is true, evaluate b assuming that c is false, and join the results. "

$\mathbb{E}[\![\, \texttt{a + b} \,]\!]\rho \overset{\text{def}}{=} [l_a + l_b \mathinner{..} u_a + u_b]$

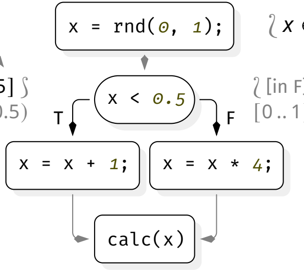
$\mathbb{E}[\![\, \texttt{a * b} \,]\!]\rho \overset{\text{def}}{=} [\min S \mathinner{..} \max S]$ with $S = \{\, l_a l_b, l_a u_b, u_a l_b, u_a u_b \,\}$
$(\texttt{a} \in [l_a \mathinner{..} u_a], \texttt{b} \in [l_b \mathinner{..} u_b])$

Recap: Applying the Interval Domain



6. Do we need widening (∇) for this domain?
   Yes, because the lattice contains infinite ascending chains, e.g., $[0\,..\,0] \sqsubseteq [0\,..\,1] \sqsubseteq [0\,..\,2] \sqsubseteq \dots$

Assume that this produces a float between 0 and 1 (inclusive).



$\wr\, x \in [0\,..\,1] \,\wr$

Over-Approx. to ignore the 0.499… problem

$\wr$ [in T] $x \in [0\,..\,0.5] \,\wr$
$[0\,..\,1] \cap [0\,..\,0.5)$

$\wr$ [in F] $x \in [0.5\,..\,1] \,\wr$
$[0\,..\,1] \cap [0.5\,..\,1]$

We use these semantics (use your intuition for the rest):

$\mathbb{E}[\![\, \texttt{if(c) a else b} \,]\!]\rho \overset{\text{def}}{=} \mathbb{E}[\![\, \texttt{a} \,]\!](\mathbb{C}[\![\, \texttt{c} \,]\!]\rho) \sqcup \mathbb{E}[\![\, \texttt{b} \,]\!](\mathbb{C}[\![\, \neg\texttt{c} \,]\!]\rho)$

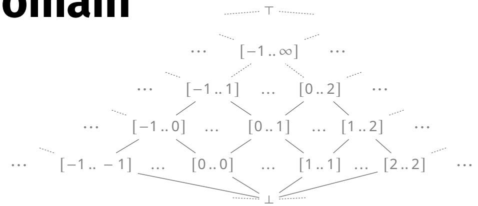
"Evaluate a assuming that c is true, evaluate b assuming that c is false, and join the results. "

$\mathbb{E}[\![\, \texttt{a + b} \,]\!]\rho \overset{\text{def}}{=} [l_a + l_b \,..\, u_a + u_b]$

$\mathbb{E}[\![\, \texttt{a * b} \,]\!]\rho \overset{\text{def}}{=} [\min S \,..\, \max S]$ with $S = \{\, l_a l_b, l_a u_b, u_a l_b, u_a u_b \,\}$
$(\texttt{a} \in [l_a \,..\, u_a], \texttt{b} \in [l_b \,..\, u_b])$

Recap: Applying the Interval Domain

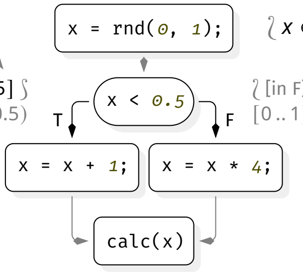
6. Do we need widening (∇) for this domain?
   Yes, because the lattice contains infinite ascending chains,
   e.g., $[0\,..\,0] \sqsubseteq [0\,..\,1] \sqsubseteq [0\,..\,2] \sqsubseteq \dots$



Assume that this produces a float between 0 and 1 (inclusive).

Over-Approx. to ignore the 0.499… problem

$\wr$ [in T] $x \in [0\,..\,0.5]$ $\int$
$[0\,..\,1] \cap [0\,..\,0.5)$



$\wr\ x \in [0\,..\,1]\ \int$

$\wr$ [in F] $x \in [0.5\,..\,1]$ $\int$
$[0\,..\,1] \cap [0.5\,..\,1]$

$\wr\ x \in [1\,..\,1.5]\ \int$

We use these semantics (use your intuition for the rest):

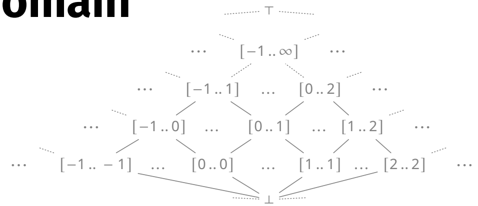
$\mathbb{E}[\![\ \texttt{if(c) a else b}\ ]\!]\rho \overset{\text{def}}{=} \mathbb{E}[\![\ \texttt{a}\ ]\!](\mathbb{C}[\![\ \texttt{c}\ ]\!]\rho) \sqcup \mathbb{E}[\![\ \texttt{b}\ ]\!](\mathbb{C}[\![\ \neg\texttt{c}\ ]\!]\rho)$

"Evaluate a assuming that c is true, evaluate b assuming that c is false, and join the results. "

$\mathbb{E}[\![\ \texttt{a + b}\ ]\!]\rho \overset{\text{def}}{=} [l_a + l_b\,..\,u_a + u_b]$

$\mathbb{E}[\![\ \texttt{a * b}\ ]\!]\rho \overset{\text{def}}{=} [\min S\,..\,\max S]$ with $S = \{\, l_a l_b, l_a u_b, u_a l_b, u_a u_b \,\}$
$(\texttt{a} \in [l_a\,..\,u_a], \texttt{b} \in [l_b\,..\,u_b])$

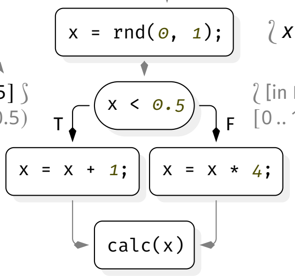
Recap: Applying the Interval Domain



6. Do we need widening (∇) for this domain?
   Yes, because the lattice contains infinite ascending chains, e.g., $[0 \,..\, 0] \sqsubseteq [0 \,..\, 1] \sqsubseteq [0 \,..\, 2] \sqsubseteq \dots$

Assume that this produces a float between 0 and 1 (inclusive).

Over-Approx. to ignore the 0.499… problem



$\wr\, x \in [0 \,..\, 1] \,\wr$

$\wr$ [in T] $x \in [0 \,..\, 0.5] \,\wr$
$[0 \,..\, 1] \cap [0 \,..\, 0.5)$

$\wr$ [in F] $x \in [0.5 \,..\, 1] \,\wr$
$[0 \,..\, 1] \cap [0.5 \,..\, 1]$

$\wr\, x \in [1 \,..\, 1.5] \,\wr$

$\wr\, x \in [2 \,..\, 4] \,\wr$

We use these semantics (use your intuition for the rest):

$\mathbb{E}[\![\texttt{if(c) a else b}]\!]\rho \overset{\text{def}}{=} \mathbb{E}[\![\texttt{a}]\!](\mathbb{C}[\![\texttt{c}]\!]\rho) \sqcup \mathbb{E}[\![\texttt{b}]\!](\mathbb{C}[\![\neg\texttt{c}]\!]\rho)$

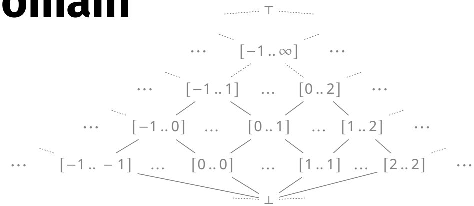
"Evaluate a assuming that c is true, evaluate b assuming that c is false, and join the results. "

$\mathbb{E}[\![\texttt{a + b}]\!]\rho \overset{\text{def}}{=} [l_a + l_b \,..\, u_a + u_b]$

$\mathbb{E}[\![\texttt{a * b}]\!]\rho \overset{\text{def}}{=} [\min S \,..\, \max S]$ with $S = \{\, l_a l_b, l_a u_b, u_a l_b, u_a u_b \,\}$
$(\texttt{a} \in [l_a \,..\, u_a], \texttt{b} \in [l_b \,..\, u_b])$

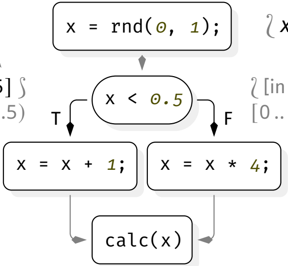
Recap: Applying the Interval Domain

6. Do we need widening (∇) for this domain?
   Yes, because the lattice contains infinite ascending chains,
   e.g., $[0\,..\,0] \sqsubseteq [0\,..\,1] \sqsubseteq [0\,..\,2] \sqsubseteq \dots$



Assume that this produces a float between 0 and 1 (inclusive).

Over-Approx. to ignore the 0.499… problem



≀ $x \in [0\,..\,1]$ ≀

≀ [in T] $x \in [0\,..\,0.5]$ ≀
$[0\,..\,1] \cap [0\,..\,0.5)$

≀ [in F] $x \in [0.5\,..\,1]$ ≀
$[0\,..\,1] \cap [0.5\,..\,1]$

≀ $x \in [1\,..\,1.5]$ ≀

≀ $x \in [2\,..\,4]$ ≀

≀ $x \in [1\,..\,4]$ ≀
$[1\,..\,1.5] \sqcup [2\,..\,4]$

We use these semantics (use your intuition for the rest):

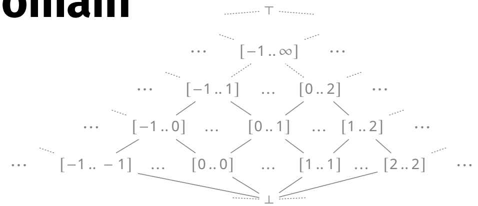
$\mathbb{E}[\![\, \texttt{if(c) a else b} \,]\!]\rho \stackrel{\text{def}}{=} \mathbb{E}[\![\, \texttt{a} \,]\!](\mathbb{C}[\![\, \texttt{c} \,]\!]\rho) \sqcup \mathbb{E}[\![\, \texttt{b} \,]\!](\mathbb{C}[\![\, \neg\texttt{c} \,]\!]\rho)$

"Evaluate a assuming that c is true, evaluate b assuming that c is false, and join the results. "

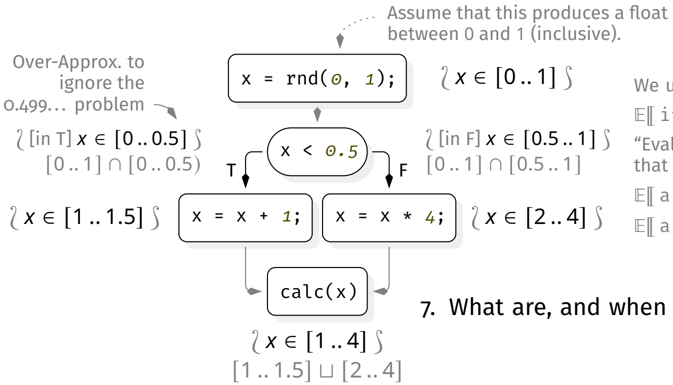
$\mathbb{E}[\![\, \texttt{a + b} \,]\!]\rho \stackrel{\text{def}}{=} [l_a + l_b\,..\,u_a + u_b]$

$\mathbb{E}[\![\, \texttt{a * b} \,]\!]\rho \stackrel{\text{def}}{=} [\min S\,..\,\max S]$ with $S = \{\, l_a l_b, l_a u_b, u_a l_b, u_a u_b \,\}$
$(\texttt{a} \in [l_a\,..\,u_a], \texttt{b} \in [l_b\,..\,u_b])$

Recap: Applying the Interval Domain



6. Do we need widening (∇) for this domain?
   Yes, because the lattice contains infinite ascending chains, e.g., $[0\,..\,0] \sqsubseteq [0\,..\,1] \sqsubseteq [0\,..\,2] \sqsubseteq \dots$



We use these semantics (use your intuition for the rest):

$\mathbb{E}[\![\texttt{if(c) a else b}]\!]\rho \overset{\text{def}}{=} \mathbb{E}[\![\texttt{a}]\!](\mathbb{C}[\![\texttt{c}]\!]\rho) \sqcup \mathbb{E}[\![\texttt{b}]\!](\mathbb{C}[\![\neg\texttt{c}]\!]\rho)$

"Evaluate a assuming that c is true, evaluate b assuming that c is false, and join the results. "

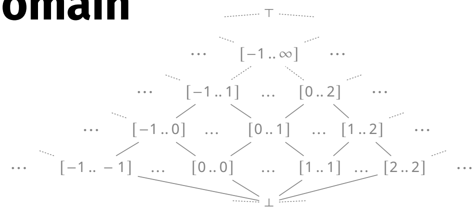
$\mathbb{E}[\![\texttt{a + b}]\!]\rho \overset{\text{def}}{=} [l_a + l_b \,..\, u_a + u_b]$

$\mathbb{E}[\![\texttt{a * b}]\!]\rho \overset{\text{def}}{=} [\min S \,..\, \max S]$ with $S = \{ l_a l_b, l_a u_b, u_a l_b, u_a u_b \}$ $(\texttt{a} \in [l_a \,..\, u_a], \texttt{b} \in [l_b \,..\, u_b])$

7. What are, and when do we need fixpoints?

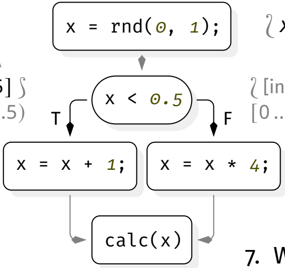
Recap: Applying the Interval Domain

6. Do we need widening (∇) for this domain?
   Yes, because the lattice contains infinite ascending chains, e.g., $[0\,..\,0] \sqsubseteq [0\,..\,1] \sqsubseteq [0\,..\,2] \sqsubseteq \dots$



Assume that this produces a float between 0 and 1 (inclusive).

Over-Approx. to ignore the 0.499… problem



$\wr\, x \in [0\,..\,1] \,\wr$

$\wr$ [in T] $x \in [0\,..\,0.5] \,\wr$
$[0\,..\,1] \cap [0\,..\,0.5)$

$\wr$ [in F] $x \in [0.5\,..\,1] \,\wr$
$[0\,..\,1] \cap [0.5\,..\,1]$

$\wr\, x \in [1\,..\,1.5] \,\wr$

$\wr\, x \in [2\,..\,4] \,\wr$

$\wr\, x \in [1\,..\,4] \,\wr$
$[1\,..\,1.5] \sqcup [2\,..\,4]$

We use these semantics (use your intuition for the rest):

$\mathbb{E}[\![\, \texttt{if(c) a else b} \,]\!]\rho \overset{\text{def}}{=} \mathbb{E}[\![\, \texttt{a} \,]\!](\mathbb{C}[\![\, \texttt{c} \,]\!]\rho) \sqcup \mathbb{E}[\![\, \texttt{b} \,]\!](\mathbb{C}[\![\, \neg\texttt{c} \,]\!]\rho)$

"Evaluate a assuming that c is true, evaluate b assuming that c is false, and join the results. "

$\mathbb{E}[\![\, \texttt{a + b} \,]\!]\rho \overset{\text{def}}{=} [l_a + l_b\,..\,u_a + u_b]$

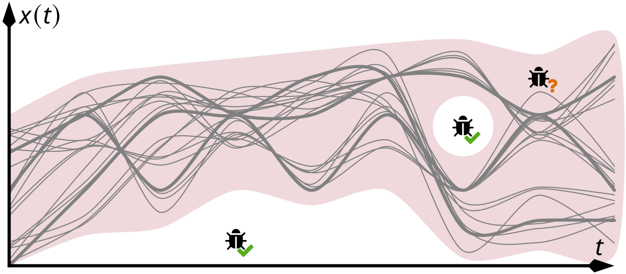
$\mathbb{E}[\![\, \texttt{a * b} \,]\!]\rho \overset{\text{def}}{=} [\min S\,..\,\max S]$ with $S = \{\, l_a l_b, l_a u_b, u_a l_b, u_a u_b \,\}$
$(\texttt{a} \in [l_a\,..\,u_a], \texttt{b} \in [l_b\,..\,u_b])$

7. What are, and when do we need fixpoints?
   $f(x) = x$. *We reach fixpoints when the repeated application of semantics produces no new info. When analyzing loops or recursive functions, we compute fixpoints to find a stable state (of all code executions).*

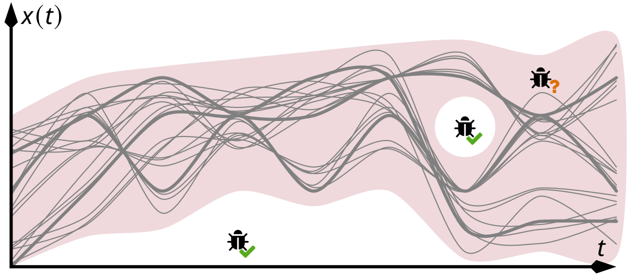
2. Introduction

What we have…

What we have...

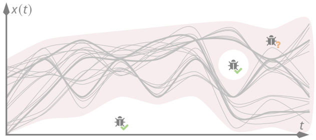


What we have...



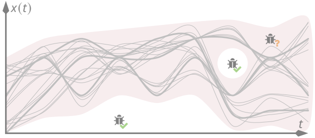
Abstractions

What we have...

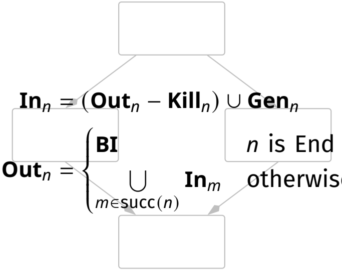


Abstractions

What we have...



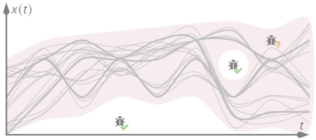
Abstractions



$$\mathbf{In}_n = (\mathbf{Out}_n - \mathbf{Kill}_n) \cup \mathbf{Gen}_n$$

$$\mathbf{Out}_n = \begin{cases} \mathbf{BI} & n \text{ is End} \\ \bigcup\limits_{m \in \mathrm{succ}(n)} \mathbf{In}_m & \text{otherwise} \end{cases}$$

What we have…



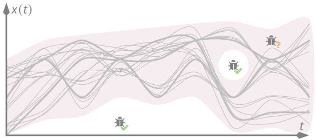
Abstractions

$$\mathbf{In}_n = (\mathbf{Out}_n - \mathbf{Kill}_n) \cup \mathbf{Gen}_n$$

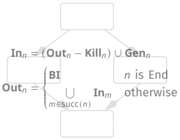
$$\mathbf{Out}_n = \begin{cases} \mathbf{BI} & n \text{ is End} \\ \bigcup\limits_{m \in \mathrm{succ}(n)} \mathbf{In}_m & \text{otherwise} \end{cases}$$

Data Flow Analysis

What we have…

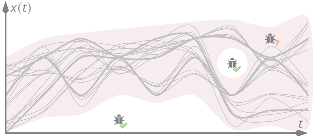


Abstractions

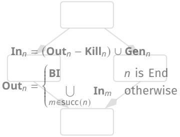


Data Flow Analysis

What we have…



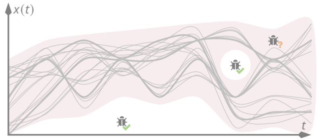
Abstractions



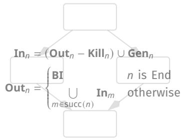
Data Flow Analysis

…

What we have… **Theory**



Abstractions



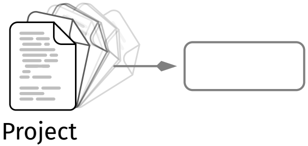
Data Flow Analysis

…

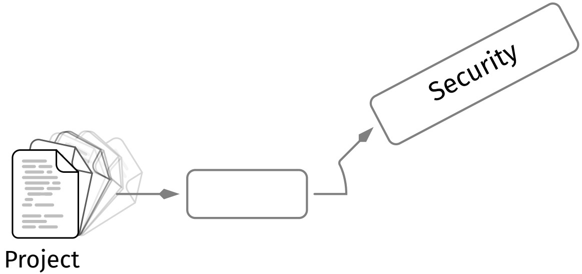
What we want...

What we want… **Tools**

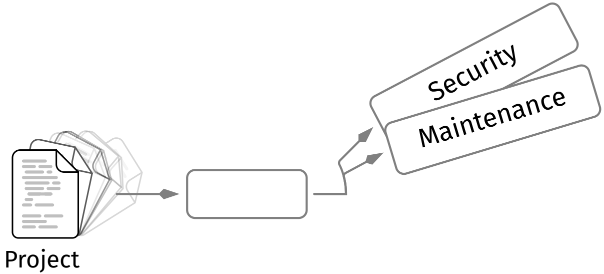
What we want… Tools



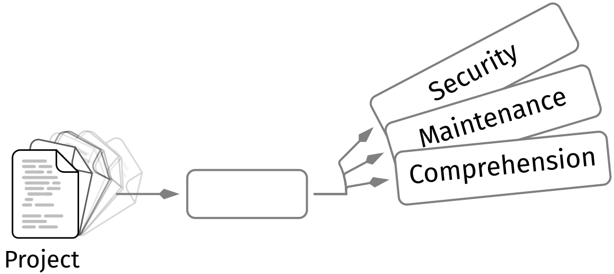
What we want… **Tools**



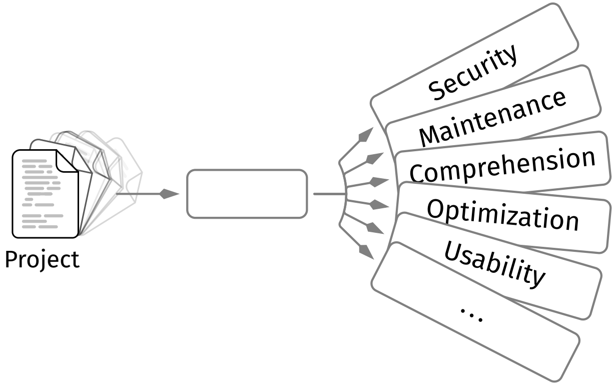
What we want… **Tools**



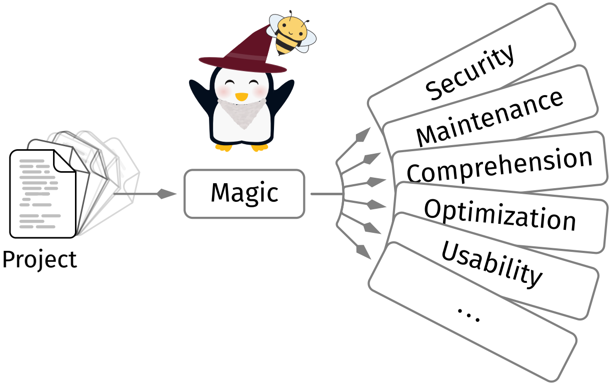
What we want… **Tools**



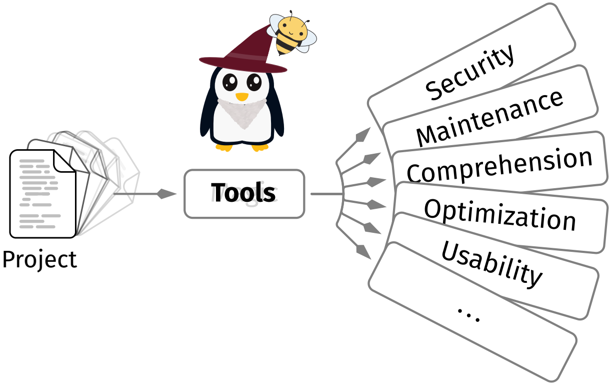
What we want… **Tools**



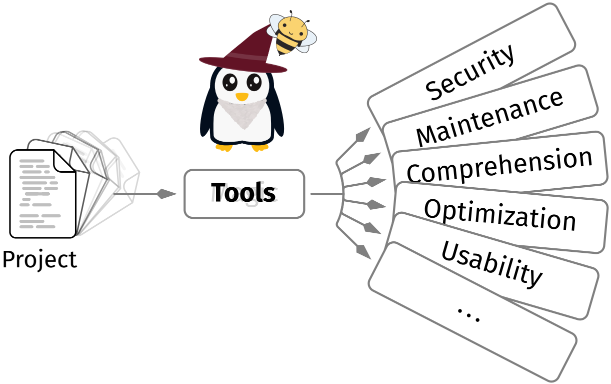
What we want… **Tools**



What we want… **Tools**

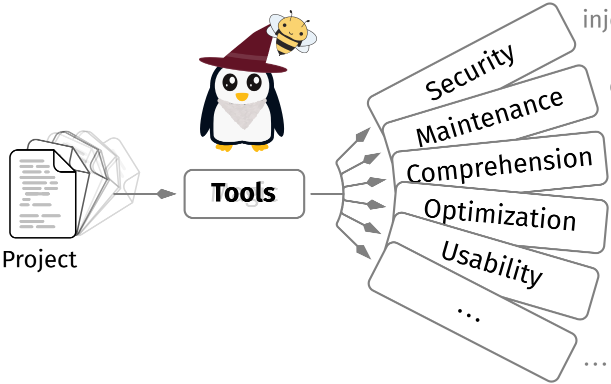


What we want… Tools



"Any sufficiently advanced technology is indistinguishable from magic." — Arthur C. Clarke

What we want… **Tools**



injections, leaks, …

code clones, deprecation, …

documentation, structure, …

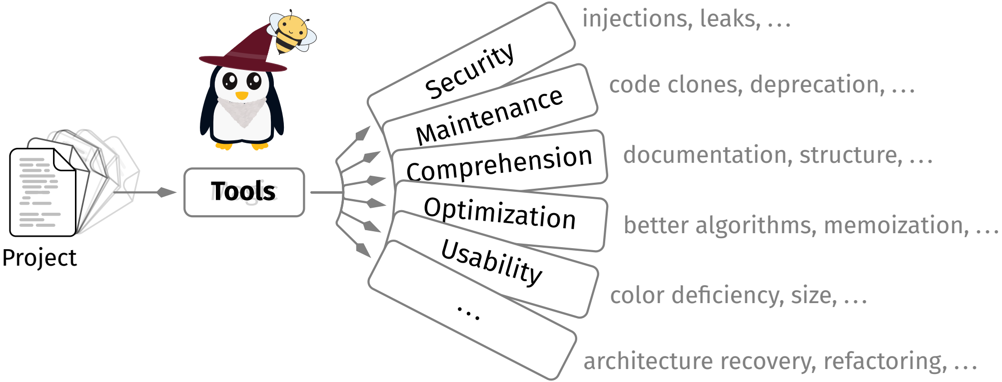
better algorithms, memoization, …

color deficiency, size, …

…

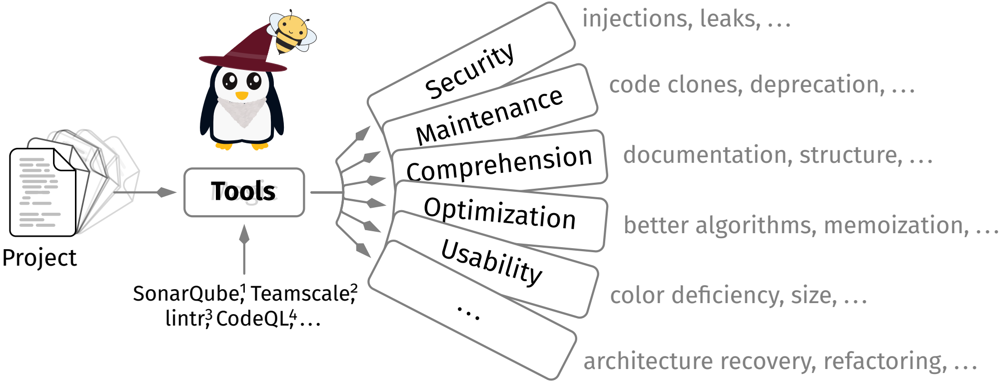
“Any sufficiently advanced technology is indistinguishable from magic.” — Arthur C. Clarke

What we want… **Tools**



“Any sufficiently advanced technology is indistinguishable from magic.” — Arthur C. Clarke

What we want… **Tools**



"Any sufficiently advanced technology is indistinguishable from magic." — Arthur C. Clarke

[1] sonarsource.com, [2] teamscale.com, [3] lintr.r-lib.org, [4] codeql.github.com

What do they… do?

What do they… do?

They take **Input**

What do they… do?

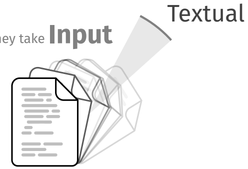
And use **Perspectives** (often combined)

They take **Input**

What do they… do?

And use **Perspectives** (often combined)

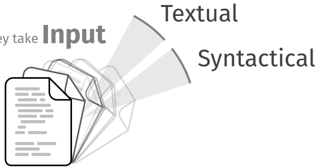
They take **Input**



What do they… do?

And use **Perspectives** (often combined)

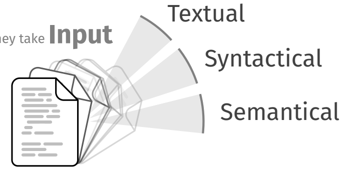
They take **Input**



What do they… do?

And use **Perspectives** (often combined)

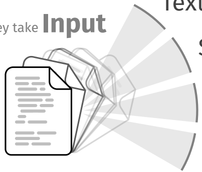
They take **Input**



What do they… do?

And use **Perspectives** (often combined)

They take **Input**



Textual

Syntactical

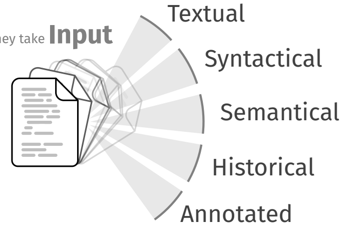
Semantical

Historical

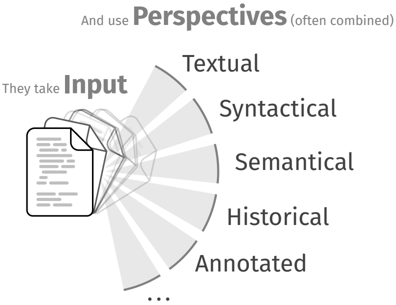
What do they… do?

They take **Input**

And use **Perspectives** (often combined)



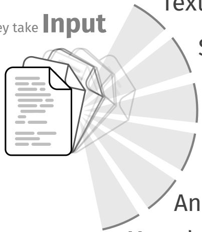
What do they… do?



What do they… do?

And use **Perspectives** (often combined)

They take **Input**



Textual

Syntactical

Semantical

Historical

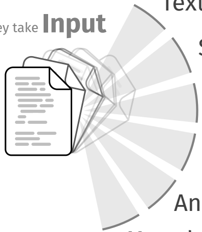
Annotated

Metadata, …

What do they… do?

And use **Perspectives** (often combined)

They take **Input**



Textual

Syntactical

Semantical

Historical

Annotated

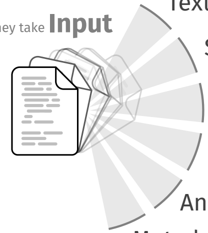
Metadata, …

Some of those are the
result of other static
or dynamic analyses

What do they… do?

They take **Input**

And use **Perspectives** (often combined)



Textual

Syntactical

Semantical

Historical

Annotated

Metadata, …

Some of those are the result of other static or dynamic analyses

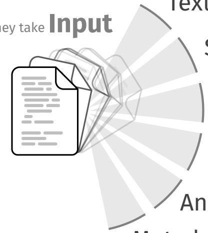
To apply **Theory**

What do they… do?

And use **Perspectives** (often combined)

To apply **Theory**

They take **Input**



Textual

Syntactical

Semantical

Historical

Annotated

Metadata, …

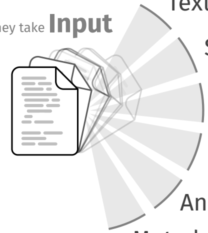
Some of those are the result of other static or dynamic analyses

Text/Code Search

What do they… do?

And use **Perspectives** (often combined)

They take **Input**



Textual

Syntactical

Semantical

Historical

Annotated

Metadata, …

Some of those are the result of other static or dynamic analyses

To apply **Theory**

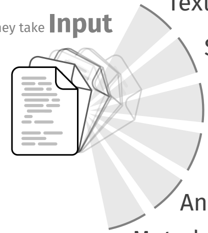
Text/Code Search

Clustering

What do they… do?

They take **Input**

And use **Perspectives** (often combined)



Textual

Syntactical

Semantical

Historical

Annotated

Metadata, …

Some of those are the
result of other static
or dynamic analyses

To apply **Theory**

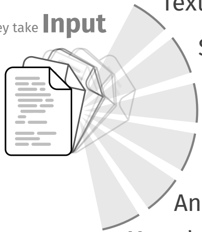
Text/Code Search

Clustering

Abstract Domains

What do they… do?

They take **Input**



And use **Perspectives** (often combined)

Textual

Syntactical

Semantical

Historical

Annotated

Metadata, …

Some of those are the
result of other static
or dynamic analyses

To apply **Theory**

Text/Code Search

Clustering

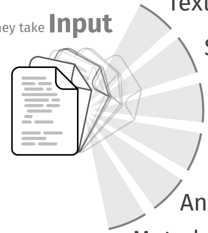
Abstract Domains

Dataflow Constraints

⋮

What do they… do?

They take **Input**



And use **Perspectives** (often combined)

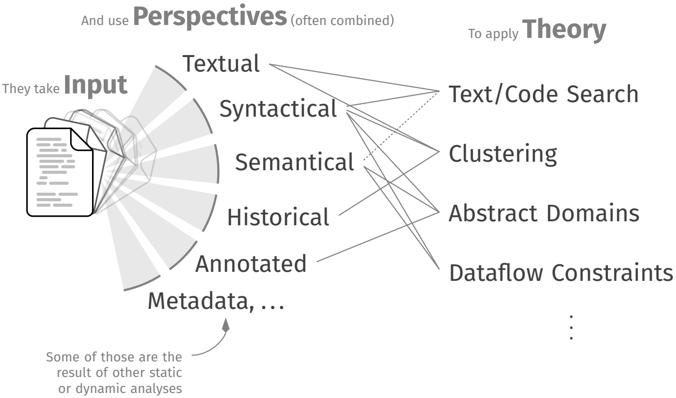
- Textual
- Syntactical
- Semantical
- Historical
- Annotated
- Metadata, …

Some of those are the result of other static or dynamic analyses

To apply **Theory**

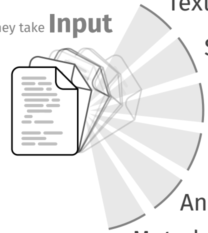
- Text/Code Search
- Clustering
- Abstract Domains
- Dataflow Constraints
- ⋮

What do they… do?



What do they… do?

They take **Input**



And use **Perspectives** (often combined)

Textual

Syntactical

Semantical

Historical

Annotated

Metadata, …

Some of those are the
result of other static
or dynamic analyses

To apply **Theory**

Text/Code Search

Clustering

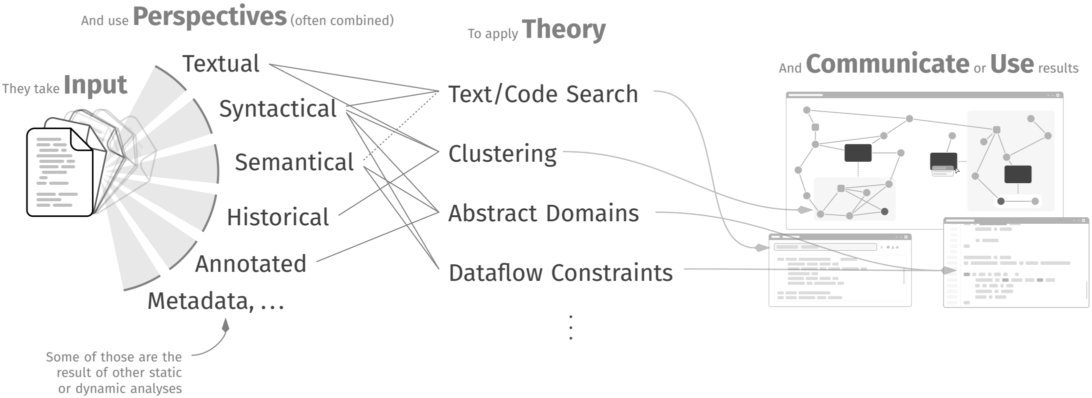
Abstract Domains

Dataflow Constraints

⋮

And **Communicate** or **Use** results

What do they… do?



3. Real-World Static Analyzers

Let’s Look at Tools

Comprehension

Security

Optimization

Usability

Maintenance

Let's Look at Tools

github.com/analysis-tools-dev/static-analysis

Let’s Look at Tools

sonar

github.com/analysis-tools-dev/static-analysis

Let's Look at Tools

Teamscale

sonar

github.com/analysis-tools-dev/static-analysis

Let’s Look at Tools

SootUp

flowR

LiquidHaskell

AbsInt

lintr

Teamscale

Infer

LiSA

CodeQL

sonar

ESLint

MOPSA analyzer

frama C

github.com/analysis-tools-dev/static-analysis

Let's Look at Tools

SootUp

flowR

lintr

LiquidHaskell

AbsInt

Teamscale

Infer

LiSA

CodeQL

sonar

Clang

GCC

ESLint

MOPSA analyzer

frama C

github.com/analysis-tools-dev/static-analysis

Let’s Look at Tools

SootUp

flowR

LiquidHaskell

AbsInt

lintr

Teamscale

Infer

LiSA

CodeQL

sonar

Clang

GCC

ESLint

MOPSA analyzer

frama C

There are countless…

github.com/analysis-tools-dev/static-analysis

Let’s Look at Tools

There are countless…

github.com/analysis-tools-dev/static-analysis

Let's Look at Tools

- SonarLint

There are countless...

github.com/analysis-tools-dev/static-analysis

Let's Look at Tools

- SonarLint
- Astrée

There are countless…

github.com/analysis-tools-dev/static-analysis

Let's Look at Tools

- SonarLint
- Astrée (sound!)

There are countless…

github.com/analysis-tools-dev/static-analysis

Let's Look at Tools

- SonarLint
- Astrée (sound!)
- LiSA

There are countless…

github.com/analysis-tools-dev/static-analysis

Let's Look at Tools

- SonarLint
- Astrée (sound!)
- LiSA

- Java Language Server

There are countless...

github.com/analysis-tools-dev/static-analysis

Let's Look at Tools

- SonarLint
- Astrée (sound!)
- LiSA
- Java Language Server
- lintr

There are countless...

github.com/analysis-tools-dev/static-analysis

Let’s Look at Tools

- SonarLint
- Astrée (sound!)
- LiSA
- Java Language Server
- lintr
- flowR

There are countless…

github.com/analysis-tools-dev/static-analysis

Let’s Look at Tools

- **SonarLint**
- Astrée (sound!)
- LiSA
- Java Language Server
- lintr
- flowR

There are countless…

github.com/analysis-tools-dev/static-analysis

SonarLint

github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)

github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules

github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules

Separate frontends and analyzers per language!

github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!

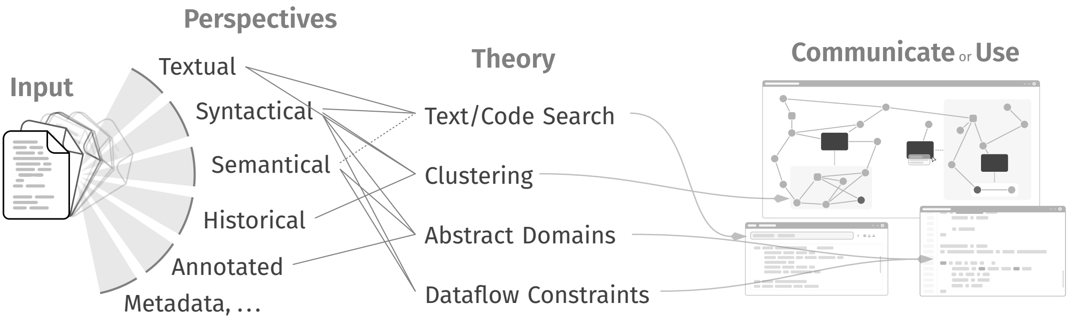
github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!



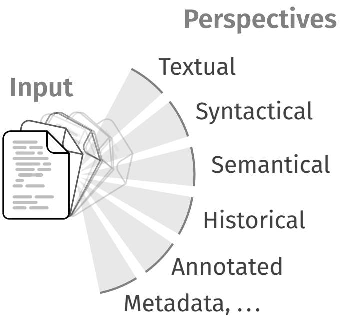
github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!



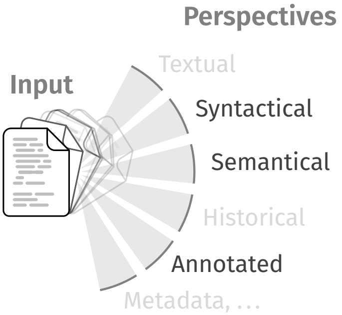
github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!



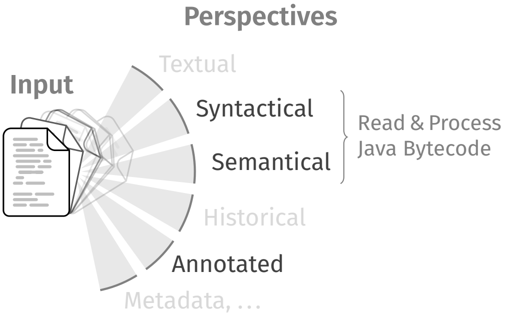
github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!



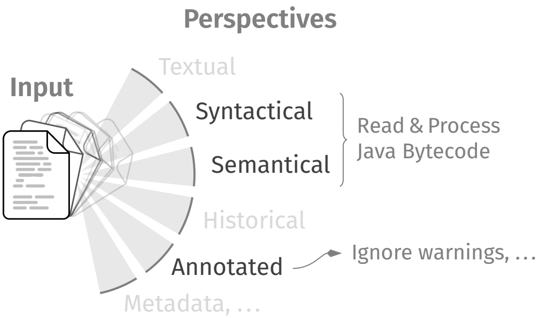
github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!



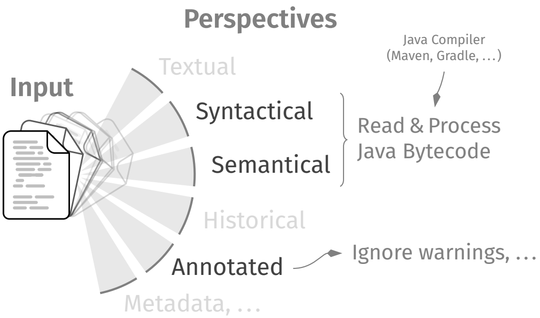
github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!



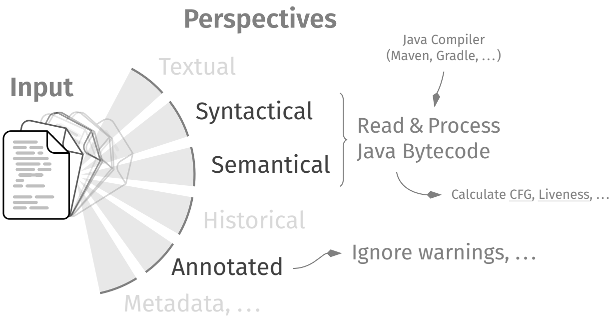
github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!



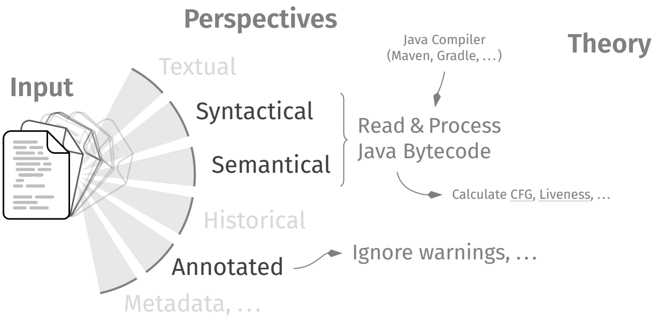
github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!



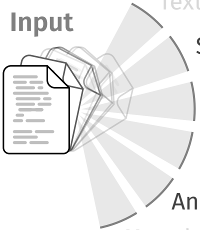
github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!



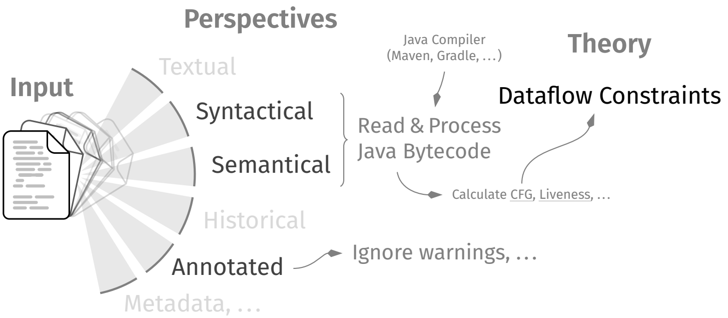
github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!



github.com/SonarSource/sonar-java

SonarLint

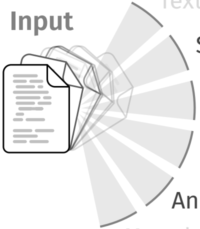
- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!

Perspectives

Input



Textual

Syntactical

Semantical

Historical

Annotated

Metadata, ...

Java Compiler
(Maven, Gradle, ...)

Read & Process
Java Bytecode

Calculate CFG, Liveness, ...

Theory

Dataflow Constraints

Ignore warnings, ...

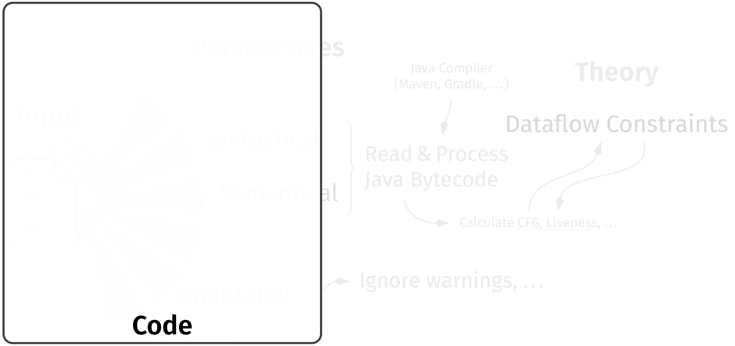
github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!



github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!

```
var x = 0;
var r = Math.random();
if(r > .5) {
  x = 1;
} else {
  x = 2;
}
System.out.print(x);
```

Code

github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!

```
var x = 0;
var r = Math.random();
if(r > .5) {
  x = 1;
} else {
  x = 2;
}
System.out.print(x);
```

Code

Control Flow Graph

github.com/SonarSource/sonar-java

SonarLint

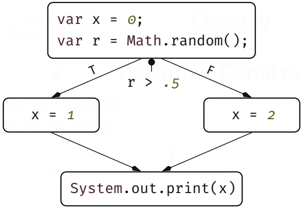
- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!

```java
var x = 0;
var r = Math.random();
if(r > .5) {
  x = 1;
} else {
  x = 2;
}
System.out.print(x);
```

Code



Control Flow Graph

github.com/SonarSource/sonar-java

SonarLint

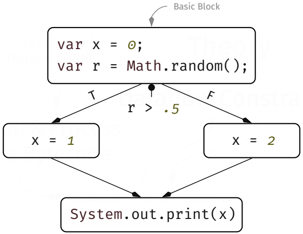
- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!

```java
var x = 0;
var r = Math.random();
if(r > .5) {
  x = 1;
} else {
  x = 2;
}
System.out.print(x);
```

Code



Control Flow Graph

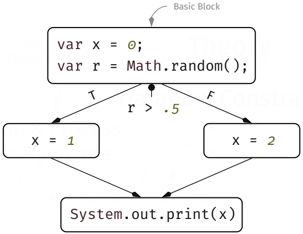
github.com/SonarSource/sonar-java

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
    Separate frontends and analyzers per language!
  - We focus on **sonar-java**
    733 rules!

```
var x = 0;
var r = Math.random();
if(r > .5) {
  x = 1;
} else {
  x = 2;
}
System.out.print(x);
```

Code



Control Flow Graph

$$\mathbf{In}_n = (\mathbf{Out}_n - \mathbf{Kill}_n) \cup \mathbf{Gen}_n$$

$$\mathbf{Out}_n = \begin{cases} \mathbf{BI} & n \text{ is End} \\ \bigcup_{m \in \mathrm{succ}(n)} \mathbf{In}_m & \text{otherwise} \end{cases}$$

With fixpoint iteration

Liveness[KSK09, p. 26]

github.com/SonarSource/sonar-java

[KSK09] *Data Flow Analysis: Theory and Practice* (Khedker, Sanyal, and Karkare)

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!

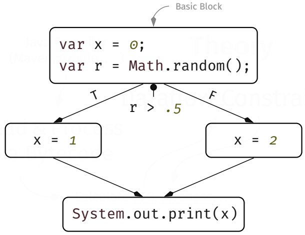
$$\mathbf{In}_n = (\mathbf{Out}_n - \mathbf{Kill}_n) \cup \mathbf{Gen}_n$$

$$\mathbf{Out}_n = \begin{cases} \mathbf{BI} & n \text{ is End} \\ \bigcup_{m \in \mathrm{succ}(n)} \mathbf{In}_m & \text{otherwise} \end{cases}$$

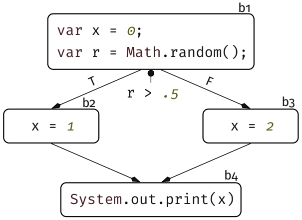
With fixpoint iteration

```java
var x = 0;
var r = Math.random();
if(r > .5) {
  x = 1;
} else {
  x = 2;
}
System.out.print(x);
```

Code



Control Flow Graph



Liveness[KSK09, p. 26]

github.com/SonarSource/sonar-java

[KSK09] *Data Flow Analysis: Theory and Practice* (Khedker, Sanyal, and Karkare)

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!

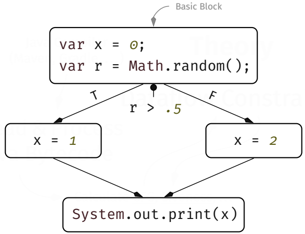
$$\mathbf{In}_n = (\mathbf{Out}_n - \mathbf{Kill}_n) \cup \mathbf{Gen}_n$$

$$\mathbf{Out}_n = \begin{cases} \mathbf{BI} & n \text{ is End} \\ \bigcup_{m \in \mathrm{succ}(n)} \mathbf{In}_m & \text{otherwise} \end{cases}$$

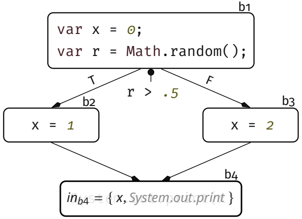
With fixpoint iteration

```java
var x = 0;
var r = Math.random();
if(r > .5) {
   x = 1;
} else {
   x = 2;
}
System.out.print(x);
```

Code



Control Flow Graph



Liveness[KSK09, p. 26]

github.com/SonarSource/sonar-java

[KSK09] *Data Flow Analysis: Theory and Practice* (Khedker, Sanyal, and Karkare)

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!

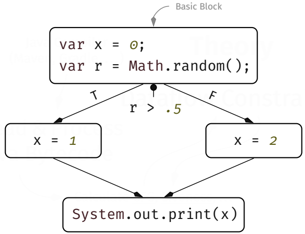
$$\mathbf{In}_n = (\mathbf{Out}_n - \mathbf{Kill}_n) \cup \mathbf{Gen}_n$$

$$\mathbf{Out}_n = \begin{cases} \mathbf{BI} & n \text{ is End} \\ \bigcup_{m \in succ(n)} \mathbf{In}_m & \text{otherwise} \end{cases}$$

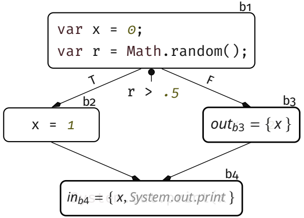
With fixpoint iteration

```
var x = 0;
var r = Math.random();
if(r > .5) {
  x = 1;
} else {
  x = 2;
}
System.out.print(x);
```

Code



Control Flow Graph



Liveness[KSK09, p. 26]

github.com/SonarSource/sonar-java

[KSK09] *Data Flow Analysis: Theory and Practice* (Khedker, Sanyal, and Karkare)

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!

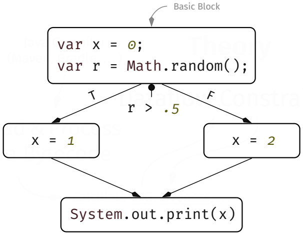
$$\mathbf{In}_n = (\mathbf{Out}_n - \mathbf{Kill}_n) \cup \mathbf{Gen}_n$$

$$\mathbf{Out}_n = \begin{cases} \mathbf{BI} & n \text{ is End} \\ \bigcup_{m \in \mathrm{succ}(n)} \mathbf{In}_m & \text{otherwise} \end{cases}$$

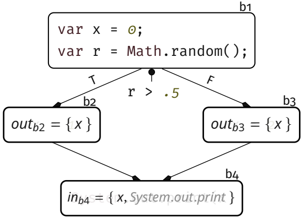
With fixpoint iteration

```
var x = 0;
var r = Math.random();
if(r > .5) {
  x = 1;
} else {
  x = 2;
}
System.out.print(x);
```

Code



Control Flow Graph



Liveness[KSK09, p. 26]

github.com/SonarSource/sonar-java

[KSK09] *Data Flow Analysis: Theory and Practice* (Khedker, Sanyal, and Karkare)

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!

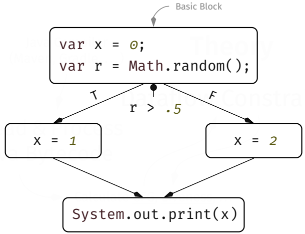
$$\textbf{In}_n = (\textbf{Out}_n - \textbf{Kill}_n) \cup \textbf{Gen}_n$$

$$\textbf{Out}_n = \begin{cases} \textbf{BI} & n \text{ is End} \\ \bigcup_{m \in \mathrm{succ}(n)} \textbf{In}_m & \text{otherwise} \end{cases}$$

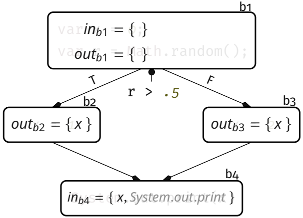
With fixpoint iteration

```
var x = 0;
var r = Math.random();
if(r > .5) {
  x = 1;
} else {
  x = 2;
}
System.out.print(x);
```

Code



Control Flow Graph



Liveness[KSK09, p. 26]

github.com/SonarSource/sonar-java

[KSK09] *Data Flow Analysis: Theory and Practice* (Khedker, Sanyal, and Karkare)

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!

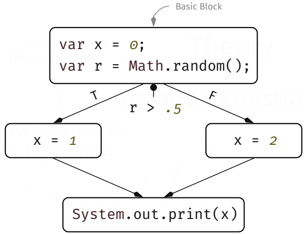
$$\mathbf{In}_n = (\mathbf{Out}_n - \mathbf{Kill}_n) \cup \mathbf{Gen}_n$$

$$\mathbf{Out}_n = \begin{cases} \mathbf{BI} & n \text{ is End} \\ \bigcup_{m \in \mathrm{succ}(n)} \mathbf{In}_m & \text{otherwise} \end{cases}$$

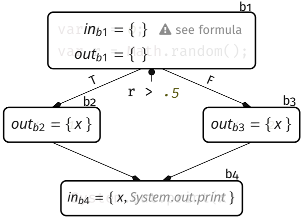
With fixpoint iteration

```
var x = 0;
var r = Math.random();
if(r > .5) {
   x = 1;
} else {
   x = 2;
}
System.out.print(x);
```

Code



Control Flow Graph



Liveness[KSK09, p. 26]

github.com/SonarSource/sonar-java

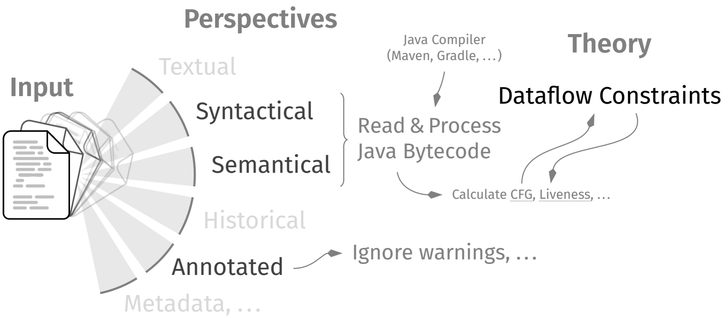
[KSK09] *Data Flow Analysis: Theory and Practice* (Khedker, Sanyal, and Karkare)

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!



github.com/SonarSource/sonar-java

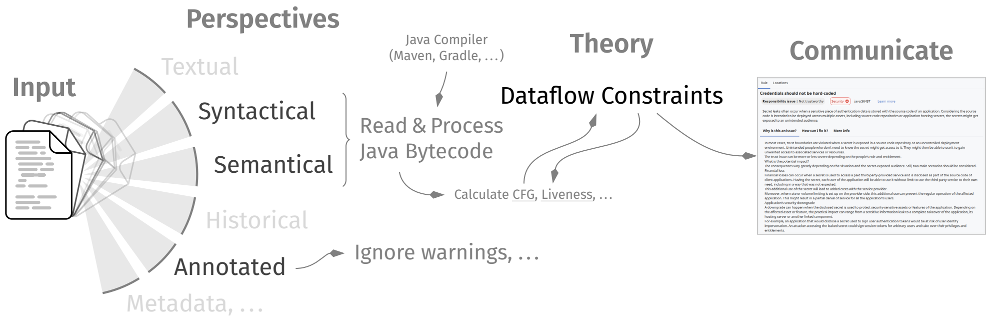
[KSK09] *Data Flow Analysis: Theory and Practice* (Khedker, Sanyal, and Karkare)

SonarLint

- Support for multiple languages (15+)
  - Widely varying support and rules
  - We focus on **sonar-java**

Separate frontends and analyzers per language!

733 rules!



github.com/SonarSource/sonar-java

[KSK09] *Data Flow Analysis: Theory and Practice* (Khedker, Sanyal, and Karkare)

SonarLint — **Unused Assignments**

Implementation of Rule S1854

SonarLint — Unused Assignments

- Analyze “Dead Stores”

Implementation of Rule S1854

SonarLint — Unused Assignments

- Analyze "Dead Stores" (in 312 loc / 260 cloc):

Implementation of Rule S1854

SonarLint — Unused Assignments

- Analyze “Dead Stores” (in 312 loc / 260 cloc):

```
int x = 0;
x = 42;
```

Implementation of Rule S1854

SonarLint — Unused Assignments

- Analyze “Dead Stores” (in 312 loc / 260 cloc):

```
int x = 0;
x = 42;
```

- Use liveness analysis to obtain out_n of each basic block in the CFG

Implementation of Rule S1854

SonarLint — Unused Assignments

- Analyze "Dead Stores" (in 312 loc / 260 cloc):

```
int x = 0;
x = 42;
```

- Use liveness analysis to obtain out_n of each basic block in the CFG
- Check if assignments (`x =`, `x++`, ...) are in out_n and resolved

Implementation of Rule S1854

SonarLint — Unused Assignments

- Analyze "Dead Stores" (in 312 loc / 260 cloc):

```
int x = 0;
x = 42;
```

- Use liveness analysis to obtain out_n of each basic block in the CFG
- Check if assignments (`x =`, `x++`, ...) are in out_n and resolved
- Check overwrites in the same basic block

Implementation of Rule S1854

SonarLint — Unused Assignments

- Analyze "Dead Stores" (in 312 loc / 260 cloc):

```
int x = 0;
x = 42;
```

- Use liveness analysis to obtain out_n of each basic block in the CFG
- Check if assignments (`x =`, `x++`, …) are in out_n and resolved
- Check overwrites in the same basic block
- Minor special handling for **`try-finally`** blocks, …

Implementation of Rule S1854

SonarLint — **Unused Assignments**

- Analyze “Dead Stores” (in 312 loc / 260 cloc):

```
int x = 0;
x = 42;
```

- Use liveness analysis to obtain out_n of each basic block in the CFG
- Check if assignments (`x =`, `x++`, `...`) are in out_n and resolved
- Check overwrites in the same basic block
- Minor special handling for `try-finally` blocks ...

```
LiveVariables liveVariables = LiveVariables.analyze(cfg);
// Liveness analysis provides information only for block boundaries,
    so we should do analysis between elements within blocks
for(CFG.Block block : cfg.blocks()) {
  checkElements(block, liveVariables.getOut(block), methodSymbol);
}
```

Implementation of Rule S1854

SonarLint — Hardcoded Credentials

Implementation of Rule S6437

SonarLint — Hardcoded Credentials

- It does not always have to be that heavy!

Implementation of Rule S6437

SonarLint — **Hardcoded Credentials**

- It does not always have to be that heavy! (116 loc / 87 cloc may suffice)

Implementation of Rule S6437

SonarLint — Hardcoded Credentials

- It does not always have to be that heavy! (116 loc / 87 cloc may suffice)
- For example, to identify hardcoded credentials:

Implementation of Rule S6437

SonarLint — Hardcoded Credentials

- It does not always have to be that heavy! (116 loc / 87 cloc may suffice)
- For example, to identify hardcoded credentials:

```java
new PasswordAuthentication("password", "secret".toCharArray());
```

Implementation of Rule S6437

SonarLint — Hardcoded Credentials

- It does not always have to be that heavy! (116 loc / 87 cloc may suffice)
- For example, to identify hardcoded credentials:

```java
new PasswordAuthentication("password", "secret".toCharArray());
```

- Traverse the Abstract Syntax Tree (AST)

Implementation of Rule S6437

SonarLint — Hardcoded Credentials

- It does not always have to be that heavy! (116 loc / 87 cloc may suffice)
- For example, to identify hardcoded credentials:

```java
new PasswordAuthentication("password", "secret".toCharArray());
```

- Traverse the Abstract Syntax Tree (AST)
- Check calls against a long list of signatures (currently 7664) with problematic indices

Implementation of Rule S6437

SonarLint — Hardcoded Credentials

- It does not always have to be that heavy! (116 loc / 87 cloc may suffice)
- For example, to identify hardcoded credentials:

```
new PasswordAuthentication("password", "secret".toCharArray());
```

- Traverse the Abstract Syntax Tree (AST)
- Check calls against a long list of signatures (currently 7664) with problematic indices
- Check if the arguments are “constant”

Implementation of Rule S6437

SonarLint — Hardcoded Credentials

- It does not always have to be that heavy! (116 loc / 87 cloc may suffice)
- For example, to identify hardcoded credentials:

```
new PasswordAuthentication("password", "secret".toCharArray());
```

- Traverse the Abstract Syntax Tree (AST)
- Check calls against a long list of signatures (currently 7664) with problematic indices
- Check if the arguments are "constant"
  visiting the dataflow links and checking for predefined "plain text"

Implementation of Rule S6437

SonarLint — Hardcoded Credentials

- It does not always have to be that heavy! (116 loc / 87 cloc may suffice)
- For example, to identify hardcoded credentials:

```
new PasswordAuthentication("password", "secret".toCharArray());
```

- Traverse the Abstract Syntax Tree (AST)
- Check calls against a long list of signatures (currently 7664) with problematic indices
- Check if the arguments are “constant”
  visiting the

```java
for (int targetArgumentIndex : method.indices) {
  ExpressionTree argument = arguments.get(targetArgumentIndex);
   var secondaryLocations = new ArrayList<JavaFileScannerContext.Location>();
   if (isExpressionDerivedFromPlainText(argument,
             secondaryLocations, new HashSet<>())) {
      reportIssue(argument, ISSUE_MESSAGE, secondaryLocations, null);
   }
}
```

Implementation of Rule S6437

Let’s Look at Tools

SootUp

flowR

LiquidHaskell

AbsInt

lintr

Teamscale

Infer

LiSA

CodeQL

sonar

Clang

GCC

ESLint

MOPSA analyzer

frama C

There are countless…

github.com/analysis-tools-dev/static-analysis

Let's Look at Tools

- SonarLint
- **Astrée (sound!)**
- LiSA
- Java Language Server
- lintr
- flowR

There are countless…

github.com/analysis-tools-dev/static-analysis

Astrée

Astrée

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software

Astrée

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, sound static analyzer for C/C++

Astrée

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, sound*y* static analyzer for C/C++

Astrée

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, soundy static analyzer for C/C++

100+ directives and intrinsics, 140+ options
250 kloc of Ocaml, 240 kloc C/C++

Astrée

AbsInt

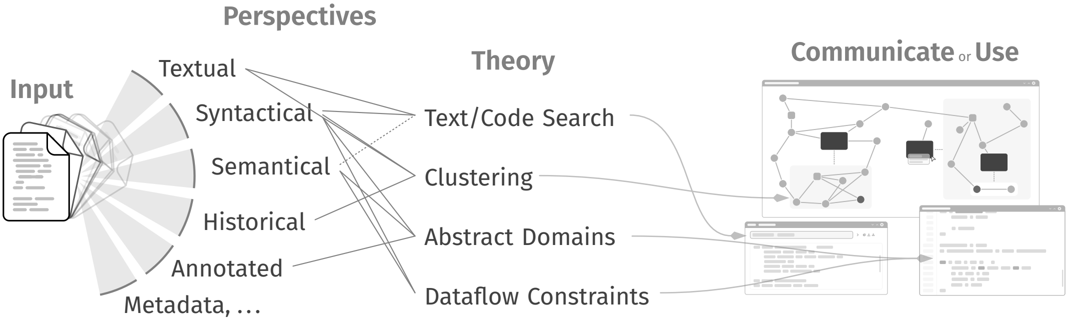
- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, soundy static analyzer for C/C++

  100+ directives and intrinsics, 140+ options
  250 kloc of Ocaml, 240 kloc C/C++
- Uses abstract domains for timing validation, buffer overflows, …

Astrée

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, soundy static analyzer for C/C++

  100+ directives and intrinsics, 140+ options
  250 kloc of Ocaml, 240 kloc C/C++
- Uses abstract domains for timing validation, buffer overflows, …

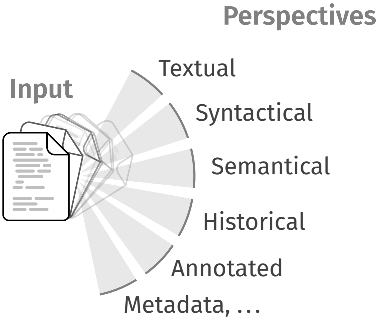


Astrée

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, soundy static analyzer for C/C++

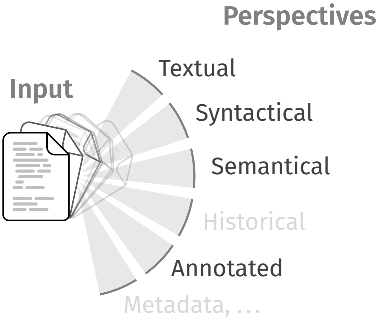
100+ directives and intrinsics, 140+ options
250 kloc of Ocaml, 240 kloc C/C++

- Uses abstract domains for timing validation, buffer overflows, …



Astrée

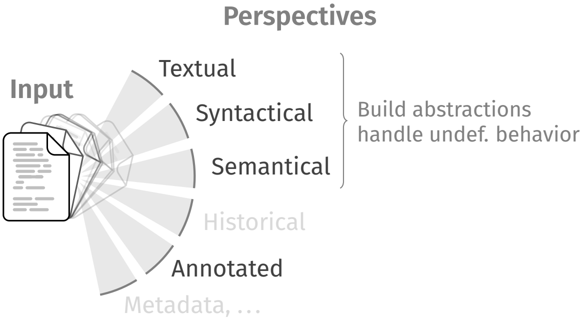
- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, soundy static analyzer for C/C++
  100+ directives and intrinsics, 140+ options
  250 kloc of Ocaml, 240 kloc C/C++
- Uses abstract domains for timing validation, buffer overflows, . . .



Astrée

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, soundy static analyzer for C/C++

  100+ directives and intrinsics, 140+ options
  250 kloc of Ocaml, 240 kloc C/C++
- Uses abstract domains for timing validation, buffer overflows, …

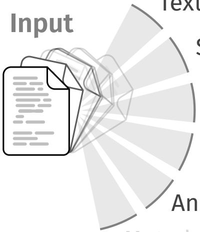


Astrée

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, sound*y* static analyzer for C/C++
  100+ directives and intrinsics, 140+ options
  250 kloc of Ocaml, 240 kloc C/C++
- Uses abstract domains for timing validation, buffer overflows, …

Perspectives

Input

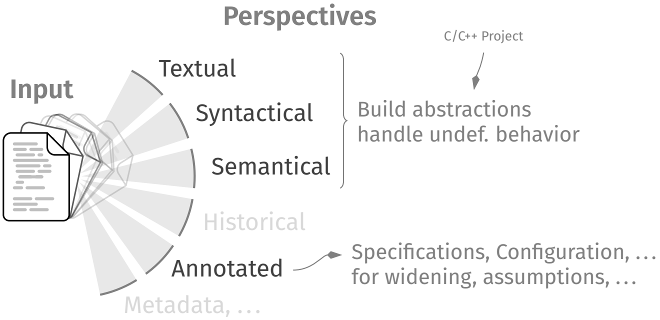


Build abstractions
handle undef. behavior

Specifications, Configuration, …
for widening, assumptions, …

Astrée

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, sound*y* static analyzer for C/C++ 100+ directives and intrinsics, 140+ options 250 kloc of Ocaml, 240 kloc C/C++
- Uses abstract domains for timing validation, buffer overflows, …



Astrée

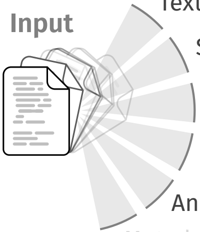
AbsInt

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, sound*y* static analyzer for C/C++

  100+ directives and intrinsics, 140+ options
  250 kloc of Ocaml, 240 kloc C/C++
- Uses abstract domains for timing validation, buffer overflows, …

Perspectives

Input



Textual
Syntactical
Semantical

C/C++ Project

Build abstractions
handle undef. behavior

CFG, Data Races, …

Historical

Annotated

Specifications, Configuration, …
for widening, assumptions, …

Metadata, …

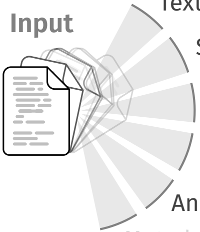
Astrée

AbsInt

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, soundy static analyzer for C/C++
  100+ directives and intrinsics, 140+ options
  250 kloc of Ocaml, 240 kloc C/C++
- Uses abstract domains for timing validation, buffer overflows, …

Perspectives

Input



Textual
Syntactical
Semantical
Historical
Annotated
Metadata, …

C/C++ Project

Build abstractions
handle undef. behavior

CFG, Data Races, …

Specifications, Configuration, …
for widening, assumptions, …

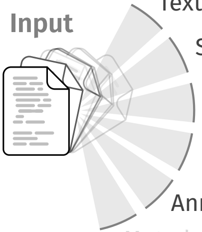
Theory

Astrée

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, sound*y* static analyzer for C/C++
  100+ directives and intrinsics, 140+ options
  250 kloc of Ocaml, 240 kloc C/C++
- Uses abstract domains for timing validation, buffer overflows, …

Perspectives

Input



Textual

Syntactical

Semantical

Historical

Annotated

Metadata, …

C/C++ Project

Build abstractions
handle undef. behavior

CFG, Data Races, …

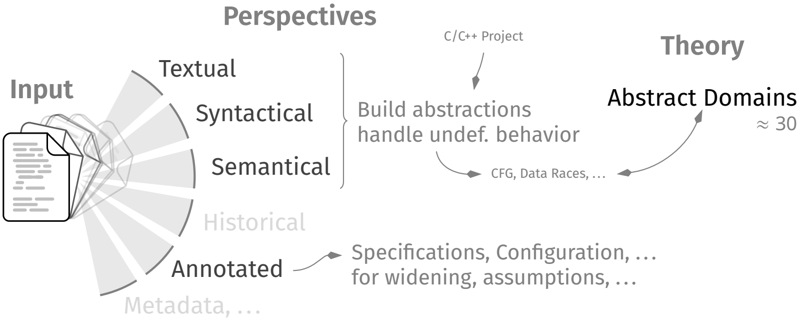
Specifications, Configuration, …
for widening, assumptions, …

Theory

Abstract Domains
≈ 30

Astrée

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, sound*y* static analyzer for C/C++
  100+ directives and intrinsics, 140+ options
  250 kloc of Ocaml, 240 kloc C/C++
- Uses abstract domains for timing validation, buffer overflows, …

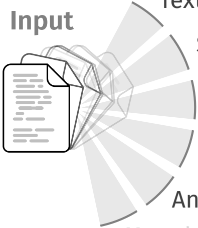


Astrée

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, soundy static analyzer for C/C++
  100+ directives and intrinsics, 140+ options
  250 kloc of Ocaml, 240 kloc C/C++
- Uses abstract domains for timing validation, buffer overflows, …

Perspectives

Input



Textual
Syntactical
Semantical
Historical
Annotated
Metadata, …

C/C++ Project

Build abstractions
handle undef. behavior

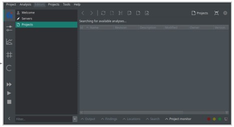
CFG, Data Races, …

Specifications, Configuration, …
for widening, assumptions, …

Theory

Abstract Domains
≈ 30

Communicate

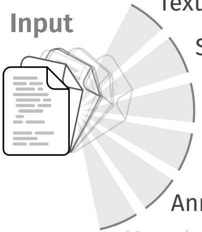


Astrée

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, sound*y* static analyzer for C/C++
  100+ directives and intrinsics, 140+ options
  250 kloc of Ocaml, 240 kloc C/C++
- Uses abstract domains for timing validation, buffer overflows, …

Perspectives

Input



Textual
Syntactical
Semantical
Historical
Annotated
Metadata, …

C/C++ Project

Build abstractions
handle undef. behavior

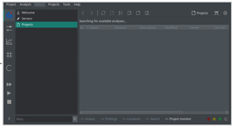
CFG, Data Races, …

Specifications, Configuration, …
for widening, assumptions, …

Theory

Abstract Domains
≈ 30

Communicate



Analysis Time
from couple of hours to days

Astrée

AbsInt

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, sound*y* static analyzer for C/C++
  100+ directives and intrinsics, 140+ options
  250 kloc of Ocaml, 240 kloc C/C++
- Uses abstract domains for timing validation, buffer overflows, …

Input

Perspectives

Textual
Syntactical
Semantical
Historical
Annotated
Metadata, …

C/C++ Project

Build abstractions
handle undef. behavior

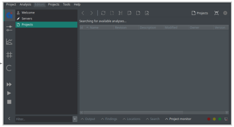
CFG, Data Races, …

Specifications, Configuration, …
for widening, assumptions, …

Theory

Abstract Domains
≈ 30

Communicate



Local precision tuning

Analysis Time
from couple of hours to days

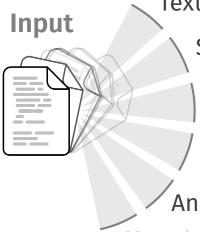
Astrée

AbsInt

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, soundy static analyzer for C/C++
  100+ directives and intrinsics, 140+ options
  250 kloc of Ocaml, 240 kloc C/C++
- Uses abstract domains for timing validation, buffer overflows, …

Perspectives

Input



Textual

Syntactical

Semantical

Historical

Annotated

Metadata, …

C/C++ Project

Build abstractions
handle undef. behavior

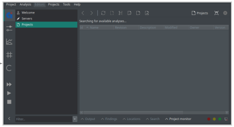
CFG, Data Races, …

Specifications, Configuration, …
for widening, assumptions, …

Theory

Abstract Domains
≈ 30

Communicate



Semantic Loop Unrolling

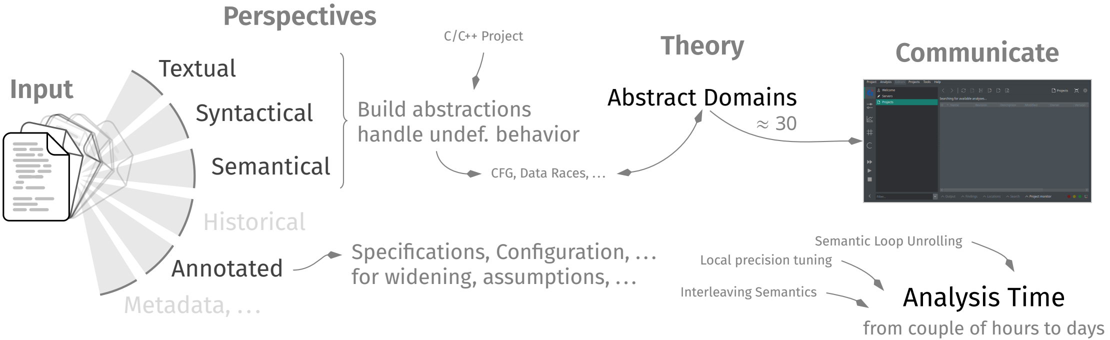
Local precision tuning

Analysis Time
from couple of hours to days

Astrée

AbsInt

- *Analyseur statique de logiciels temps-réel embarqués*
  Static analyzer for real-time embedded software
- Proprietary, soundy static analyzer for C/C++
  100+ directives and intrinsics, 140+ options
  250 kloc of Ocaml, 240 kloc C/C++
- Uses abstract domains for timing validation, buffer overflows, …



Let’s Look at Tools

SootUp
flowR
LiquidHaskell
AbsInt
lintr
Teamscale
Infer
LiSA
CodeQL
sonar
Clang
GCC
ESLint
MOPSA analyzer
frama C

There are countless…

github.com/analysis-tools-dev/static-analysis

Let’s Look at Tools

- SonarLint
- Astrée (sound!)
- **LiSA**
- Java Language Server
- lintr
- flowR

There are countless…

github.com/analysis-tools-dev/static-analysis

LiSA

github.com/lisa-analyzer/lisa

LiSA

- (Largely) Language Independent Library for Static Analysis

github.com/lisa-analyzer/lisa

LiSA

- (Largely) Language Independent Library for Static Analysis

Similar to Mopsa or Apron

github.com/lisa-analyzer/lisa

LiSA

- (Largely) Language Independent Library for Static Analysis

Similar to Mopsa or Apron

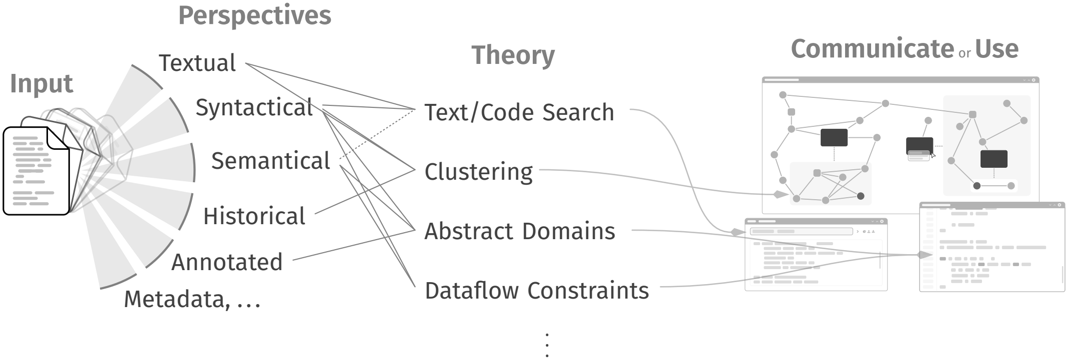
- Custom frontends for Rust, Go, Python, EVM, ...

github.com/lisa-analyzer/lisa

LiSA

- (Largely) Language Independent Library for Static Analysis
- Custom frontends for Rust, Go, Python, EVM, …

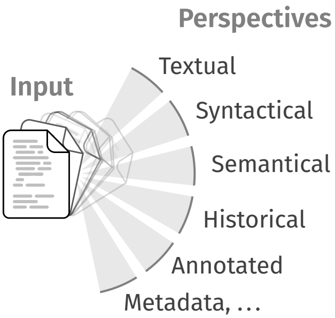
Similar to Mopsa or Apron



github.com/lisa-analyzer/lisa

LiSA

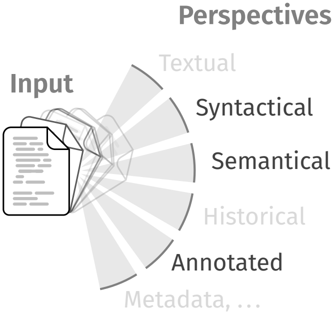
- (Largely) Language Independent Library for Static Analysis — Similar to Mopsa or Apron
- Custom frontends for Rust, Go, Python, EVM, . . .



github.com/lisa-analyzer/lisa

LiSA

- (Largely) Language Independent Library for Static Analysis — Similar to Mopsa or Apron
- Custom frontends for Rust, Go, Python, EVM, ...



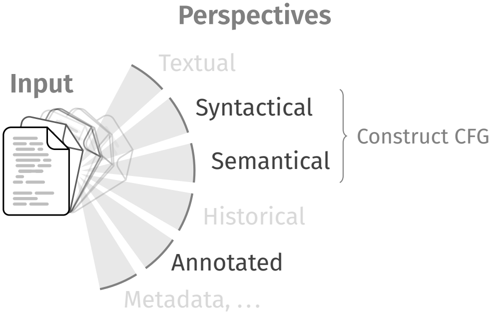
github.com/lisa-analyzer/lisa

LiSA

- (Largely) Language Independent Library for Static Analysis

Similar to Mopsa or Apron

- Custom frontends for Rust, Go, Python, EVM, ...



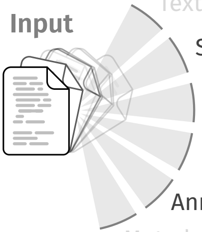
github.com/lisa-analyzer/lisa

LiSA

- (Largely) Language Independent Library for Static Analysis

Similar to Mopsa or Apron

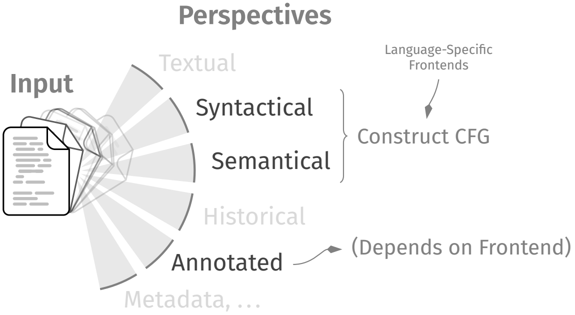
- Custom frontends for Rust, Go, Python, EVM, ...



github.com/lisa-analyzer/lisa

LiSA

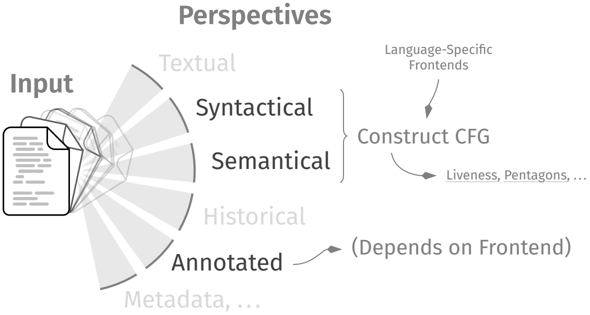
- (Largely) Language Independent Library for Static Analysis
  Similar to Mopsa or Apron
- Custom frontends for Rust, Go, Python, EVM, ...



github.com/lisa-analyzer/lisa

LiSA

- (Largely) Language Independent Library for Static Analysis — Similar to Mopsa or Apron
- Custom frontends for Rust, Go, Python, EVM, …



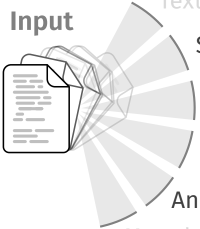
github.com/lisa-analyzer/lisa

LiSA

- (Largely) Language Independent Library for Static Analysis — Similar to Mopsa or Apron
- Custom frontends for Rust, Go, Python, EVM, ...

Perspectives

Input



Textual

Syntactical

Semantical

Construct CFG

Language-Specific Frontends

Liveness, Pentagons, ...

Historical

Annotated — (Depends on Frontend)

Metadata, ...

Theory

github.com/lisa-analyzer/lisa

LiSA

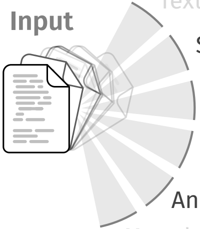
- (Largely) Language Independent Library for Static Analysis

Similar to Mopsa or Apron

- Custom frontends for Rust, Go, Python, EVM, …

Perspectives

Input



Textual

Syntactical

Semantical

Historical

Annotated

Metadata, …

Language-Specific
Frontends

Construct CFG

Liveness, Pentagons, …

(Depends on Frontend)

Theory

Abstract Domains

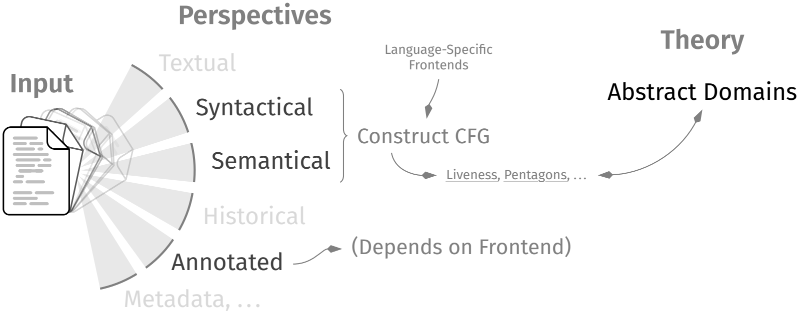
github.com/lisa-analyzer/lisa

LiSA

- (Largely) Language Independent Library for Static Analysis

Similar to Mopsa or Apron

- Custom frontends for Rust, Go, Python, EVM, ...



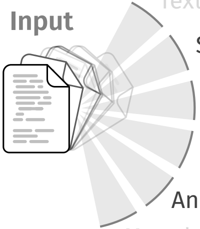
github.com/lisa-analyzer/lisa

LiSA

- (Largely) Language Independent Library for Static Analysis — Similar to Mopsa or Apron
- Custom frontends for Rust, Go, Python, EVM, …

Perspectives

Input



Textual
Syntactical
Semantical
Historical
Annotated
Metadata, …

Language-Specific Frontends

Construct CFG

Liveness, Pentagons, …

(Depends on Frontend)

Theory

Abstract Domains

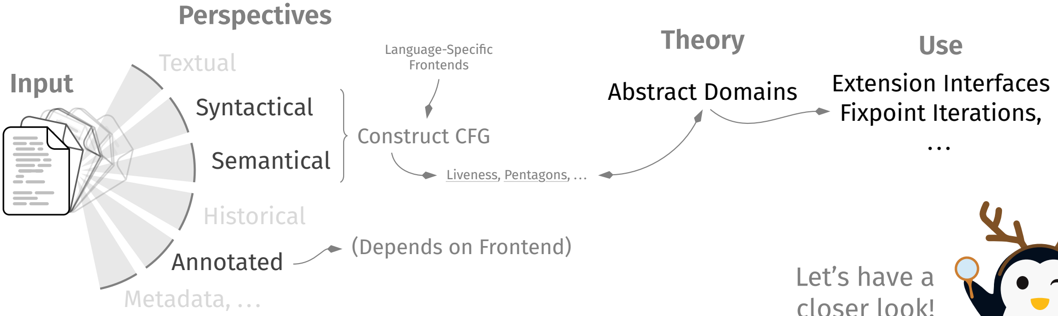
Use

Extension Interfaces
Fixpoint Iterations,
…

github.com/lisa-analyzer/lisa

LiSA

- (Largely) Language Independent Library for Static Analysis — Similar to Mopsa or Apron
- Custom frontends for Rust, Go, Python, EVM, …



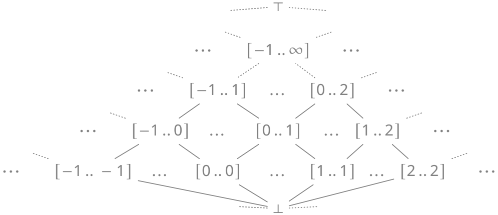
Let's have a closer look!

github.com/lisa-analyzer/lisa

LiSA — Interval Analysis[Cou21, p. 389]

[Cou21] “Principles of Abstract Interpretation” (Cousot)

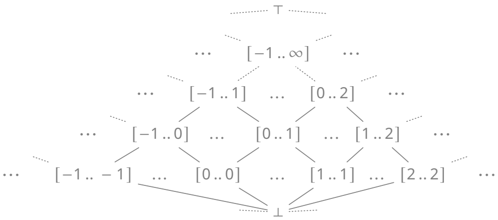
LiSA — Interval Analysis[Cou21, p. 389]



[Cou21] "Principles of Abstract Interpretation" (Cousot)

LiSA — Interval Analysis[Cou21, p. 389]

$\top = [-\infty \,..\, \infty]$ Top

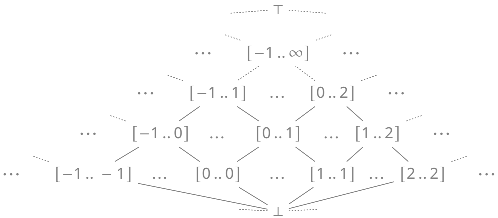


[Cou21] “Principles of Abstract Interpretation” (Cousot)

LiSA — Interval Analysis[Cou21, p. 389]

$\top = [-\infty\,..\,\infty]$ Top

$\bot = \emptyset$ Bottom



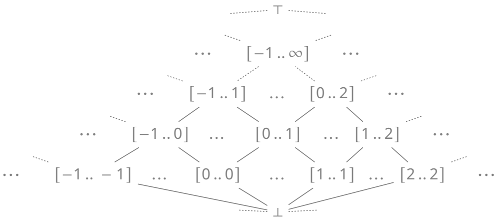
[Cou21] "Principles of Abstract Interpretation" (Cousot)

LiSA — Interval Analysis[Cou21, p. 389]

$$\top = [-\infty \mathinner{..} \infty] \qquad \text{Top}$$

$$\bot = \varnothing \qquad \text{Bottom}$$

$$\bigsqcup_k [\ell_k \mathinner{..} h_k] = [\min(\ell_k) \mathinner{..} \max(h_k)] \qquad \text{Join}$$



[Cou21] "Principles of Abstract Interpretation" (Cousot)

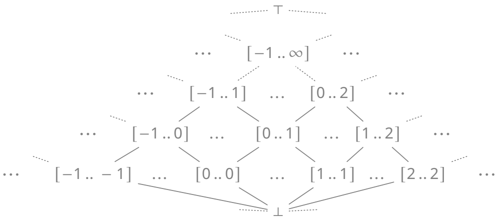
LiSA — Interval Analysis[Cou21, p. 389]

$$\top = [-\infty \mathinner{..} \infty] \qquad \text{Top}$$

$$\bot = \varnothing \qquad \text{Bottom}$$

$$\bigsqcup_k [\ell_k \mathinner{..} h_k] = [\min(\ell_k) \mathinner{..} \max(h_k)] \qquad \text{Join}$$

$$\bigsqcap_k [\ell_k \mathinner{..} h_k] = [\max(\ell_k) \mathinner{..} \min(h_k)] \qquad \text{Meet}$$



[Cou21] "Principles of Abstract Interpretation" (Cousot)

LiSA — Interval Analysis[Cou21, p. 389]

lisa-analyses/src/main/java/it/unive/lisa/analysis/numeric/Interval.java (simplified)

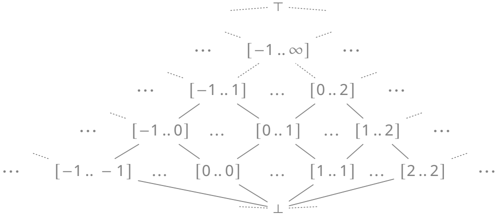
```java
Interval TOP = new Interval(IntInterval.INFINITY);
```

$$\top = [-\infty \mathinner{..} \infty] \qquad \text{Top}$$

$$\bot = \varnothing \qquad \text{Bottom}$$

$$\bigsqcup_k [\ell_k \mathinner{..} h_k] = [\min(\ell_k) \mathinner{..} \max(h_k)] \qquad \text{Join}$$

$$\bigsqcap_k [\ell_k \mathinner{..} h_k] = [\max(\ell_k) \mathinner{..} \min(h_k)] \qquad \text{Meet}$$



[Cou21] "Principles of Abstract Interpretation" (Cousot)

LiSA — Interval Analysis[Cou21, p. 389]

lisa-analyses/src/main/java/it/unive/lisa/analysis/numeric/Interval.java (simplified)

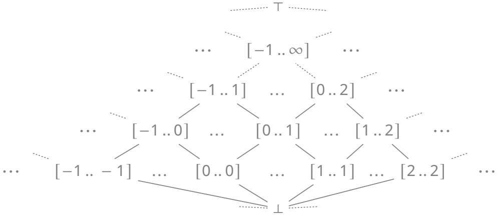
```
Interval TOP = new Interval(IntInterval.INFINITY);
Interval BOTTOM = new Interval(null);
```

$$\top = [-\infty \,..\, \infty] \qquad \text{Top}$$

$$\bot = \varnothing \qquad \text{Bottom}$$

$$\bigsqcup_k [\ell_k \,..\, h_k] = [\min(\ell_k) \,..\, \max(h_k)] \qquad \text{Join}$$

$$\bigsqcap_k [\ell_k \,..\, h_k] = [\max(\ell_k) \,..\, \min(h_k)] \qquad \text{Meet}$$



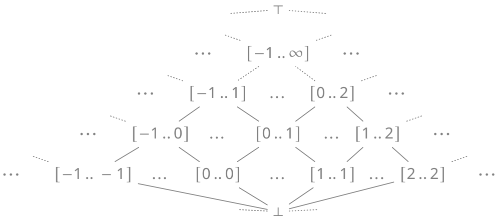
[Cou21] "Principles of Abstract Interpretation" (Cousot)

LiSA — Interval Analysis[Cou21, p. 389]

lisa-analyses/src/main/java/it/unive/lisa/analysis/numeric/Interval.java (simplified)

```java
Interval TOP = new Interval(IntInterval.INFINITY);
Interval BOTTOM = new Interval(null);

public Interval lubAux(Interval other) {
  var newL = getLow().min(other.getLow());
  var newH = getHigh().max(other.getHigh());
  return new Interval(newLow, newHigh);
}
```

$\top = [-\infty .. \infty]$	Top
$\bot = \emptyset$	Bottom
$\bigsqcup_k [\ell_k .. h_k] = [\min(\ell_k) .. \max(h_k)]$	Join
$\bigsqcap_k [\ell_k .. h_k] = [\max(\ell_k) .. \min(h_k)]$	Meet



[Cou21] "Principles of Abstract Interpretation" (Cousot)

LiSA — Interval Analysis[Cou21, p. 389]

lisa-analyses/src/main/java/it/unive/lisa/analysis/numeric/Interval.java (simplified)

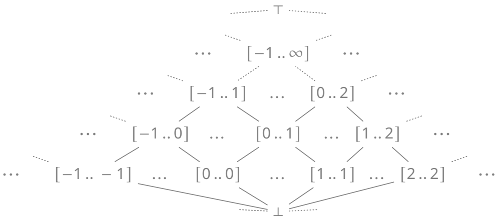
```java
Interval TOP = new Interval(IntInterval.INFINITY);
Interval BOTTOM = new Interval(null);

public Interval lubAux(Interval other) {
  var newL = getLow().min(other.getLow());
  var newH = getHigh().max(other.getHigh());
  return new Interval(newLow, newHigh);
}

public Interval glbAux(Interval other) {
  var newL = getLow().max(other.getLow());
  var newH = getHigh().min(other.getHigh());

  return new Interval(newLow, newHigh);
}
```

$\top = [-\infty \mathinner{..} \infty]$ Top

$\bot = \varnothing$ Bottom

$\bigsqcup_k [\ell_k \mathinner{..} h_k] = [\min(\ell_k) \mathinner{..} \max(h_k)]$ Join

$\bigsqcap_k [\ell_k \mathinner{..} h_k] = [\max(\ell_k) \mathinner{..} \min(h_k)]$ Meet



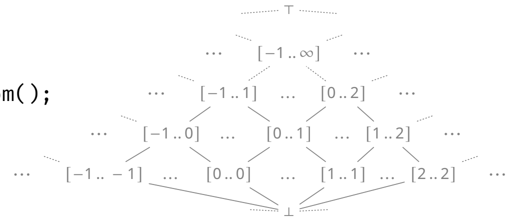
[Cou21] “Principles of Abstract Interpretation” (Cousot)

LiSA — Interval Analysis[Cou21, p. 389]

lisa-analyses/src/main/java/it/unive/lisa/analysis/numeric/Interval.java (simplified)

```java
Interval TOP = new Interval(IntInterval.INFINITY);
Interval BOTTOM = new Interval(null);

public Interval lubAux(Interval other) {
  var newL = getLow().min(other.getLow());
  var newH = getHigh().max(other.getHigh());
  return new Interval(newLow, newHigh);
}

public Interval glbAux(Interval other) {
  var newL = getLow().max(other.getLow());
  var newH = getHigh().min(other.getHigh());
  if(newLow.compareTo(newHigh) > 0) return bottom();
  return new Interval(newLow, newHigh);
}
```

$\top = [-\infty \mathinner{..} \infty]$	Top
$\bot = \varnothing$	Bottom
$\bigsqcup_k [\ell_k \mathinner{..} h_k] = [\min(\ell_k) \mathinner{..} \max(h_k)]$	Join
$\bigsqcap_k [\ell_k \mathinner{..} h_k] = [\max(\ell_k) \mathinner{..} \min(h_k)]$	Meet



[Cou21] "Principles of Abstract Interpretation" (Cousot)

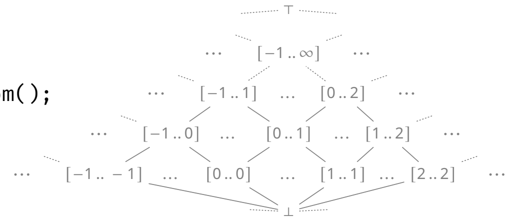
LiSA — Interval Analysis[Cou21, p. 389]

lisa-analyses/src/main/java/it/unive/lisa/analysis/numeric/Interval.java (simplified)

```java
Interval TOP = new Interval(IntInterval.INFINITY);
Interval BOTTOM = new Interval(null);

public Interval lubAux(Interval other) {
  var newL = getLow().min(other.getLow());
  var newH = getHigh().max(other.getHigh());
  return new Interval(newLow, newHigh);
}

public Interval glbAux(Interval other) {
  var newL = getLow().max(other.getLow());
  var newH = getHigh().min(other.getHigh());
  if(newLow.compareTo(newHigh) > 0) return bottom();
  return new Interval(newLow, newHigh);
}
```

Widening, Narrowing, Assume, Satisfies, …

$\top = [-\infty \mathinner{..} \infty]$	Top
$\bot = \varnothing$	Bottom
$\bigsqcup_k [\ell_k \mathinner{..} h_k] = [\min(\ell_k) \mathinner{..} \max(h_k)]$	Join
$\bigsqcap_k [\ell_k \mathinner{..} h_k] = [\max(\ell_k) \mathinner{..} \min(h_k)]$	Meet



[Cou21] “Principles of Abstract Interpretation” (Cousot)

LiSA — Interval Analysis[Cou21, p. 389]

lisa-analyses/src/main/java/it/unive/lisa/analysis/numeric/Interval.java (simplified)

```java
Interval TOP = new Interval(IntInterval.INFINITY);
Interval BOTTOM = new Interval(null);

public Interval lubAux(Interval other) {
  var newL = getLow().min(other.getLow());
  var newH = getHigh().max(other.getHigh());
  return new Interval(newLow, newHigh);
}

public Interval glbAux(Interval other) {
  var newL = getLow().max(other.getLow());
  var newH = getHigh().min(other.getHigh());
  if(newLow.compareTo(newHigh) > 0) return bottom();
  return new Interval(newLow, newHigh);
}
```

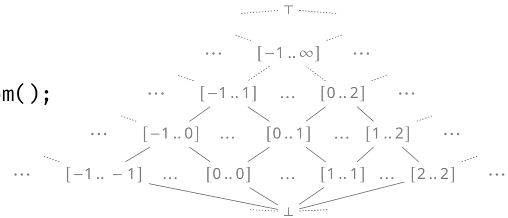
Widening, Narrowing, Assume, Satisfies, …

$\top = [-\infty \mathinner{..} \infty]$ Top

$\bot = \emptyset$ Bottom

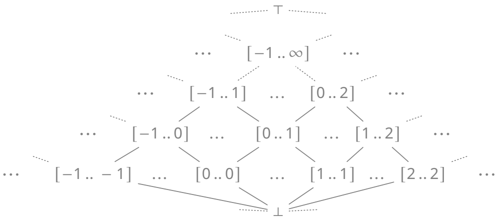
$\bigsqcup_k [\ell_k \mathinner{..} h_k] = [\min(\ell_k) \mathinner{..} \max(h_k)]$ Join

$\bigsqcap_k [\ell_k \mathinner{..} h_k] = [\max(\ell_k) \mathinner{..} \min(h_k)]$ Meet



[Cou21] "Principles of Abstract Interpretation" (Cousot)

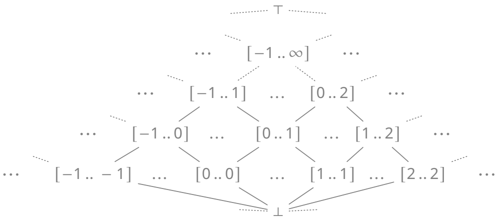
LiSA — Interval Analysis[Cou21, p. 389] Semantics



[Cou21] "Principles of Abstract Interpretation" (Cousot)

LiSA — Interval Analysis[Cou21, p. 389] Semantics

When to create which interval?



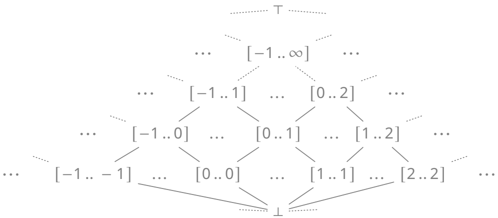
[Cou21] “Principles of Abstract Interpretation” (Cousot)

LiSA — Interval Analysis[Cou21, p. 389] Semantics

When to create which interval?

lisa-analyses/src/main/java/it/unive/lisa/analysis/numeric/Interval.java (simplified)

```
public Interval evalNonNullConstant(Constant constant,
    ProgramPoint pp, SemanticOracle oracle) {






}
```



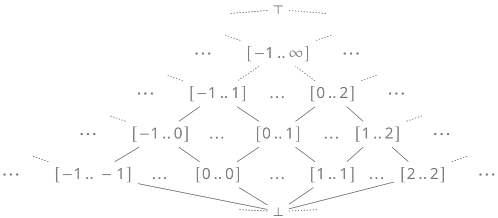
[Cou21] “Principles of Abstract Interpretation” (Cousot)

LiSA — Interval Analysis[Cou21, p. 389] Semantics

When to create which interval?

lisa-analyses/src/main/java/it/unive/lisa/analysis/numeric/Interval.java (simplified)

```
public Interval evalNonNullConstant(Constant constant,
    ProgramPoint pp, SemanticOracle oracle) {
  if(constant.getValue() instanceof Integer) {
    var i = (Integer) constant.getValue();
    return new Interval(i, i);
  }

}
```



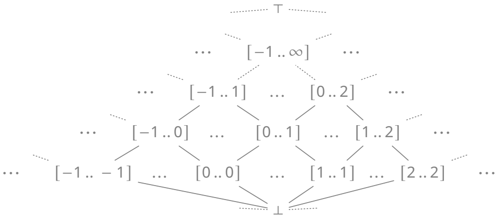
[Cou21] “Principles of Abstract Interpretation” (Cousot)

LiSA — Interval Analysis[Cou21, p. 389] Semantics

When to create which interval?

lisa-analyses/src/main/java/it/unive/lisa/analysis/numeric/Interval.java (simplified)

```java
public Interval evalNonNullConstant(Constant constant,
      ProgramPoint pp, SemanticOracle oracle) {
  if(constant.getValue() instanceof Integer) {
    var i = (Integer) constant.getValue();
    return new Interval(i, i);
  }
  return top();
}
```



[Cou21] “Principles of Abstract Interpretation” (Cousot)

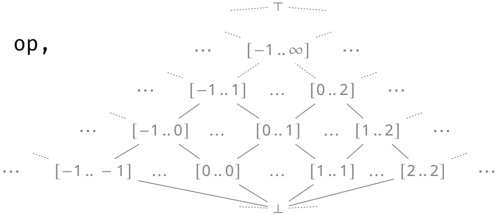
LiSA — Interval Analysis[Cou21, p. 389] Semantics

When to create which interval?

lisa-analyses/src/main/java/it/unive/lisa/analysis/numeric/Interval.java (simplified)

```
public Interval evalNonNullConstant(Constant constant,
    ProgramPoint pp, SemanticOracle oracle) {
  if(constant.getValue() instanceof Integer) {
    var i = (Integer) constant.getValue();
    return new Interval(i, i);
  }
  return top();
}
```

```
public Interval evalUnaryExpression(UnaryOperator op,
  Interval arg, ...) { ... }
```



[Cou21] “Principles of Abstract Interpretation” (Cousot)

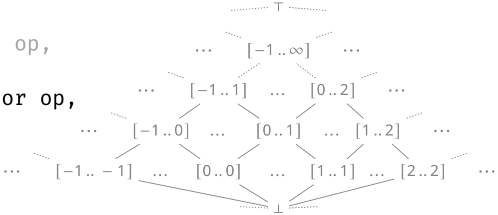
LiSA — Interval Analysis[Cou21, p. 389]

Semantics

When to create which interval?

lisa-analyses/src/main/java/it/unive/lisa/analysis/numeric/Interval.java (simplified)

```
public Interval evalNonNullConstant(Constant constant,
      ProgramPoint pp, SemanticOracle oracle) {
   if(constant.getValue() instanceof Integer) {
      var i = (Integer) constant.getValue();
      return new Interval(i, i);
   }
   return top();
}

public Interval evalUnaryExpression(UnaryOperator op,
    Interval arg, ...) { ... }
public Interval evalBinaryExpression(BinaryOperator op,
    Interval left, Interval right, ...) { ... }
```



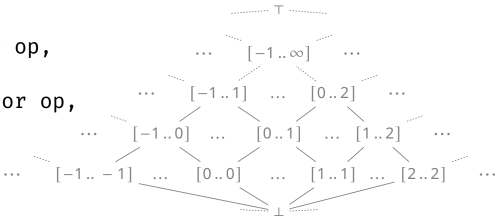
[Cou21] “Principles of Abstract Interpretation” (Cousot)

LiSA — Interval Analysis[Cou21, p. 389] Semantics

When to create which interval?

lisa-analyses/src/main/java/it/unive/lisa/analysis/numeric/Interval.java (simplified)

```
public Interval evalNonNullConstant(Constant constant,
      ProgramPoint pp, SemanticOracle oracle) {
   if(constant.getValue() instanceof Integer) {
      var i = (Integer) constant.getValue();
      return new Interval(i, i);
   }
   return top();
}

public Interval evalUnaryExpression(UnaryOperator op,
    Interval arg, ...) { ... }
public Interval evalBinaryExpression(BinaryOperator op,
    Interval left, Interval right, ...) { ... }
```



[Cou21] “Principles of Abstract Interpretation” (Cousot)

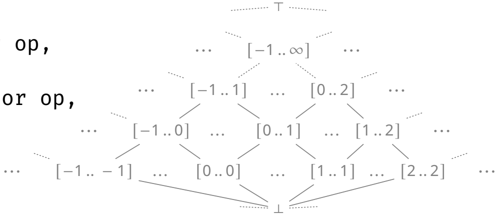
LiSA — Interval Analysis[Cou21, p. 389] Semantics

When to create which interval?

lisa-analyses/src/main/java/it/unive/lisa/analysis/numeric/Interval.java (simplified)

```java
public Interval evalNonNullConstant(Constant constant,
    ProgramPoint pp, SemanticOracle oracle) {
  if(constant.getValue() instanceof Integer) {
    var i = (Integer) constant.getValue();
    return new Interval(i, i);
  }
  return top();
}

public Interval evalUnaryExpression(UnaryOperator op,
   Interval arg, ...) { ... }
public Interval evalBinaryExpression(BinaryOperator op,
   Interval left, Interval right, ...) { ... }
```

Fold-like Evaluation



[Cou21] “Principles of Abstract Interpretation” (Cousot)

Let's Look at Tools

SootUp

flowR

LiquidHaskell

AbsInt

lintr

Teamscale

Infer

LiSA

CodeQL

sonar

Clang

GCC

ESLint

MOPSA analyzer

frama C

There are countless…

github.com/analysis-tools-dev/static-analysis

Let's Look at Tools

- SonarLint
- Astrée (sound!)
- LiSA

- **Java Language Server**
- lintr
- flowR

There are countless…

github.com/analysis-tools-dev/static-analysis

Java Language Server

github.com/eclipse-jdtls/eclipse.jdt.ls

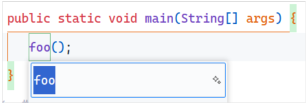
Java Language Server

- Uses the Language Server Protocol to provide static analysis for Java

github.com/eclipse-jdtls/eclipse.jdt.ls

Java Language Server

- Uses the Language Server Protocol to provide static analysis for Java



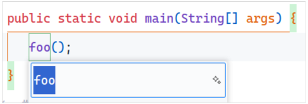
Rename Refactoring

github.com/eclipse-jdtls/eclipse.jdt.ls

Java Language Server

- Uses the Language Server Protocol to provide static analysis for Java

Code Actions



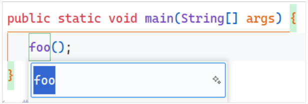
Rename Refactoring

github.com/eclipse-jdtls/eclipse.jdt.ls

Java Language Server

- Uses the Language Server Protocol to provide static analysis for Java

Code Actions



Rename Refactoring

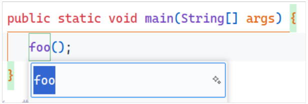
...

github.com/eclipse-jdtls/eclipse.jdt.ls

Java Language Server

- Uses the Language Server Protocol to provide static analysis for Java

Code Actions



Rename Refactoring

...

- Relies on the Eclipse JDT Language Server

github.com/eclipse-jdtls/eclipse.jdt.ls

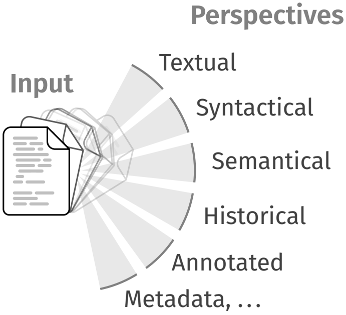
Java Language Server

- Uses the Language Server Protocol to provide static analysis for Java
  Renaming, Code Actions, ...
- Relies on the Eclipse JDT Language Server

github.com/eclipse-jdtls/eclipse.jdt.ls

Java Language Server

- Uses the Language Server Protocol to provide static analysis for Java
  Renaming, Code Actions, …
- Relies on the Eclipse JDT Language Server

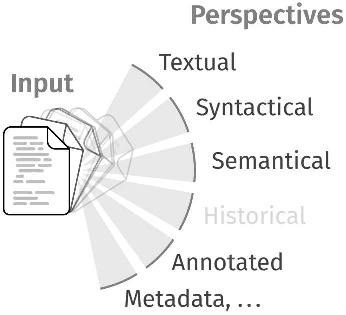


github.com/eclipse-jdtls/eclipse.jdt.ls

Java Language Server

- Uses the Language Server Protocol to provide static analysis for Java
  Renaming, Code Actions, ...

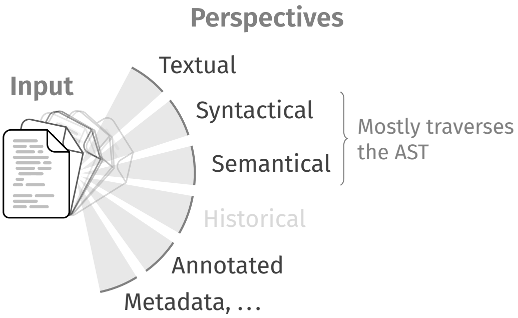
- Relies on the Eclipse JDT Language Server



github.com/eclipse-jdtls/eclipse.jdt.ls

Java Language Server

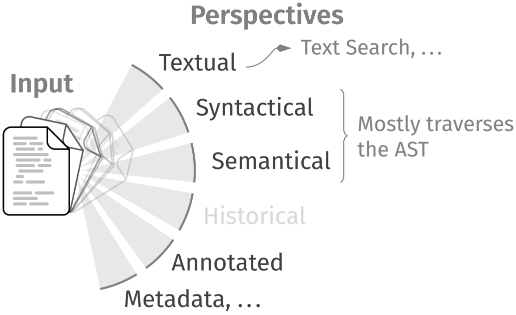
- Uses the Language Server Protocol to provide static analysis for Java
  Renaming, Code Actions, ...
- Relies on the Eclipse JDT Language Server



github.com/eclipse-jdtls/eclipse.jdt.ls

Java Language Server

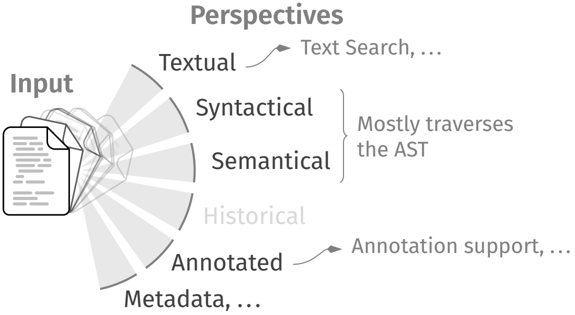
- Uses the Language Server Protocol to provide static analysis for Java
  Renaming, Code Actions, ...
- Relies on the Eclipse JDT Language Server



github.com/eclipse-jdtls/eclipse.jdt.ls

Java Language Server

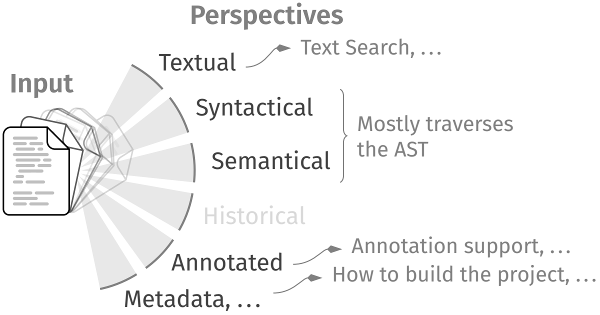
- Uses the Language Server Protocol to provide static analysis for Java
  Renaming, Code Actions, ...
- Relies on the Eclipse JDT Language Server



github.com/eclipse-jdtls/eclipse.jdt.ls

Java Language Server

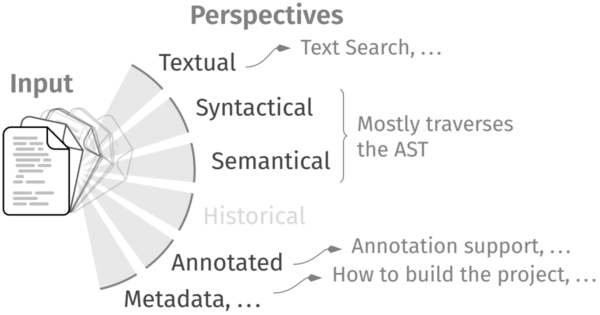
- Uses the Language Server Protocol to provide static analysis for Java
  Renaming, Code Actions, ...
- Relies on the Eclipse JDT Language Server



github.com/eclipse-jdtls/eclipse.jdt.ls

Java Language Server

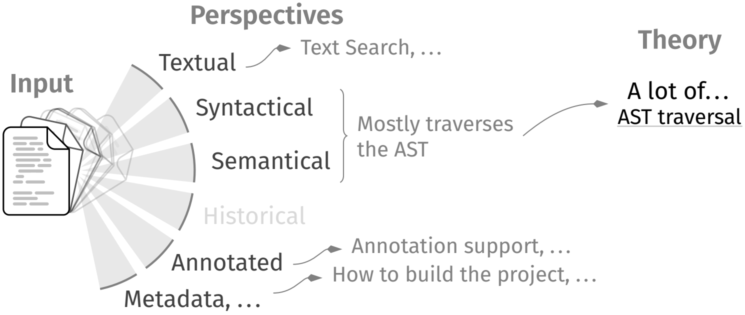
- Uses the Language Server Protocol to provide static analysis for Java
  Renaming, Code Actions, . . .
- Relies on the Eclipse JDT Language Server



github.com/eclipse-jdtls/eclipse.jdt.ls

Java Language Server

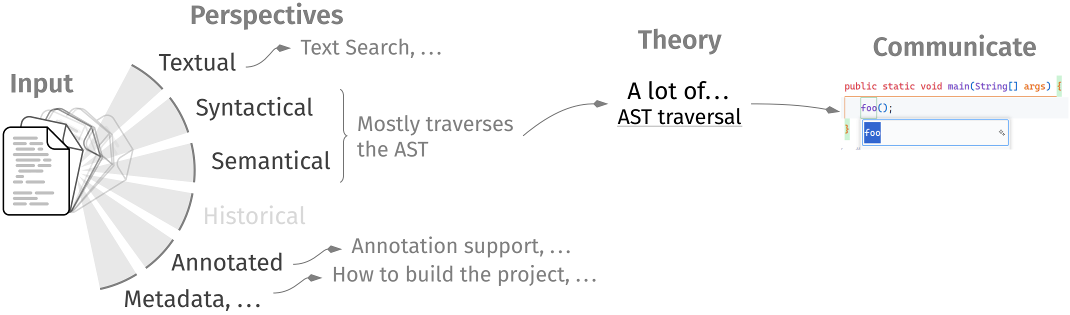
- Uses the Language Server Protocol to provide static analysis for Java
  Renaming, Code Actions, ...
- Relies on the Eclipse JDT Language Server



github.com/eclipse-jdtls/eclipse.jdt.ls

Java Language Server

- Uses the Language Server Protocol to provide static analysis for Java
  Renaming, Code Actions, ...
- Relies on the Eclipse JDT Language Server



github.com/eclipse-jdtls/eclipse.jdt.ls

Let’s Look at Tools

SootUp

flowR

LiquidHaskell

AbsInt

lintr

Teamscale

Infer

LiSA

CodeQL

sonar

Clang

GCC

ESLint

MOPSA analyzer

frama C

There are countless…

github.com/analysis-tools-dev/static-analysis

Let's Look at Tools

- SonarLint
- Astrée (sound!)
- LiSA
- Java Language Server
- **lintr**
- flowR

There are countless…

github.com/analysis-tools-dev/static-analysis

lintr

github.com/r-lib/lintr

[Flü+19] “R melts brains: an IR for first-class environments and lazy effectful arguments” (Flückiger et al.)

lintr

- A linter for the R programming language

github.com/r-lib/lintr

[Flü+19] "R melts brains: an IR for first-class environments and lazy effectful arguments" (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

github.com/r-lib/lintr

[Flü+19] “R melts brains: an IR for first-class environments and lazy effectful arguments” (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```r
x <- 4
```

github.com/r-lib/lintr

[Flü+19] “R melts brains: an IR for first-class environments and lazy effectful arguments” (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```r
x <- 4
f <- function() x
```

github.com/r-lib/lintr

[Flü+19] “R melts brains: an IR for first-class environments and lazy effectful arguments” (Flückiger et al.)

lintr

Why is this... special?

- A linter for the R programming language

```r
x <- 4
f <- function() x
f()
```

github.com/r-lib/lintr

[Flü+19] "R melts brains: an IR for first-class environments and lazy effectful arguments" (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```r
x <- 4
f <- function() x
f() # 4
```

github.com/r-lib/lintr

[Flü+19] “R melts brains: an IR for first-class environments and lazy effectful arguments” (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```
x <- 4
f <- function() x


f()
```

github.com/r-lib/lintr

[Flü+19] "R melts brains: an IR for first-class environments and lazy effectful arguments" (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```r
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f()
```

github.com/r-lib/lintr

[Flü+19] "R melts brains: an IR for first-class environments and lazy effectful arguments" (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

github.com/r-lib/lintr

[Flü+19] "R melts brains: an IR for first-class environments and lazy effectful arguments" (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```r
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```r
if(TRUE) print(3) # 3
```

github.com/r-lib/lintr

[Flü+19] “R melts brains: an IR for first-class environments and lazy effectful arguments” (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```
'if' <- function(...) 42
if(TRUE) print(3) # 42
```

github.com/r-lib/lintr

[Flü+19] "R melts brains: an IR for first-class environments and lazy effectful arguments" (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```r
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```r
`if` <- function(...) 42
if(TRUE) print(3) # 42
```

```r
x <- 2

x <- 21
```

github.com/r-lib/lintr

[Flü+19] "R melts brains: an IR for first-class environments and lazy effectful arguments" (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```
‘if‘ <- function(...) 42
if(TRUE) print(3) # 42
```

```
x <- 2
‘<-‘ <- ‘*‘
x <- 21 # 42
```

github.com/r-lib/lintr

[Flü+19] “R melts brains: an IR for first-class environments and lazy effectful arguments” (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```r
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```r
`if` <- function(...) 42
if(TRUE) print(3) # 42
```

```r
x <- 2
`<-` <- `*`
x <- 21 # 42
```

- Common static analysis strategies have their… problems with R

github.com/r-lib/lintr

[Flü+19] "R melts brains: an IR for first-class environments and lazy effectful arguments" (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```
‘if‘ <- function(...) 42
if(TRUE) print(3) # 42
```

```
x <- 2
‘<-‘ <- ‘*‘
x <- 21 # 42
```

- Common static analysis strategies have their… problems with R
- Most of R’s users are no computer scientists

github.com/r-lib/lintr

[Flü+19] “R melts brains: an IR for first-class environments and lazy effectful arguments” (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```r
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```r
`if` <- function(...) 42
if(TRUE) print(3) # 42
```

```r
x <- 2
`<-` <- `*`
x <- 21 # 42
```

- Common static analysis strategies have their… problems with R
- Most of R's users are no computer scientists (just a small set of existing work)

github.com/r-lib/lintr

[Flü+19] "R melts brains: an IR for first-class environments and lazy effectful arguments" (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```r
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```r
`if` <- function(...) 42
if(TRUE) print(3) # 42
```

```r
x <- 2
`<-` <- `*`
x <- 21 # 42
```

- Common static analysis strategies have their… problems with R
- Most of R’s users are no computer scientists (just a small set of existing work)
- So… how does *lintr* do it?

github.com/r-lib/lintr

[Flü+19] “R melts brains: an IR for first-class environments and lazy effectful arguments” (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```r
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```r
`if` <- function(...) 42
if(TRUE) print(3) # 42
```

```r
x <- 2
`<-` <- `*`
x <- 21 # 42
```

- Common static analysis strategies have their… problems with R
- Most of R's users are no computer scientists (just a small set of existing work)
- So… how does *lintr* do it?
  - Dataflow Constraints?

github.com/r-lib/lintr

[Flü+19] "R melts brains: an IR for first-class environments and lazy effectful arguments" (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```
‘if‘ <- function(...) 42
if(TRUE) print(3) # 42
```

```
x <- 2
‘<-‘ <- ‘*‘
x <- 21 # 42
```

- Common static analysis strategies have their… problems with R
- Most of R’s users are no computer scientists (just a small set of existing work)
- So… how does *lintr* do it?
  - ~~Dataflow Constraints?~~

github.com/r-lib/lintr

[Flü+19] “R melts brains: an IR for first-class environments and lazy effectful arguments” (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```r
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```r
`if` <- function(...) 42
if(TRUE) print(3) # 42
```

```r
x <- 2
`<-` <- `*`
x <- 21 # 42
```

- Common static analysis strategies have their… problems with R
- Most of R's users are no computer scientists (just a small set of existing work)
- So… how does *lintr* do it?
  - ~~Dataflow Constraints?~~
  - Abstract Domains?

github.com/r-lib/lintr

[Flü+19] "R melts brains: an IR for first-class environments and lazy effectful arguments" (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```r
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```r
`if` <- function(...) 42
if(TRUE) print(3) # 42
```

```r
x <- 2
`<-` <- `*`
x <- 21 # 42
```

- Common static analysis strategies have their… problems with R
- Most of R's users are no computer scientists (just a small set of existing work)
- So… how does *lintr* do it?
  - ~~Dataflow Constraints?~~
  - ~~Abstract Domains?~~

github.com/r-lib/lintr

[Flü+19] "R melts brains: an IR for first-class environments and lazy effectful arguments" (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```r
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```r
`if` <- function(...) 42
if(TRUE) print(3) # 42
```

```r
x <- 2
`<-` <- `*`
x <- 21 # 42
```

- Common static analysis strategies have their… problems with R
- Most of R's users are no computer scientists (just a small set of existing work)
- So… how does *lintr* do it?
  - ~~Dataflow Constraints?~~
  - ~~Abstract Domains?~~
  - Control Flow Graphs?

github.com/r-lib/lintr

[Flü+19] "R melts brains: an IR for first-class environments and lazy effectful arguments" (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```r
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```r
`if` <- function(...) 42
if(TRUE) print(3) # 42
```

```r
x <- 2
`<-` <- `*`
x <- 21 # 42
```

- Common static analysis strategies have their… problems with R
- Most of R's users are no computer scientists (just a small set of existing work)
- So… how does *lintr* do it?
  - ~~Dataflow Constraints?~~
  - ~~Abstract Domains?~~
  - ~~Control Flow Graphs?~~

github.com/r-lib/lintr

[Flü+19] "R melts brains: an IR for first-class environments and lazy effectful arguments" (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```r
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```r
`if` <- function(...) 42
if(TRUE) print(3) # 42
```

```r
x <- 2
`<-` <- `*`
x <- 21 # 42
```

- Common static analysis strategies have their… problems with R
- Most of R's users are no computer scientists (just a small set of existing work)
- So… how does *lintr* do it?
  - ~~Dataflow Constraints?~~
  - ~~Abstract Domains?~~
  - ~~Control Flow Graphs?~~
  - AST Traversal?

github.com/r-lib/lintr

[Flü+19] "R melts brains: an IR for first-class environments and lazy effectful arguments" (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```r
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```r
`if` <- function(...) 42
if(TRUE) print(3) # 42
```

```r
x <- 2
`<-` <- `*`
x <- 21 # 42
```

- Common static analysis strategies have their… problems with R
- Most of R's users are no computer scientists (just a small set of existing work)
- So… how does *lintr* do it?
  - ~~Dataflow Constraints?~~
  - ~~Abstract Domains?~~
  - ~~Control Flow Graphs?~~
  - AST ~~Traversal?~~

github.com/r-lib/lintr

[Flü+19] "R melts brains: an IR for first-class environments and lazy effectful arguments" (Flückiger et al.)

lintr

Why is this… special?

- A linter for the R programming language

```r
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```r
`if` <- function(...) 42
if(TRUE) print(3) # 42
```

```r
x <- 2
`<-` <- `*`
x <- 21 # 42
```

- Common static analysis strategies have their… problems with R
- Most of R's users are no computer scientists (just a small set of existing work)
- So… how does *lintr* do it?
  - ~~Dataflow Constraints?~~
  - ~~Abstract Domains?~~
  - ~~Control Flow Graphs?~~
  - AST ~~Traversal?~~ (mostly) Pattern Matching and Evaluation!

github.com/r-lib/lintr

[Flü+19] "R melts brains: an IR for first-class environments and lazy effectful arguments" (Flückiger et al.)

lintr — Under the Hood

≈ 100 rules

lintr — Under the Hood

≈ 100 rules

```
magic <- "
   expr[LEFT_ASSIGN or EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | expr_or_assign_or_help[EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | equal_assign[EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | //SYMBOL_FUNCTION_CALL[text() = 'assign']/parent::expr/following-sibling::
       expr[2][FUNCTION or OP-LAMBDA]
   | //SYMBOL_FUNCTION_CALL[text() = 'setMethod']/parent::expr/following-sibling::
       expr[3][FUNCTION or OP-LAMBDA]"
```

lintr — Under the Hood

≈ 100 rules

```
magic <- "
   expr[LEFT_ASSIGN or EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | expr_or_assign_or_help[EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | equal_assign[EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | //SYMBOL_FUNCTION_CALL[text() = 'assign']/parent::expr/following-sibling::
       expr[2][FUNCTION or OP-LAMBDA]
   | //SYMBOL_FUNCTION_CALL[text() = 'setMethod']/parent::expr/following-sibling::
       expr[3][FUNCTION or OP-LAMBDA]"
```

- What does this do?

lintr — Under the Hood

≈ 100 rules

R/object_usage_linter.R

```
magic <- "
   expr[LEFT_ASSIGN or EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | expr_or_assign_or_help[EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | equal_assign[EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | //SYMBOL_FUNCTION_CALL[text() = 'assign']/parent::expr/following-sibling::
       expr[2][FUNCTION or OP-LAMBDA]
   | //SYMBOL_FUNCTION_CALL[text() = 'setMethod']/parent::expr/following-sibling::
       expr[3][FUNCTION or OP-LAMBDA]"
```

- What does this do?
  This XPATH expression matches assignments to functions

lintr — Under the Hood

≈ 100 rules

R/object_usage_linter.R

```
magic <- "
   expr[LEFT_ASSIGN or EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | expr_or_assign_or_help[EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | equal_assign[EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | //SYMBOL_FUNCTION_CALL[text() = 'assign']/parent::expr/following-sibling::
       expr[2][FUNCTION or OP-LAMBDA]
   | //SYMBOL_FUNCTION_CALL[text() = 'setMethod']/parent::expr/following-sibling::
       expr[3][FUNCTION or OP-LAMBDA]"
```

- What does this do?
  This XPATH expression matches assignments to functions
- It is very rigid (no alias tracking, flow sensitivity, …)

lintr — Under the Hood

≈ 100 rules

R/object_usage_linter.R

```
magic <- "
   expr[LEFT_ASSIGN or EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | expr_or_assign_or_help[EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | equal_assign[EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | //SYMBOL_FUNCTION_CALL[text() = 'assign']/parent::expr/following-sibling::
       expr[2][FUNCTION or OP-LAMBDA]
   | //SYMBOL_FUNCTION_CALL[text() = 'setMethod']/parent::expr/following-sibling::
       expr[3][FUNCTION or OP-LAMBDA]"
```

- What does this do?
  This XPATH expression matches assignments to functions
- It is very rigid (no alias tracking, flow sensitivity, ...)
- And it...

lintr — Under the Hood

≈ 100 rules

R/object_usage_linter.R

```
magic <- "
   expr[LEFT_ASSIGN or EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | expr_or_assign_or_help[EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | equal_assign[EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | //SYMBOL_FUNCTION_CALL[text() = 'assign']/parent::expr/following-sibling::
       expr[2][FUNCTION or OP-LAMBDA]
   | //SYMBOL_FUNCTION_CALL[text() = 'setMethod']/parent::expr/following-sibling::
       expr[3][FUNCTION or OP-LAMBDA]"
```

- What does this do?
  This XPATH expression matches assignments to functions
- It is very rigid (no alias tracking, flow sensitivity, …)
- And it… cheats:

lintr — Under the Hood

≈ 100 rules

R/object_usage_linter.R

```
magic <- "
   expr[LEFT_ASSIGN or EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | expr_or_assign_or_help[EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | equal_assign[EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | //SYMBOL_FUNCTION_CALL[text() = 'assign']/parent::expr/following-sibling::
       expr[2][FUNCTION or OP-LAMBDA]
   | //SYMBOL_FUNCTION_CALL[text() = 'setMethod']/parent::expr/following-sibling::
       expr[3][FUNCTION or OP-LAMBDA]"
```

- What does this do?
  This XPATH expression matches assignments to functions
- It is very rigid (no alias tracking, flow sensitivity, …)
- And it… cheats:
  `try_silently(eval(envir = env,parse(text = code, keep.source = TRUE)))`

lintr — Under the Hood

≈ 100 rules

R/object_usage_linter.R

```
magic <- "
   expr[LEFT_ASSIGN or EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | expr_or_assign_or_help[EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | equal_assign[EQ_ASSIGN]/expr[2][FUNCTION or OP-LAMBDA]
   | //SYMBOL_FUNCTION_CALL[text() = 'assign']/parent::expr/following-sibling::
       expr[2][FUNCTION or OP-LAMBDA]
   | //SYMBOL_FUNCTION_CALL[text() = 'setMethod']/parent::expr/following-sibling::
       expr[3][FUNCTION or OP-LAMBDA]"
```

- What does this do?
  This XPATH expression matches assignments to functions
- It is very rigid (no alias tracking, flow sensitivity, …)
- And it… cheats:
  `try_silently(eval(envir = env,parse(text = code, keep.source = TRUE)))`
  It simply runs (parts of) the program (including side-effects), …

Let's Look at Tools

SootUp

flowR

LiquidHaskell

AbsInt

lintr

Teamscale

Infer

LiSA

CodeQL

sonar

Clang

GCC

ESLint

MOPSA analyzer

frama C

There are countless…

github.com/analysis-tools-dev/static-analysis

Let's Look at Tools

- SonarLint
- Astrée (sound!)
- LiSA
- Java Language Server
- lintr
- **flowR**

There are countless…

github.com/analysis-tools-dev/static-analysis

Existing Work

Existing Work

		references	op. assignments	dyn. assignments	value trace	control flow	non-std. eval.	special operators	function calls	libraries	reflection	side effects	static scope	dynamic scope	type inference	pointer analysis	external files	pre-processors	hooks	FFI	expose info.	goal	method
	flowR	●	●	○	○	●	○	●	●	○	○	○	●	○			○				●	static analysis	AST visitor
[Cha23]	{checkglobals}	●	●	○			○	○	○	○			○							○		missing libs.	AST visitor
[Lan+23]	{CodeAnalysis}	●	●	○	○	●	○	○	○	○			●		○		○				●	static analysis	AST visitor
[Lan+18]	{CodeDepends}	●	●	○			○	○	○	○		○	●								●	static analysis	AST visitor
[Tie23]	{codetools}	●	●	○	○	○	○	○	○				●								●	static analysis	AST visitor
[Csa23]	{cyclocomp}					○							●									static analysis	AST visitor
[Rob16]	{DependenciesGraphs}								○	○											○	visualize	regex
[Kar21]	{dfgraph}	●	○			○			○												○	static analysis	AST visitor
[Fab23]	{flow}	●	○			●		○		○							○				○	vis., debug	AST visitor
[App24]	{foodwebr}								○												○	visualize	regex
[Ben22]	{globals}	●	●		○	○	○	○	○				●									distributed env.	AST visitor
[Lai23]	{languageserver}	●	●	○					○	○			●								○	editor support	XPath, visitor
[Hes+23]	{lintr}	●	●	○		○		○		○			●									linting	XPath, visitor
[Bra18]	{mvbutils}								○													utility	regex
[van23]	{PaRe}		○			○			○													code review	regex
[Pad21]	{pkgstats}	●	○					○	○				○									pkg. insights	ctags & gtags
[Lau22]	{Rclean}	●	●	○			○	○	○	○			●									debug/refactor	PDG traversal
[Rod21]	{rco}	●	●		●	○							●									optimization	AST visitor
[Pat19]	{rflowgraph}	●							○	○											○	call graph	AST visitor
[UT19]	{rstatic}	●	●		○	●		○					●								●	static analysis	AST visitor
[UT21]	{RTypeInference}	●	●		○	●		○					●		●						○	type inference	AST visitor
[BG20]	{SimilaR}	●	○			○			○				●									plagiarism	PDG, visitor
[Kal+14]	FastR	●	●		○			○	○	○			●									execute R	AST visitor
[R C23]	GNU R	●	●	○	●	○	○	●	○				●									execute R	bytecode
[19]	MRO	●	●	○	○	○																execute R	bytecode
[Nea14]	pqR	●	●	○	●	○	○	●	○				●									execute R	bytecode
[AS12]	Random	●	○	○	○	○				○			○									abstract int.	trace & visitor
[Gar04]	RCC	●	●		●	○	○		○			○	●									transpile to C	CFG, bytecode
[Ber13]	renjin	●	●	○	○	●		○	○			○	●		●							execute R	SSA, CFG
[Flü+20]	Ř	●	●	○	●	●	○	●	○				●	○	●							execute R	SSA, bytecode
[Sen+17]	ROSA	●	●			○			○		○		●		●	○						optimization	visitor
[Pos23]	RStudio	●	●	○					○	○			●									editor support	AST visitor

Existing Work

		references	op. assignments	dyn. assignments	value trace	control flow	non-std. eval.	special operators	function calls	libraries	reflection	side effects	static scope	dynamic scope	type inference	pointer analysis	external files	pre-processors	hooks	FFI	expose info.	goal	method
	flowR	●	●	○	○	●	○	●	●	○	○	○	●	○			○				●	static analysis	AST visitor
[Cha23]	{checkglobals}	●	●	○			○	○	○	○			○							○		missing libs.	AST visitor
[Lan+23]	{CodeAnalysis}	●	●	○	○	●	○	○	○	○			●		○		○				●	static analysis	AST visitor
[Lan+18]	{CodeDepends}	●	●	○			○	○	○	○		○	●								●	static analysis	AST visitor
[Tie23]	{codetools}	●	●	○	○	○	○	○	○				●								●	static analysis	AST visitor
[Csa23]	{cyclocomp}					○							●									static analysis	AST visitor
[Rob16]	{DependenciesGraphs}								○	○											○	visualize	regex
[Kar21]	{dfgraph}	●	○			○			○												○	static analysis	AST visitor
[Fab23]	{flow}	●	○			●		○		○							○				○	vis., debug	AST visitor
[App24]	{foodwebr}								○												○	visualize	regex
[Ben22]	{globals}	●	●		○	○	○	○	○				●									distributed env.	AST visitor
[Lai23]	{languageserver}	●	●	○					○	○			●								○	editor support	XPath, visitor
[Hes+23]	{lintr}	●	●	○		○		○		○			●									linting	XPath, visitor
[Bra18]	{mvbutils}								○													utility	regex
[van23]	{PaRe}		○			○			○													code review	regex
[Pad21]	{pkgstats}	●	○					○	○				○									pkg. insights	ctags & gtags
[Lau22]	{Rclean}	●	●	○			○	○	○	○			●									debug/refactor	PDG traversal
[Rod21]	{rco}	●	●		●	○							●									optimization	AST visitor
[Pat19]	{rflowgraph}	●							○	○											○	call graph	AST visitor
[UT19]	{rstatic}	●	●		○	●		○					●								●	static analysis	AST visitor
[UT21]	{RTypeInference}	●	●		○	●		○					●		●						○	type inference	AST visitor
[BG20]	{SimilaR}	●	○			○			○				●									plagiarism	PDG, visitor
[Kal+14]	FastR	●	●		○			○	○	○			●									execute R	AST visitor
[R C23]	GNU R	●	●	○	●	○	○	●	○				●									execute R	bytecode
[19]	MRO	●	●	○	○	○																execute R	bytecode
[Nea14]	pqR	●	●	○	●	○	○	●	○				●									execute R	bytecode
[AS12]	Random	●	○	○	○	○				○			○									abstract int.	trace & visitor
[Gar04]	RCC	●	●		●	○	○		○			○	●									transpile to C	CFG, bytecode
[Ber13]	renjin	●	●	○	○	●		○	○			○	●		●							execute R	SSA, CFG
[Flü+20]	Ř	●	●	○	●	●	○	●	○				●	○	●							execute R	SSA, bytecode
[Sen+17]	ROSA	●	●			○			○		○		●		●	○						optimization	visitor
[Pos23]	RStudio	●	●	○					○	○			●									editor support	AST visitor

Existing Work

Ref.	Tool	references	op. assignments	dyn. assignments	value trace	control flow	non-std. eval.	special operators	function calls	libraries	reflection	side effects	static scope	dynamic scope	type inference	pointer analysis	external files	pre-processors	hooks	FFI	expose info.	goal	method
	flowR	●	●	○	○	●	○	●	●	○	○	○	●	○			○				●	static analysis	AST visitor
[Cha23]	{checkglobals}	●	●	○			○	○	○	○			○							○		missing libs.	AST visitor
[Lan+23]	{CodeAnalysis}	●	●	○	○	●	○	○	○	○			●		○		○				●	static analysis	AST visitor
[Lan+18]	{CodeDepends}	●	●	○			○	○	○	○		○	●								●	static analysis	AST visitor
[Tie23]	{codetools}	●	●	○	○	○	○	○	○				●								●	static analysis	AST visitor
[Csa23]	{cyclocomp}					○							●									static analysis	AST visitor
[Rob16]	{DependenciesGraphs}								○	○											○	visualize	regex
[Kar21]	{dfgraph}	●	○			○			○												○	static analysis	AST visitor
[Fab23]	{flow}	●	○			●		○		○							○				○	vis., debug	AST visitor
[App24]	{foodwebr}								○												○	visualize	regex
[Ben22]	{globals}	●	●		○	○	○	○	○				●									distributed env.	AST visitor
[Lai23]	{languageserver}	●	●	○					○	○			●								○	editor support	XPath, visitor
[Hes+23]	{lintr}	●	●	○		○		○		○			●									linting	XPath, visitor
[Bra18]	{mvbutils}								○													utility	regex
[van23]	{PaRe}		○			○			○													code review	regex
[Pad21]	{pkgstats}	●	○					○	○				○									pkg. insights	ctags & gtags
[Lau22]	{Rclean}	●	●	○			○	○	○	○			●									debug/refactor	PDG traversal
[Rod21]	{rco}	●	●		●	○							●									optimization	AST visitor
[Pat19]	{rflowgraph}	●							○	○											○	call graph	AST visitor
[UT19]	{rstatic}	●	●		○	●		○					●								●	static analysis	AST visitor
[UT21]	{RTypeInference}	●	●		○	●		○					●		●						○	type inference	AST visitor
[BG20]	{SimilaR}	●	○			○			○				●									plagiarism	PDG, visitor
[Kal+14]	FastR	●	●		○			○	○	○			●									execute R	AST visitor
[R C23]	GNU R	●	●	○	●	○	○	●	○				●									execute R	bytecode
[19]	MRO	●	●	○	○	○																execute R	bytecode
[Nea14]	pqR	●	●	○	●	○	○	●	○				●									execute R	bytecode
[AS12]	Random	●	○	○	○	○				○			○									abstract int.	trace & visitor
[Gar04]	RCC	●	●		●	○	○		○			○	●									transpile to C	CFG, bytecode
[Ber13]	renjin	●	●	○	○	●		○	○			○	●		●							execute R	SSA, CFG
[Flü+20]	Ř	●	●	○	●	●	○	●	○				●	○	●							execute R	SSA, bytecode
[Sen+17]	ROSA	●	●			○			○		○		●		●	○						optimization	visitor
[Pos23]	RStudio	●	●	○					○	○			●									editor support	AST visitor

flowR

github.com/flowr-analysis/flowr

flowR

- A static analysis framework for R

github.com/flowr-analysis/flowr

flowR

- A static analysis framework for R
- Developed here, at Ulm University

github.com/flowr-analysis/flowr

flowR

- A static analysis framework for R
- Developed here, at Ulm University

github.com/flowr-analysis/flowr

flowR

- A static analysis framework for R
- Developed here, at Ulm University

  Florian Sihler, Julian Schubert, Oliver Gerstl, Lars Pfrenger, Johanna Scheck, Felix Schlegel, Ruben Dunkel, Thomas Schöller, Tim Schmidt, …

github.com/flowr-analysis/flowr

flowR

- A static analysis framework for R
- Developed here, at Ulm University

  Florian Sihler, Julian Schubert, Oliver Gerstl, Lars Pfrenger, Johanna Scheck, Felix Schlegel, Ruben Dunkel, Thomas Schöller, Tim Schmidt, …

github.com/flowr-analysis/flowr

flowR

- A static analysis framework for R
- Developed here, at Ulm University

  Florian Sihler, Julian Schubert, Oliver Gerstl, Lars Pfrenger, Johanna Scheck, Felix Schlegel, Ruben Dunkel, Thomas Schöller, Tim Schmidt, …
- Let's get back to R:

github.com/flowr-analysis/flowr

flowR

- A static analysis framework for R
- Developed here, at Ulm University

  Florian Sihler, Julian Schubert, Oliver Gerstl, Lars Pfrenger, Johanna Scheck, Felix Schlegel, Ruben Dunkel, Thomas Schöller, Tim Schmidt, …
- Let's get back to R:

```r
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```r
`if` <- function(...) 42
if(TRUE) print(3) # 42
```

```r
x <- 2
`<-` <- `*`
x <- 21 # 42
```

github.com/flowr-analysis/flowr

flowR

- A static analysis framework for R
- Developed here, at Ulm University

  Florian Sihler, Julian Schubert, Oliver Gerstl, Lars Pfrenger, Johanna Scheck, Felix Schlegel, Ruben Dunkel, Thomas Schöller, Tim Schmidt, …
- Let's get back to R:

```r
x <- 4
f <- function() x
body(f) <- quote(y)
y <- 42
f() # 42
```

```r
`if` <- function(...) 42
if(TRUE) print(3) # 42
```

```r
x <- 2
`<-` <- `*`
x <- 21 # 42
```

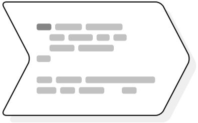
- We have to intertwine dataflow- and control-flow analysis…

github.com/flowr-analysis/flowr

flowR — Architecture

github.com/flowr-analysis/flowr/tree/main/src/dataflow

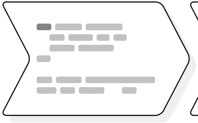
flowR — Architecture



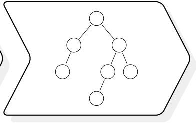
Parse

github.com/flowr-analysis/flowr/tree/main/src/dataflow

flowR — Architecture



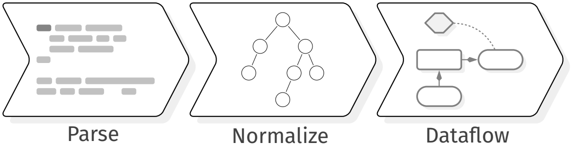
Parse



Normalize

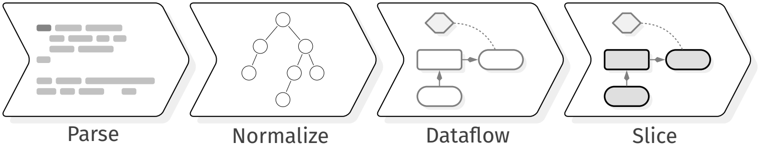
github.com/flowr-analysis/flowr/tree/main/src/dataflow

flowR — Architecture



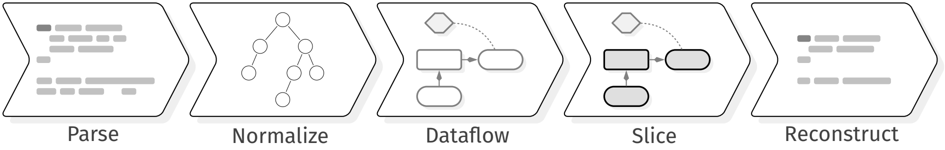
github.com/flowr-analysis/flowr/tree/main/src/dataflow

flowR — Architecture



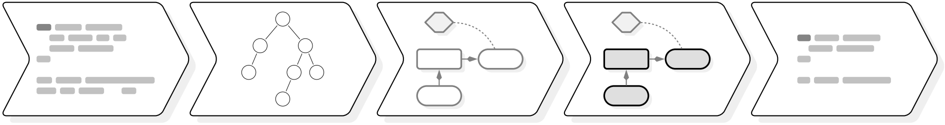
github.com/flowr-analysis/flowr/tree/main/src/dataflow

flowR — Architecture



github.com/flowr-analysis/flowr/tree/main/src/dataflow

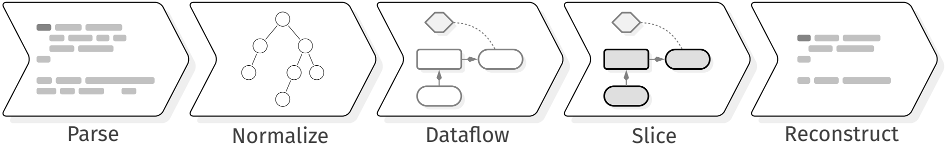
flowR — Architecture




```r
sum  <- 0
prod <- 1
n    <- 10

for (i in 1:(n-1)) {
   sum  <- sum + i
   prod <- prod * i
}

cat("Sum:", sum, "\n")
cat("Product:", prod, "\n")
```

github.com/flowr-analysis/flowr/tree/main/src/dataflow

flowR — Architecture

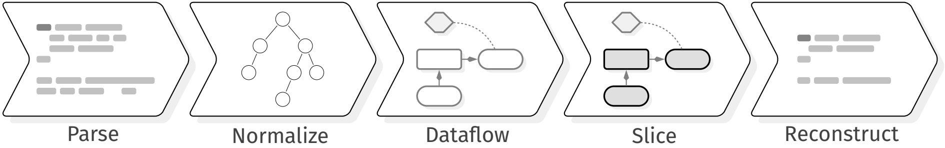



```r
sum  <- 0
prod <- 1
n    <- 10

for (i in 1:(n-1)) {
   sum  <- sum + i
   prod <- prod * i
}

cat("Sum:", sum, "\n")
cat("Product:", prod, "\n")
```

slice(10, **sum**)

github.com/flowr-analysis/flowr/tree/main/src/dataflow

flowR — Architecture




```
sum  <- 0
prod <- 1
n    <- 10

for (i in 1:(n-1)) {
   sum  <- sum + i
   prod <- prod * i
}

cat("Sum:", sum, "\n")
cat("Product:", prod, "\n")
```

slice(10, **sum**)

```
sum  <- 0
prod <- 1
n    <- 10

for (i in 1:(n-1)) {
   sum  <- sum + i
   prod <- prod * i
}

cat("Sum:", sum, "\n")
cat("Product:", prod, "\n")
```

github.com/flowr-analysis/flowr/tree/main/src/dataflow

flowr — Dataflow

Without Value tracking

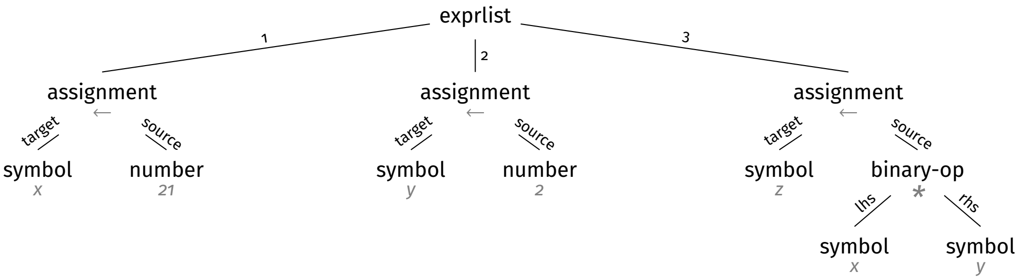
flowr — Dataflow

```
x₀ <- 21
y₀ <- 2
z₀ <- x₁ * y₁
```

Without Value tracking

flowr — Dataflow

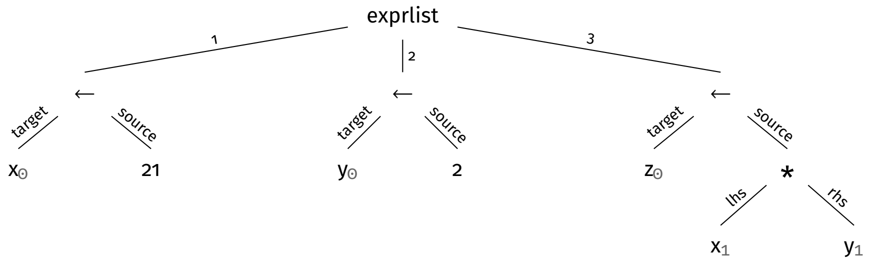
```
x₀ <- 21
y₀ <- 2
z₀ <- x₁ * y₁
```



Without Value tracking

flowr — Dataflow

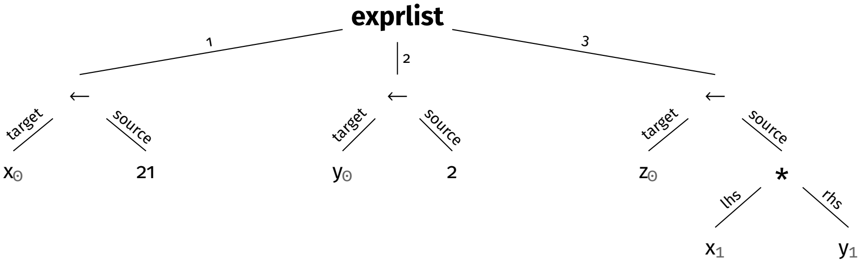
```
x⊙ <- 21
y⊙ <- 2
z⊙ <- x1 * y1
```



Without Value tracking

flowr — Dataflow

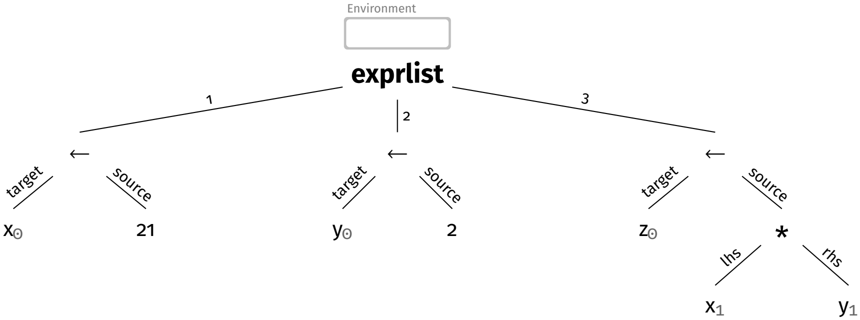
```
x₀ <- 21
y₀ <- 2
z₀ <- x₁ * y₁
```



Without Value tracking

flowr — Dataflow

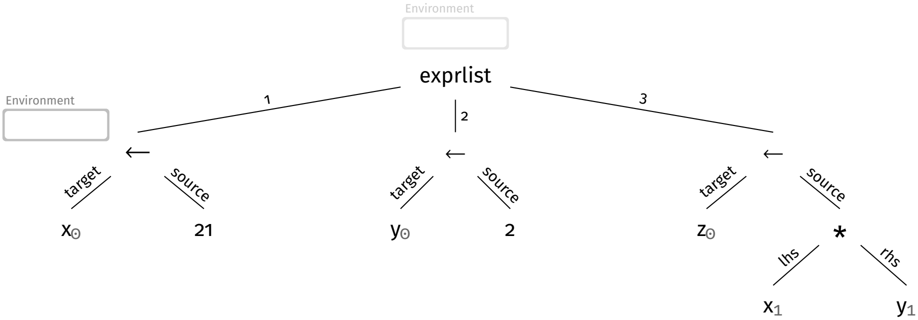
```
x₀ <- 21
y₀ <- 2
z₀ <- x₁ * y₁
```



Without Value tracking

flowr — Dataflow

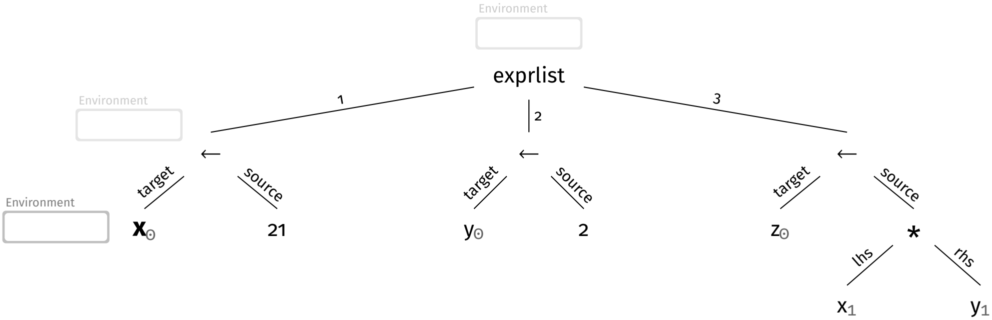
```
x₀ <- 21
y₀ <- 2
z₀ <- x₁ * y₁
```



Without Value tracking

flowr — Dataflow

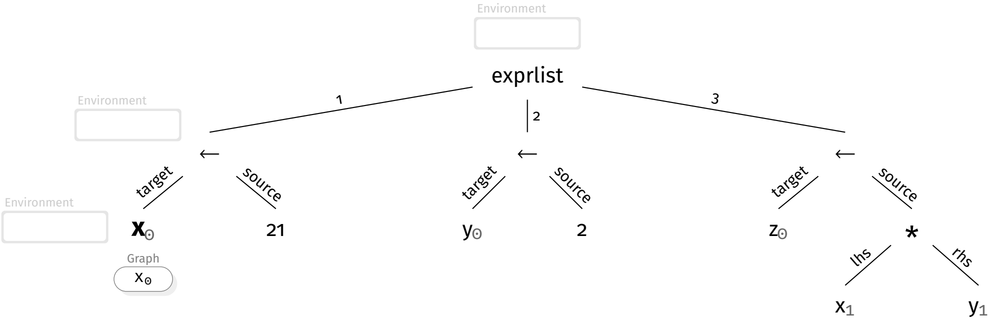
```
x₀ <- 21
y₀ <- 2
z₀ <- x₁ * y₁
```



Without Value tracking

flowr — Dataflow

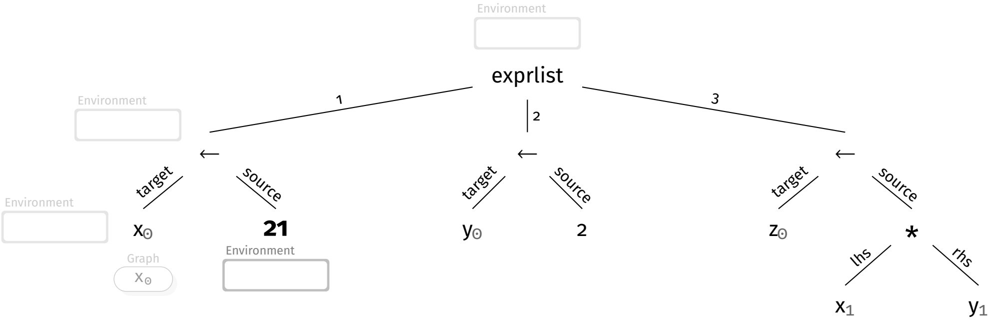
```
x₀ <- 21
y₀ <- 2
z₀ <- x₁ * y₁
```



Without Value tracking

flowr — Dataflow

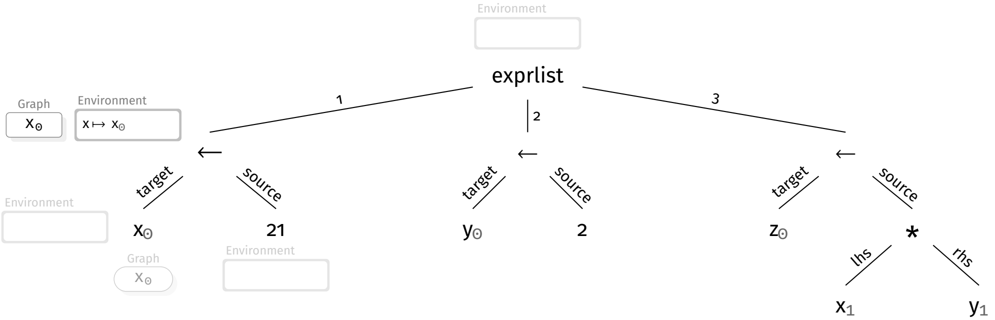
```
x⊙ <- 21
y⊙ <- 2
z⊙ <- x1 * y1
```



Without Value tracking

flowr — Dataflow

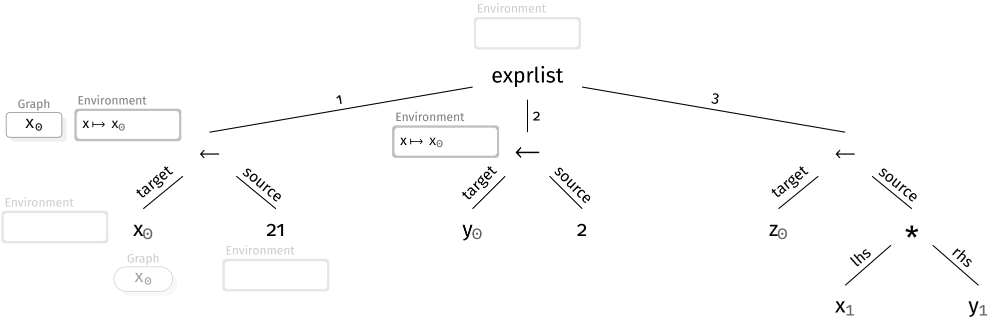
```
x⊙ <- 21
y⊙ <- 2
z⊙ <- x1 * y1
```



Without Value tracking

flowr — Dataflow

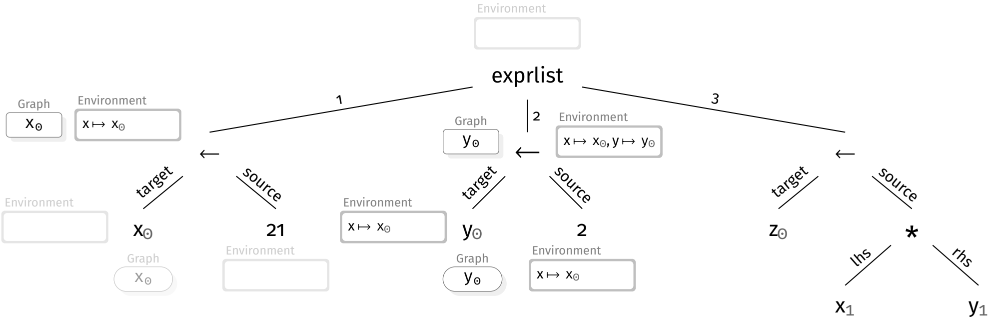
```
x₀ <- 21
y₀ <- 2
z₀ <- x₁ * y₁
```



Without Value tracking

flowr — Dataflow

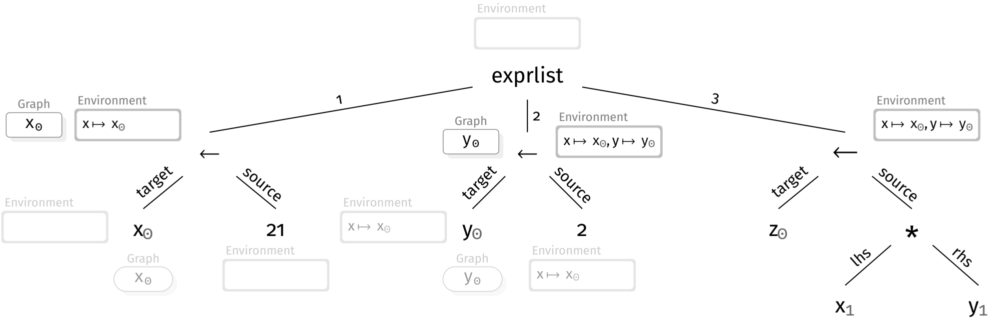
```
x⊙ <- 21
y⊙ <- 2
z⊙ <- x1 * y1
```



Without Value tracking

flowr — Dataflow

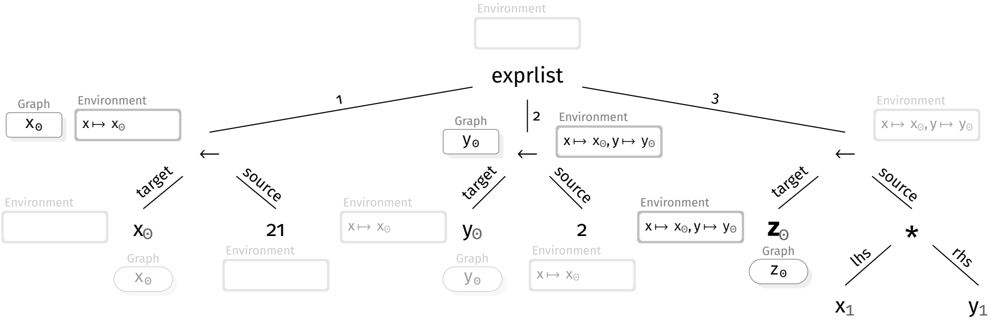
```
x⊙ <- 21
y⊙ <- 2
z⊙ <- x1 * y1
```



Without Value tracking

flowr — Dataflow

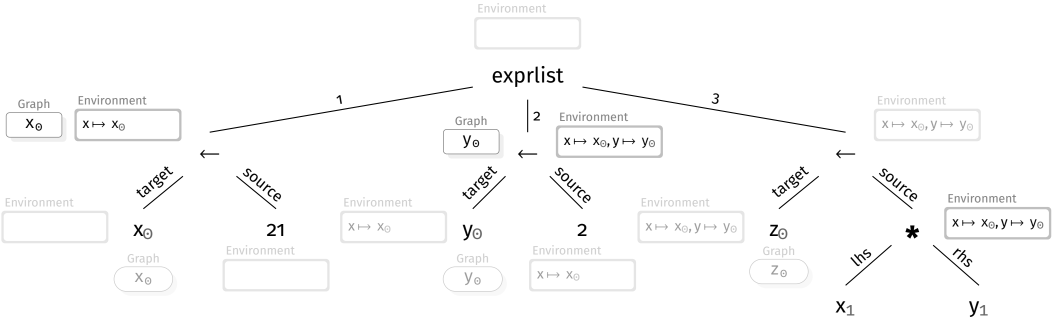
```
x⊙ <- 21
y⊙ <- 2
z⊙ <- x1 * y1
```



Without Value tracking

flowr — Dataflow

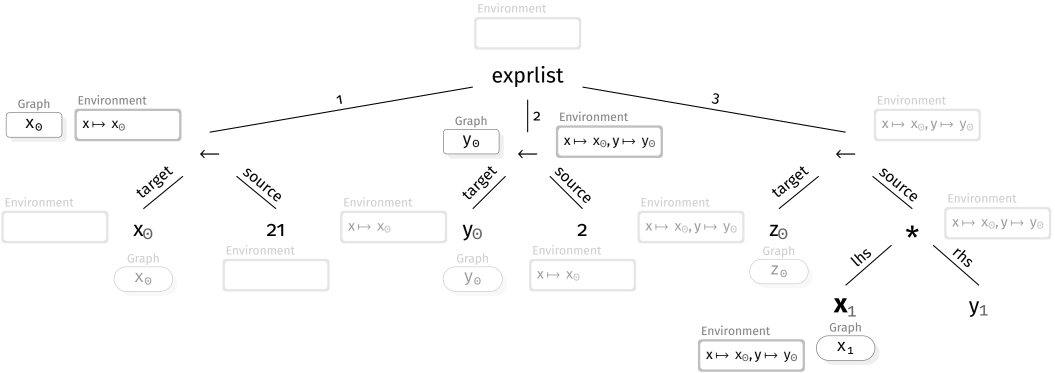
```
x⊙ <- 21
y⊙ <- 2
z⊙ <- x1 * y1
```



Without Value tracking

flowr — Dataflow

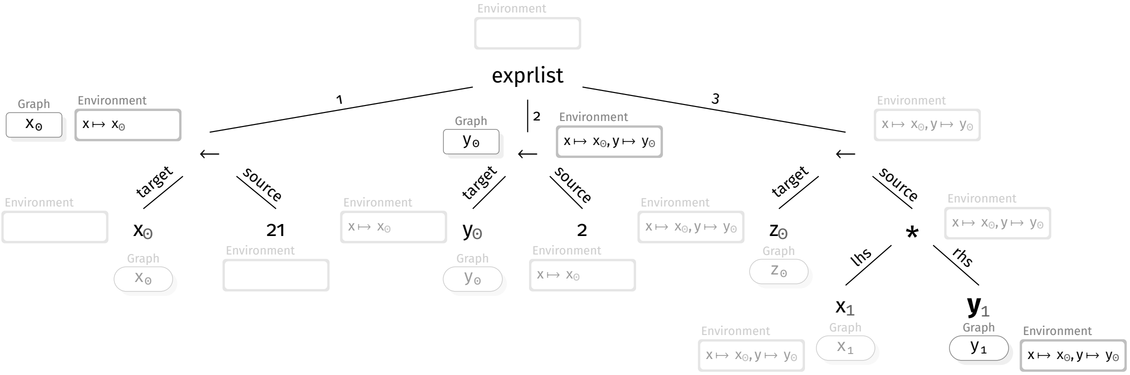
```
x⊙ <- 21
y⊙ <- 2
z⊙ <- x1 * y1
```



Without Value tracking

flowr — Dataflow

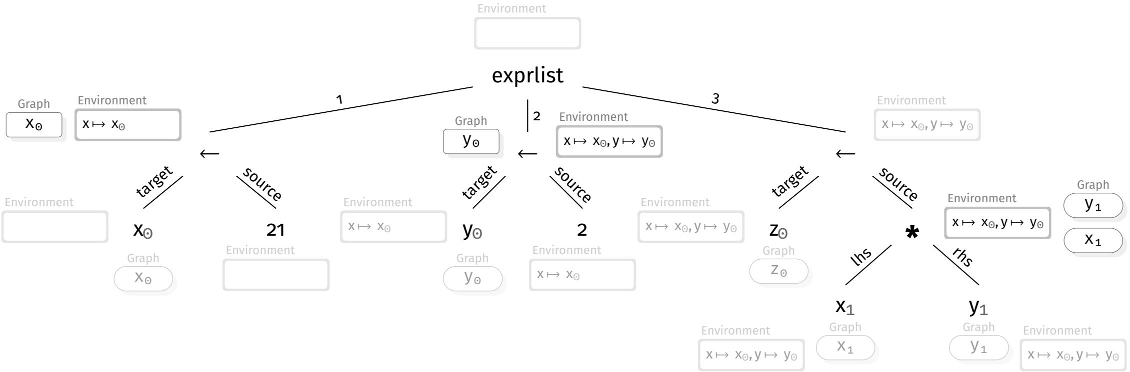
```
x⊙ <- 21
y⊙ <- 2
z⊙ <- x1 * y1
```



Without Value tracking

flowr — Dataflow

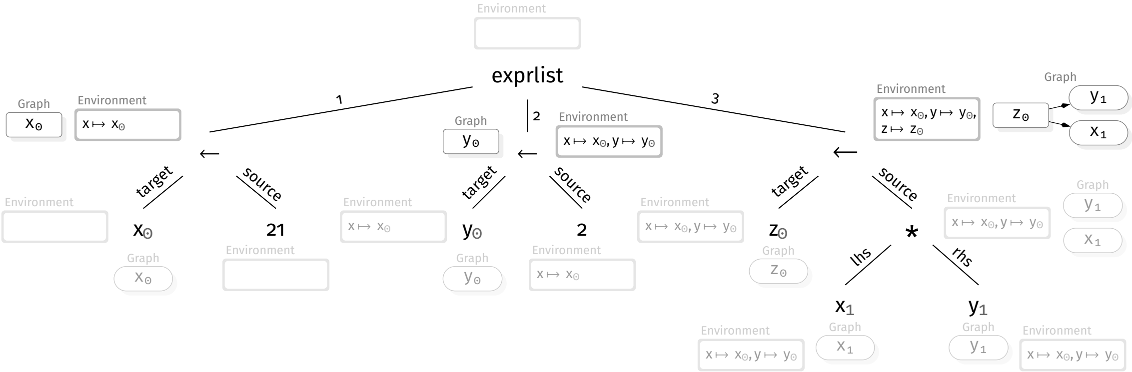
```
x₀ <- 21
y₀ <- 2
z₀ <- x₁ * y₁
```



Without Value tracking

flowr — Dataflow

$x_\odot$ <- *21*
$y_\odot$ <- *2*
$z_\odot$ <- x_1 * y_1



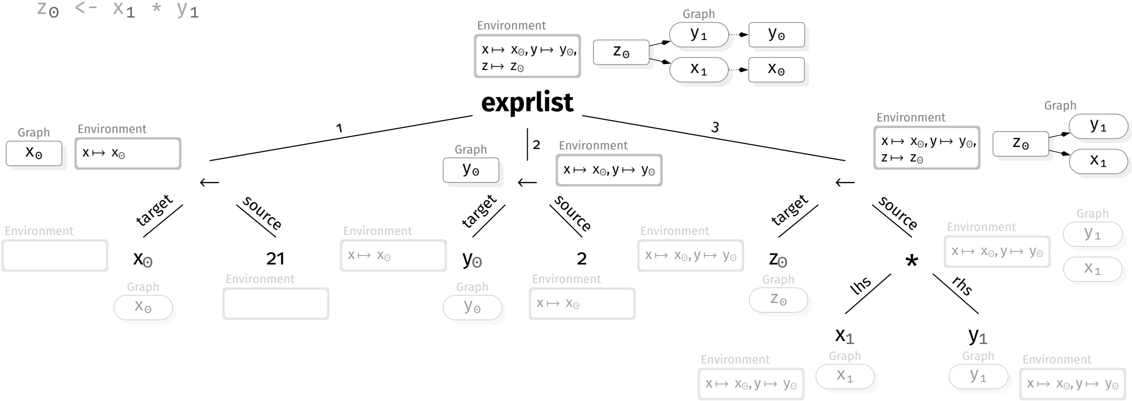
Without Value tracking

flowr — Dataflow

$x_{\odot}$ <- 21

$y_{\odot}$ <- 2

$z_{\odot}$ <- x_{1} * y_{1}



Without Value tracking

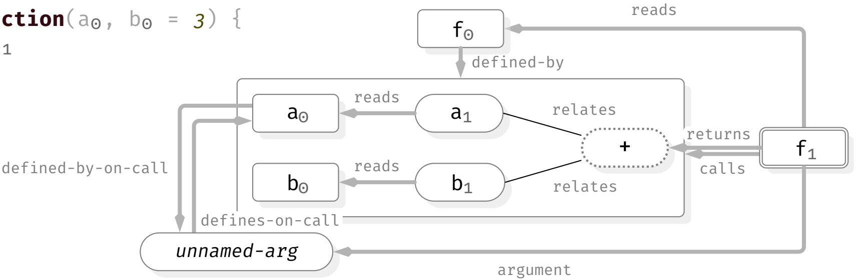
flowR — There Is More…

flowR — There Is More…

```
f₀ <- function(a₀, b₀ = 3) {
   a₁ + b₁
}
f₁(39)
```

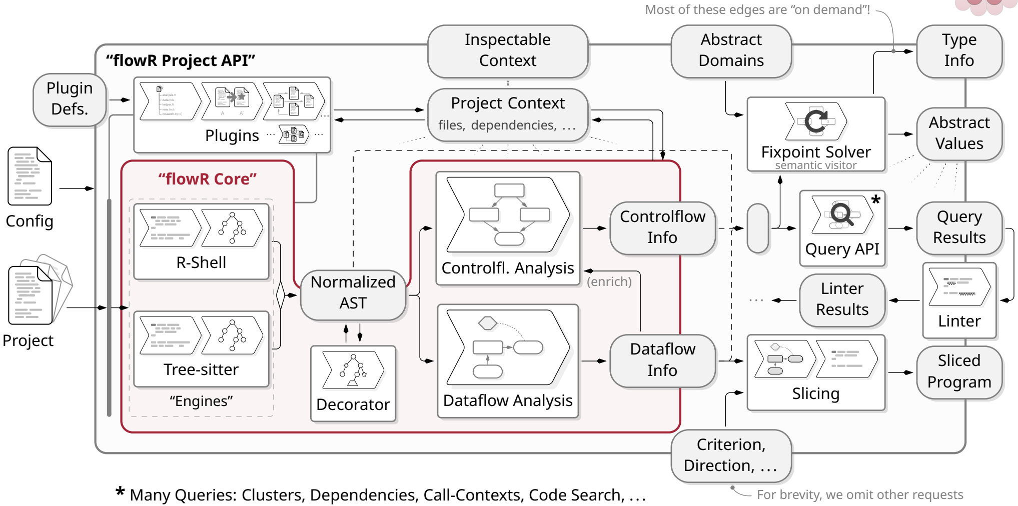
flowR — There Is More…

```
f₀ <- function(a₀, b₀ = 3) {
   a₁ + b₁
}
f₁(39)
```



flowR — A LOT More…

flowR — A LOT More...



* Many Queries: Clusters, Dependencies, Call-Contexts, Code Search, ...

4. Conclusion

Soundness and Completeness Revisited

Soundness and Completeness Revisited

- We want to prove properties of programs (e.g., no overflow, shapes, . . .)

Soundness and Completeness Revisited

- We want to prove properties of programs (e.g., no overflow, shapes, ...)
- However, thanks to Rice [Ric53] we know:

Soundness and Completeness Revisited

- We want to prove properties of programs (e.g., no overflow, shapes, ...)
- However, thanks to Rice [Ric53] we know:

  Rice's theorem states that all nontrivial semantic properties of programs are undecidable. [Cou21, p. 100]

Soundness and Completeness Revisited

- We want to prove properties of programs (e.g., no overflow, shapes, ...)
- However, thanks to Rice [Ric53] we know:

  Rice's theorem states that all nontrivial semantic properties of programs are undecidable. [Cou21, p. 100]

Soundness and Completeness Revisited

- We want to prove properties of programs (e.g., no overflow, shapes, …)
- However, thanks to Rice [Ric53] we know:

  Rice's theorem states that all nontrivial semantic properties of programs are undecidable. [Cou21, p. 100]

Soundness

Completeness

Soundness and Completeness Revisited

- We want to prove properties of programs (e.g., no overflow, shapes, ...)
- However, thanks to Rice [Ric53] we know:

  Rice's theorem states that all nontrivial semantic properties of programs are undecidable. [Cou21, p. 100]

Soundness

- All properties we derive are true (but we may miss some)
- If we report bugs for violated properties, we produce no false negative

Completeness

Soundness and Completeness Revisited

- We want to prove properties of programs (e.g., no overflow, shapes, ...)
- However, thanks to Rice [Ric53] we know:

  Rice's theorem states that all nontrivial semantic properties of programs are undecidable. [Cou21, p. 100]

Soundness

- All properties we derive are true (but we may miss some)
- If we report bugs for violated properties, we produce no false negative

Completeness

- We are able to infer all interesting properties in the program
- If we report bugs for violated properties, we produce no false positive

Handling Errors

Handling Errors

- Analyzers tend to **favor soundyness over completeness**

Handling Errors

- Analyzers tend to **favor soundyness over completeness**
  Better to report a false positive than to miss a bug

Handling Errors

- Analyzers tend to **favor soundyness over completeness**
  Better to report a false positive than to miss a bug
- Yet many cases are hard to handle (e.g., `eval` with *any* possible effect)

Handling Errors

- Analyzers tend to **favor soundyness over completeness**
  Better to report a false positive than to miss a bug
- Yet many cases are hard to handle (e.g., `eval` with *any* possible effect)
- Issues are usually ranked by severity (and sometimes confidence)

Handling Errors

- Analyzers tend to **favor soundyness over completeness**
  Better to report a false positive than to miss a bug
- Yet many cases are hard to handle (e.g., `eval` with *any* possible effect)
- Issues are usually ranked by severity (and sometimes confidence)
- Cutoffs may be applied if there are too many alarms

Handling Errors

- Analyzers tend to **favor soundyness over completeness**
  Better to report a false positive than to miss a bug
- Yet many cases are hard to handle (e.g., `eval` with *any* possible effect)
- Issues are usually ranked by severity (and sometimes confidence)
- Cutoffs may be applied if there are too many alarms
- These is still a lot of research required [EN08; Cou21, p. 705]

Oh There Are so Many Tools…

Oh There Are so Many Tools…

SootUp
flowR
LiquidHaskell
AbsInt
lintr
Teamscale
Infer
LiSA
CodeQL
sonar
Clang
GCC
ESLint
MOPSA analyzer
frama C

Oh There Are so Many Tools…

LiquidHaskell Verify properties at compile time

Oh There Are so Many Tools...

SootUp

flowR

LiquidHaskell

AbsInt

lintr

Teamscale

Infer

LiSA

CodeQL

sonar

Clang

GCC

ESLint

MOPSA analyzer

frama C

Oh There Are so Many Tools...

Use inference rules for linting ⊢ Infer

Oh There Are so Many Tools…

Clang

Oh There Are so Many Tools…

Determine registers, scheduling, types …
For example:

$$\frac{\Gamma \vdash e_1 : \texttt{int} \quad \Gamma \vdash e_2 : \texttt{int}}{\Gamma \vdash e_1 + e_2 : \texttt{int}}$$

Clang

And There Are Many More Strategies

And There Are Many More Strategies

- Static Analysis

And There Are Many More Strategies

- Static Analysis
  analyzing the code without executing it, for all possible runtime scenarios

And There Are Many More Strategies

- Static Analysis
  analyzing the code without executing it, for all possible runtime scenarios
- Dynamic Analysis

And There Are Many More Strategies

- Static Analysis
  analyzing the code without executing it, for all possible runtime scenarios
- Dynamic Analysis
  executing the program with specific input(s) to observe its behavior

And There Are Many More Strategies

- Static Analysis
  analyzing the code without executing it, for all possible runtime scenarios
- Dynamic Analysis
  executing the program with specific input(s) to observe its behavior
- Hybrid Analysis

And There Are Many More Strategies

- Static Analysis
  analyzing the code without executing it, for all possible runtime scenarios
- Dynamic Analysis
  executing the program with specific input(s) to observe its behavior
- Hybrid Analysis
  combining both strategies

And There Are Many More Strategies

- Static Analysis
  analyzing the code without executing it, for all possible runtime scenarios
- Dynamic Analysis
  executing the program with specific input(s) to observe its behavior
- Hybrid Analysis
  combining both strategies

Program analysis is important. Additionally AND combined with testing.

And There Are Many More Strategies

- Static Analysis
  analyzing the code without executing it, for all possible runtime scenarios
- Dynamic Analysis
  executing the program with specific input(s) to observe its behavior
- Hybrid Analysis
  combining both strategies

Program analysis is important. Additionally AND combined with testing.

Back to the Questions

Back to the Questions

1. How would you capture what a *property* is? ✔
2. How would you phrase that one property is “better” than another? ✔
3. For what operations would you *not* use a control-flow graph?
4. Why can’t there be a fully automatic, sound, and complete static analyzer for general programs?
5. What (big) additional challenges do you see in the real-world?

Back to the Questions

1. How would you capture what a *property* is? ✔
2. How would you phrase that one property is “better” than another? ✔
3. For what operations would you *not* use a control-flow graph? ✔
4. Why can’t there be a fully automatic, sound, and complete static analyzer for general programs?
5. What (big) additional challenges do you see in the real-world?

Back to the Questions

1. How would you capture what a *property* is? ✔
2. How would you phrase that one property is “better” than another? ✔
3. For what operations would you *not* use a control-flow graph? ✔
4. Why can’t there be a fully automatic, sound, and complete static analyzer for general programs? ✔
5. What (big) additional challenges do you see in the real-world?

Back to the Questions

1. How would you capture what a *property* is? ✔
2. How would you phrase that one property is "better" than another? ✔
3. For what operations would you *not* use a control-flow graph? ✔
4. Why can't there be a fully automatic, sound, and complete static analyzer for general programs? ✔
5. What (big) additional challenges do you see in the real-world? ✔

Back to the Questions

1. How would you capture what a *property* is? ✔
2. How would you phrase that one property is “better” than another? ✔
3. For what operations would you *not* use a control-flow graph? ✔
4. Why can’t there be a fully automatic, sound, and complete static analyzer for general programs? ✔
5. What (big) additional challenges do you see in the real-world? ✔
6. How would you describe soundness and completeness?

Back to the Questions

1. How would you capture what a *property* is? ✔
2. How would you phrase that one property is “better” than another? ✔
3. For what operations would you *not* use a control-flow graph? ✔
4. Why can’t there be a fully automatic, sound, and complete static analyzer for general programs? ✔
5. What (big) additional challenges do you see in the real-world? ✔
6. How would you describe soundness and completeness? ✔

Back to the Questions

1. How would you capture what a *property* is? ✔
2. How would you phrase that one property is “better” than another? ✔
3. For what operations would you *not* use a control-flow graph? ✔
4. Why can’t there be a fully automatic, sound, and complete static analyzer for general programs? ✔
5. What (big) additional challenges do you see in the real-world? ✔
6. How would you describe soundness and completeness? ✔
7. How would you describe Abstraction Interpretation?

Back to the Questions

1. How would you capture what a *property* is? ✔
2. How would you phrase that one property is “better” than another? ✔
3. For what operations would you *not* use a control-flow graph? ✔
4. Why can’t there be a fully automatic, sound, and complete static analyzer for general programs? ✔
5. What (big) additional challenges do you see in the real-world? ✔
6. How would you describe soundness and completeness? ✔
7. How would you describe Abstraction Interpretation? ✔

Thanksies!

slides and source code

feedback form

References I

[19] *Microsoft R Open Source*. 2019. URL: https://github.com/microsoft/microsoft-r-open.

[App24] Lewin Appleton-Fox. *foodwebr: Visualise Function Dependencies*. 2024. URL: https://github.com/lewinfox/foodwebr.

[AS12] Gianluca Amato and Francesca Scozzari. “Random: R-Based Analyzer for Numerical Domains”. In: *Logic for Programming, Artificial Intelligence, and Reasoning - 18th International Conference, LPAR-18, Mérida, Venezuela, March 11-15, 2012. Proceedings*. Ed. by Nikolaj S. Bjørner and Andrei Voronkov. Vol. 7180. Lecture Notes in Computer Science. Springer, 2012, pp. 375–382. DOI: 10.1007/978-3-642-28717-6_29. URL: https://doi.org/10.1007/978-3-642-28717-6%5C_29.

[Ben22] Henrik Bengtsson. *globals: Identify Global Objects in R Expressions*. R package version 0.16.2. 2022. URL: https://CRAN.R-project.org/package=globals.

[Ber13] Alexander Bertram. “Renjin: A new r interpreter built on the jvm”. In: *The r user conference, useR! 2013 july 10-12 2013 university of castilla-la mancha, albacete, spain*. Vol. 10. 30. 2013, p. 105.

[BG20] Maciej Bartoszuk and Marek Gagolewski. *SimilaR: R Source Code Similarity Evaluation*. R package version 1.0.8. 2020. URL: https://CRAN.R-project.org/package=SimilaR.

[Bra18] Mark V. Bravington. *mvbutils: Workspace Organization, Code and Documentation Editing, Package Prep and Editing, Etc.* R package version 2.8.232. 2018. URL: https://CRAN.R-project.org/package=mvbutils.

[Cha23] Joris Chau. *checkglobals: Static Analysis of R-Code Dependencies*. R package version 0.1.0. 2023. URL: https://CRAN.R-project.org/package=checkglobals.

[Cou21] Patrick Cousot. “Principles of Abstract Interpretation”. In: (2021).

[Csa23] Gabor Csardi. *cyclocomp: Cyclomatic Complexity of R Code*. R package version 1.1.1. 2023. URL: https://CRAN.R-project.org/package=cyclocomp.

[EN08] Pär Emanuelsson and Ulf Nilsson. “A Comparative Study of Industrial Static Analysis Tools”. In: *Proceedings of the 3rd International Workshop on Systems Software Verification, SSV 2008, Sydney, Australia, February 25-27, 2008*. Ed. by Ralf Huuck, Gerwin Klein, and Bastian Schlich. Vol. 217. Electronic Notes in Theoretical Computer Science. Elsevier, 2008, pp. 5–21. DOI: 10.1016/J.ENTCS.2008.06.039. URL: https://doi.org/10.1016/j.entcs.2008.06.039.

[Fab23] Antoine Fabri. *flow: View and Browse Code Using Flow Diagrams*. R package version 0.2.0. 2023. URL: https://CRAN.R-project.org/package=flow.

[Flü+19] Olivier Flückiger et al. “R melts brains: an IR for first-class environments and lazy effectful arguments”. In: *Proceedings of the 15th ACM SIGPLAN International Symposium on Dynamic Languages, DLS 2019, Athens, Greece, October 20, 2019*. Ed. by Stefan Marr and Juan Fumero. ACM, 2019, pp. 55–66. DOI: 10.1145/3359619.3359744. URL: https://doi.org/10.1145/3359619.3359744.

References II

[Flü+20] Olivier Flückiger et al. "Sampling optimized code for type feedback". In: *Proceedings of the 16th ACM SIGPLAN International Symposium on Dynamic Languages*. 2020, pp. 99–111.

[Gar04] John Garvin. *RCC: A compiler for the R language for statistical computing*. Rice University, 2004.

[Hes+23] Jim Hester et al. *lintr: A 'Linter' for R Code*. R package version 3.1.0. 2023. URL: https://CRAN.R-project.org/package=lintr.

[Kal+14] Tomas Kalibera et al. "A fast abstract syntax tree interpreter for R". In: *ACM SIGPLAN Notices* 49.7 (2014), pp. 89–102.

[Kar21] Dan Kary. *dfgraph: Visualize R Code with Data Flow Graphs*. R package version 3.1.0. 2021. URL: https://github.com/dkary/dfgraph.

[KSK09] Uday Khedker, Amitabha Sanyal, and Bageshri Karkare. *Data Flow Analysis: Theory and Practice*. 1st. USA: CRC Press, Inc., 2009. ISBN: 0849328802.

[Lai23] Randy Lai. *languageserver: Language Server Protocol*. R package version 0.3.16. 2023. URL: https://CRAN.R-project.org/package=languageserver.

[Lan+18] Duncan Lang et al. *CodeDepends. Analysis of R Code for Reproducible Research and Code Comprehension*. manual. 2018. URL: https://CRAN.R-project.org/package=CodeDepends.

[Lan+23] Duncan Lang et al. *CodeAnalysis. Tools for static analysis of R code*. manual. 2023. URL: https://github.com/duncantl/CodeAnalysis.

[Lau22] Matthew Lau. *Rclean: A Tool for Writing Cleaner, More Transparent Code*. R package version 1.1.8. 2022. URL: https://github.com/MKLau/Rclean.

[Nea14] Radford M Neal. "Speed Improvements in pqR: Current Status and Future Plans". In: (2014). URL: https://glizen.com/radfordneal/ftp/pqR-dsc.pdf.

[Pad21] Mark Padgham. *pkgstats*. R package version 0.0.3. 2021. URL: https://github.com/ropensci-review-tools/pkgstats.

[Pat19] Evan Patterson. *flowgraph: Flow graps for R*. 2019. URL: https://github.com/IBM/rflowgraph.

[Pos23] Posit team. *RStudio: Integrated Development Environment for R*. Posit Software, PBC. Boston, MA, 2023. URL: http://www.posit.co/.

[R C23] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. Vienna, Austria, 2023. URL: https://www.R-project.org/.

[Ric53] Henry Gordon Rice. "Classes of recursively enumerable sets and their decision problems". In: *Transactions of the American Mathematical society* 74.2 (1953), pp. 358–366.

[Rob16] Titouan Robert. *DependenciesGraphs: Dependencies visualization between functions and environments*. R package version 0.3. 2016.

References III

[Rod21] Juan Cruz Rodriguez. *rco: The R Code Optimizer*. R package version 1.0.2. 2021. URL: https://CRAN.R-project.org/package=rco.

[RY20] Xavier Rival and Kwangkeun Yi. "Introduction to Static Analysis: An Abstract Interpretation Perspective". In: (2020).

[Sen+17] Rathijit Sen et al. *ROSA: R Optimizations with Static Analysis*. 2017. arXiv: 1704.02996 [cs.PL].

[Tie23] Luke Tierney. *codetools: Code Analysis Tools for R*. R package version 0.2-19. 2023. URL: https://CRAN.R-project.org/package=codetools.

[UT19] Nick Ulle and Duncan Temple Lang. *rstatic: Low-level Static Analysis Tools for R Code*. R package version 0.1.0-0006. 2019.

[UT21] Nick Ulle and Duncan Temple Lang. *RTypeInference: Infer Types of Inputs and Outputs for R Expressions*. R package version 0.5.0-0030. 2021.

[van23] Maarten van Kessel. *PaRe: A Way to Perform Code Review or QA on Other Packages*. R package version 0.1.12. 2023. URL: https://github.com/darwin-eu-dev/PaRe.